

赤外線マスク同期を用いた 複数 Arduino による輪唱演奏システム

情報工学科 2 年

25G1046

櫛濱 和輝

—— チーム 9 ——

久家 力孔 (24G1052) 手代木 巧 (24G1098)

渡邊 乃斗 (24G1145) 家田 祐希 (25G1010)

木下 遼介 (25G1043)

2026 年 7 月 30 日

1 はじめに

近年、IoT 技術の普及によって、複数のデバイスがネットワークを介してリアルタイムに連携するシステムが様々な分野で実用化されている [1]. IoT 技術やプログラミング教育の重要性が高まるなか [2], STEAM 教育やものづくりを通じた実践的学習の有効性が示されており [3, 4], 大学教育においても PBL やハッカソン形式の授業が学生の主体性向上に寄与することが報告されている [5, 6, 7]. IoT を題材としたチーム開発型 PBL の効果も実証されており [8], ハッカソンを人材育成に活用する事例 [9] やオンラインでの実践報告 [10] も増えている. さらに, IoT ロボットを用いた協調学習 [11] やプログラミング教育を通じた多世代間の教え合いの可能性 [12] も示されている. 製造業における機械の協調制御やスマートホームの機器連携など, 複数の自律した機器を同期させる技術は現代の組み込みシステムに広く求められている. 合奏はこの同期という課題が音として直接聞こえる題材である. Rasch によると, 合奏において奏者間の発音タイミングのずれはおよそ 30~50 ms の範囲に収まっており, これを大きく超えるずれは聴き手に同期の乱れとして知覚される [13]. つまり, 複数の機器で合奏を成立させるには, 数十 ms 以内の精度で状態を共有し続ける必要がある.

電子楽器の同期に関する既存技術としては, MIDI や Wi-Fi・Bluetooth を用いたシステムが挙げられる. MIDI は音高やタイミング, 強さといった精密な情報を伝達できる規格である [14] が, 専用のインターフェースと対応ライブラリが必要であり, 同期のきっかけだけを伝えたい本プロジェクトの用途には過剰な仕様となる. Wi-Fi や Bluetooth を用いた合奏システムは遅延補正の技術も発達しているものの, 通信インフラの品質に動作が左右されるため, 多数の機器が電波を発する実験室環境では干渉や接続不安定のリスクが残る. そこで本プロジェクトでは, 伝達する情報を「どの楽器がいま演奏に参加しているか」という最小限の同期情報に限定し, 赤外線通信で実現する方針を採った. 赤外線はネットワーク環境を必要とせず, LED と受光モジュールだけで構成できるため, 小規模な組み込みシステムに低コストで導入できる [15]. 各方式の比較と選定の根拠は 2 章で詳述する.

本プロジェクトでは, 6 台の Arduino を用いたオーケストラシステム「赤外線通信で歌うカエル」を制作した. 1 台を指揮側の親機, 4 台をピアノ・フルート・トランペット・ドラムを担当する演奏側の子機, 残る 1 台を LCD 表示用の子機とし, 楽曲「かえるのうた」を輪唱形式で演奏する. 親機は演奏中の楽器集合を表すマスクデータを赤外線ですべての子機へ一斉送信し, 各演奏子機は自分の担当ビットが有効なときにのみ譜面を 1 コマ進め, PC 上の Processing へ音符情報を送って楽器音を再生させる. 演奏者はボタン操作で各楽器を任意のタイミングで参加・退場させることができ, 可変抵抗によるテンポの連続調整にも対応する. LCD 表示子機は同じ赤外線信号を受け取り, かえるの口パクアニメーションと歌詞を LCD に表示する.

報告者は本システムのうち, ピアノ音源の合成と Processing 側の再生処理, ピアノ担当子機と Arduino との通信用関数, ドラム担当子機のプログラム, ブレッドボードの設計, およびテンポ管理プログラムの実装を担当した. テンポ管理は渡邊との共同担当である. また, 開発終盤には親機

と子機で楽譜 1 周分の tick 数が食い違い特定の音が周期的に抜けるバグの修正と、休符が休符として聞こえない問題の修正を行った。

評価では、単体テストで各楽器音色と通信機能を確認したのち、第 9 週末に親機 1 台と演奏子機 4 台による輪唱の実機テストを実施し、ボタン押下順に楽器が参加するカノン演奏が音抜けや休符抜けなく成立すること、および可変抵抗によるテンポ変更により演奏が追従することを確認した。同期性能については、親機の送信時刻と子機の受信時刻の差をマイクロ秒単位で記録する機構を実装し、10,323 tick 分のログから演奏タイミング誤差と赤外線通信の受信成功率を定量的に評価した。楽器音の音色は 18 人を対象とした 5 段階のアンケートで評価し、LCD 表示と演奏の同期は演奏の動画から発音と表示のフレーム差を求めることで評価した。最終発表では、表示子機を含む全構成を接続したシステムテストを実施し、演奏・参加操作・テンポ変更・LCD 表示のすべてが正常に動作することを確認している。本報告書では、システム全体の構成と報告者の担当部分の実装を述べたうえで、評価結果とその原因分析、および残された課題について考察する。

2 関連技術と技術選定

本章では、本システムの中核である機器間同期と楽器音合成について、既存技術を調査した結果と、そこから本プロジェクトの構成を選定した根拠を述べる。

2.1 機器間同期方式の比較

複数の演奏機器を同期させる方式の候補として、MIDI, Wi-Fi, Bluetooth Low Energy (BLE), および赤外線リモコン通信の 4 つを比較した。比較の観点は、本プロジェクトの要件である「1 対多の一斉送信ができること」「機器間の発音タイミング差を 30 ms 以内に保てること」「実験室環境で安定して動作すること」「限られた期間と予算で実装できること」の 4 点である。比較結果を表 1 に示す。

MIDI は演奏情報を精密に伝達できる規格であるが、本システムが伝えたい情報は「どの楽器がいま演奏に参加しているか」を表す 4 ビットのマスクだけであり、規格の大部分を使わないまま専用インターフェースの実装コストだけを負うことになる。

Wi-Fi と BLE は任意のデータを送れる汎用性を持つが、本システムの用途には 3 つの難点がある。第 1 に、本システムは 1 台の親機から 5 台の子機へ同じ情報を同時に届ける構成であり、1 対多の送信が中心となる。しかし Wi-Fi ではブロードキャスト送信が網全体の通信量と各機器の処理負荷を増やすため避けるべきとされており [17], 1 対多という要件そのものが不利に働く。第 2 に、接続の確立に時間がかかる。BLE では機器の発見だけで 1~30 s を要するという報告があり [19], 6 台を同時に立ち上げる運用では演奏開始までの時間が読めない。第 3 に、ジッタが大きい。McPherson らはマイコンと無線を組み合わせた楽器システムの遅延を実測し、試したすべての構成でジッタが 1 ms を超え、Wi-Fi 経由では 5.7~57 ms の範囲に広がることを報告している [20]。7.1 節で述べるとおり合奏の同期で問題になるのは遅延の平均値ではなくそのばらつきであるから、この点は決定的である。

表 1 機器間同期方式の比較と本プロジェクトへの適合性

観点	MIDI[14]	Wi-Fi	BLE	赤外線 [15, 16]
伝送できる情報	演奏情報全般	任意 (TCP/UDP)	任意 (GATT)	数バイトのコマンド
1 対多の一斉送信	デジタルチェーン接続が必要	可能だが、網と各機器の負荷が増すため非推奨とされる [17]	アダプタイズで可能	物理層が本質的にブロードキャスト [18]
遅延の報告例	—	往復 5.30 ms (理想), 8.09 ms (負荷時) [17]	0.6–10 ms (誤り率が良好な場合) [19]	本システムで片道 28.1 ms
ジッタの報告例	—	全構成で 1 ms 超, Wi-Fi 経由で 5.7–57 ms [20]	接続間隔が 7.5 ms 刻みに量子化 [21]	本システムで 0.02 ms
接続の確立	不要	アクセスポイントへの接続が必要	機器の発見に 1–30 s [19]	不要
追加部品	MIDI シールド等	Wi-Fi モジュール	BLE モジュール	LED と受光モジュール
本用途への適合	機能過剰	1 対多が不利	接続確立が遅い	適合

一方、赤外線リモコン通信は本来が 1 対多のブロードキャストを前提とした方式であり、送信側は赤外線 LED、受信側は受光モジュールのみで構成できる。Rubenstein らは多数の小型ロボットを赤外線で制御するシステムにおいて、上方に置いた 1 台の赤外線送信機から全ロボットへ、台数によらず一定の時間でプログラムを配信できることを示している [18]。1 対多の送信に追加のコストがかからないという性質は、本システムが求めるものと一致する。また、赤外線を共通の同期信号として複数の機器を揃える手法は近年も研究されており、Meyer らは赤外 LED による光学的なブロードキャストで 25 台以上の異種カメラを 1.34 ms (RMSE) の精度で同期させている [22]。赤外線が数十 ms を大きく下回る同期精度を達成しうることを示す例である。伝送できる情報量は数バイトと少ないが、本システムが必要とする情報量は 4 ビットであるから制約にならない。一方で、赤外線には固有の弱点もある。可視光と同じく直進する光であるため、送受信経路に遮蔽物があると届かない。また太陽光や蛍光灯に含まれる赤外成分が雑音となり、受信を妨げうる。さらに、リモコン用の伝送フォーマットは数バイトのコマンドを送ることを前提としており、まとまったデータの転送には向かない。本システムはこれらの制約をいずれも許容できると判断した。使用環境は屋内の限られた空間であり、送受信の距離も数メートル以内である。伝送するデータは 4 ビットのマスクのみでフォーマットの制約を受けない。遮蔽については、実機での動作確認の際に、室内に複数人がいて送受信経路上に人体や機材が部分的に入る状態でも受信できることを確認している。以上より、要件に対して過不足のない赤外線通信を採用した。なお、屋外や強い照明下での動作は本プロジェクトでは検証しておらず、残された課題である (7.14 節)。

2.2 赤外線リモコンプロトコルの選定

赤外線リモコンの伝送フォーマットとして広く用いられているのは、NEC フォーマット [15] と Sony SIRC [16] である。この 2 つは、赤外線通信の実験用プラットフォームを扱った研究において

表 2 赤外線フォーマットのフレーム長比較 (仕様値からの算出)

フォーマット	フレーム長	エラー検出	目標 30 ms との関係
NEC (32 ビット固定)	68.1 ms	反転バイトによる検出あり	単独で目標を超過
Sony SIRC 20 ビット	27.0 ms	なし	余裕がない
Sony SIRC 15 ビット	21.0 ms	なし	余裕が小さい
Sony SIRC 12 ビット	18.0 ms	なし	採用

も、対応すべき標準フォーマットの代表として挙げられている [23]。両者は 1 フレームの送信に要する時間が大きく異なり、この差が本システムの同期性能に直接影響する。そこで、両フォーマットのフレーム長を仕様から算出して比較した。

Sony SIRC は単位時間 $T_S = 600 \mu\text{s}$ を基準とし、ヘッダをマーク $4T_S$ とスペース T_S 、ビット 0 をマーク T_S とスペース T_S 、ビット 1 をマーク $2T_S$ とスペース T_S で表現する [16]。12 ビットフォーマットにおいて値 1 のビット数を k とすると、1 フレームの送信時間 L_{SIRC} は

$$L_{\text{SIRC}}(k) = \{5 + 2(12 - k) + 3k\}T_S = (29 + k)T_S \quad (1)$$

と表せる。一方 NEC フォーマットは単位時間 $T_N = 562.5 \mu\text{s}$ を基準とし、リーダをマーク $16T_N$ とスペース $8T_N$ 、ビット 0 を $2T_N$ 、ビット 1 を $4T_N$ 、末尾にストップビット T_N を置く [15]。NEC はアドレスとコマンドの各 8 ビットに対して反転バイトを付加する 32 ビット固定長であるため、値 1 のビット数は送る値によらず必ず 16 個となる。したがってフレーム長 L_{NEC} は値によらず

$$L_{\text{NEC}} = (24 + 2 \times 16 + 4 \times 16 + 1)T_N = 121T_N \simeq 68.1 \text{ ms} \quad (2)$$

で一定である。本システムが送信するマスクは下位 4 ビットのみであり、マスク値 0x01 のときは $k = 1$ であるから、式 (1) より $L_{\text{SIRC}} = 30T_S = 18.0 \text{ ms}$ となる。両者の比較を表 2 に示す。

式 (2) が示すとおり、NEC フォーマットではフレーム送信だけで 68.1 ms を要し、これは目標値 30 ms を単独で超過する。反転バイトによるエラー検出という利点はあるが、本システムの要件を満たせない。そこで、ビット幅を 12/15/20 ビットから選べる Sony SIRC のうち最短の 12 ビットを採用した。これによりフレーム長は 18.0 ms となり、NEC の約 1/3.8 に短縮される。エラー検出機能を持たない点は本方式の弱点であり、これについては 7.4 節で実測に基づいて論じる。

なお、本プロジェクトでは開発の中盤まで NEC フォーマットを用いており、アドレス部 8 ビットに tick 番号の下位 8 ビットを載せることで、子機が受信のたびに自分の譜面位置を絶対値として復元できる構成をとっていた。しかし遅延の実測値が目標を大きく超えたため、第 9 週に Sony SIRC 12 ビットへ移行した。この移行によって遅延目標は達成できた一方、SIRC の 12 ビットには tick 番号を載せる余地がなく、位置の自己修復機能は失われている。この設計上のトレードオフについては 7.6 節で述べる。

2.3 ピアノ音源の合成方式

報告者が担当したピアノ音源については、音源データの再生 (サンプリング)、倍音の加算合成、および弦の振動を数値的に解く物理モデル合成の 3 方式を比較した。サンプリングは最も自然な音

が得られるが、音源ファイルを外部から用意する必要があり、本課題の趣旨である「合成による音作り」から外れる。物理モデル合成は表現力が高い反面、実時間で微分方程式を解く計算量が必要であり、Processing で演奏に間に合わせるのは難しい。加算合成は、ピアノ弦の物理的性質を式として取り込みながら計算量を抑えられる中間的な方式である。

ピアノの音色を特徴づける物理的性質として、弦の剛性によって倍音周波数が整数倍からわずかに高い方へずれるインハーモニシティが知られている。Fletcher らはピアノ音の倍音構造を実測してこの性質が音色の知覚に寄与することを示し [24]、Young はピアノ 8 台の測定から、インハーモニシティ係数 B が中央ハで第 2 倍音のずれ約 1.2 cent に相当する大きさとなり、中央ハより上では 8 半音ごとに倍になることを報告している [25]。また西口は、ピアノの発音機構と物理モデルを整理し、倍音ごとに減衰時定数が異なることを述べている [26]。本システムではこれらの知見を式 (4) および式 (5) として加算合成に取り込んだ。実装で採用した係数と文献値との差は 4.6 節で述べる。

3 作成したシステムの概要

本システムは、同期制御機能、楽曲制御機能、および LCD 表示機能から構成される。同期制御は親機からの赤外線信号の送受信、楽曲制御は音符情報の生成・演奏タイミング制御・音声出力、LCD 表示はかえるのアニメーションと歌詞のスクロール表示をそれぞれ担う。まずシステム全体の構成を説明する。

3.1 システム構成

システムの構成を図 1 に示す。本システムは、指揮側の親機 Arduino 1 台、演奏側の子機 Arduino 4 台、LCD 表示用の子機 Arduino 1 台の計 6 台の Arduino と、楽器音を再生する PC 4 台から構成される。いずれの Arduino も Arduino Uno R4 を用いた。親機には赤外線 LED、演奏開始・リセット・子機参加用のタクトスイッチ、およびテンポ調整用の可変抵抗を接続する。演奏子機と LCD 表示子機にはいずれも赤外線受光モジュール OSRB38C9AA[27] を接続する。演奏子機は USB シリアルで PC と接続し、LCD 表示子機は 16 文字 2 行の LCD キャラクタディスプレイを直接駆動する。PC 側では Processing が動作し、子機から届いた音符情報をもとに楽器音を合成してスピーカから出力する。楽器音の生成には Processing のサウンドライブラリである Minim[28] を用いた。

システム全体のデータの流れは一方向である。親機は参加楽器マスクを赤外線でブロードキャストし、5 台の子機がこれを同時に受信する。演奏子機は自分の担当ビットが立っていれば譜面配列を参照し、該当する音符情報を Processing へ送信する。Processing は受信した音符情報から楽器音を合成して再生する。LCD 表示子機は同じフレームを受けて歌詞と口パクの表示を進める。

3.2 製品分解構造 (PBS)

本システムの製品分解構造 (PBS) を図 2 に示す。システムは Processing (PC 側)、Arduino 親機、Arduino 子機の 3 つのサブシステムから構成される。

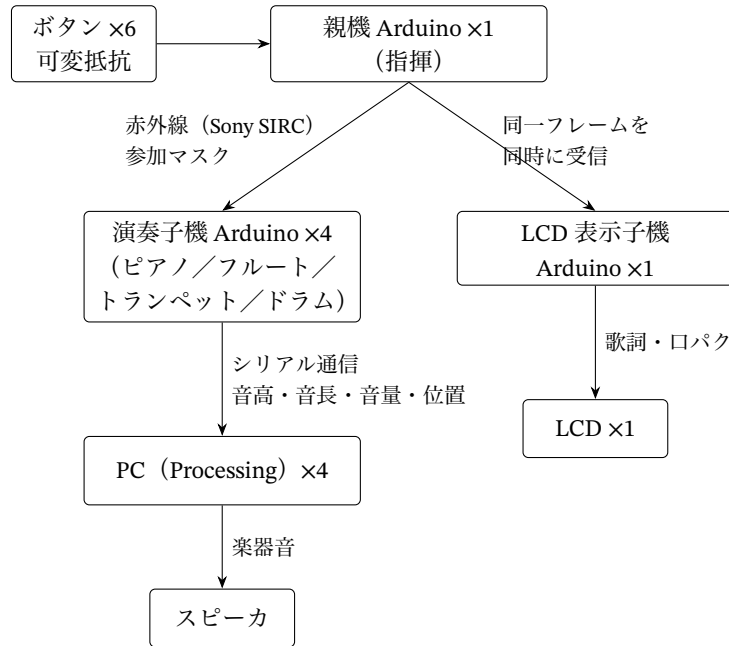


図 1 システム構成とデータの流れ

表 3 赤外線マスクのビット割り当て

ビット	値	対応する子機
Bit 0	0x01	子機 0 (ピアノ)
Bit 1	0x02	子機 1 (フルート)
Bit 2	0x04	子機 2 (トランペット)
Bit 3	0x08	子機 3 (ドラム)

3.3 赤外線同期プロトコル

親機と子機との同期には、2.2 節で選定した Sony SIRC 12 ビットフォーマット [16] を用いた。送受信には Arduino 用ライブラリの IRremote[29] を利用している。SIRC の 12 ビットはコマンド 7 ビットとアドレス 5 ビットで構成され、式 (1) より 1 フレームの送信時間は 18.0 ms である。

コマンド部 7 ビットのうち下位 4 ビットを演奏中の楽器集合を表すマスクとして使用し、上位 3 ビットは常に 0 とする。各ビットと子機の対応を表 3 に示す。アドレス部 5 ビットは常に 0 である。各子機は受信したマスクのうち自分の担当ビットだけを参照して演奏可否を判定する。楽譜のデータそのものは送らず、各子機が自分のパートの譜面配列を保持する分散構成とした。これにより 1 tick あたりの通信は 1 フレームで済み、子機の台数が増えても通信量が変わらない。

親機は演奏中、テンポに応じた周期で tick を刻む。tick 間隔 T_{tick} はテンポを BPM として

$$T_{\text{tick}} = \frac{60000}{2 \text{ BPM}} \quad [\text{ms}] \quad (3)$$

であり、1 tick が 8 分音符 1 個に相当する。120 BPM のとき T_{tick} は 250 ms になる。親機は参加中の子機が 1 台以上ある tick でのみ赤外線フレームを送信する。

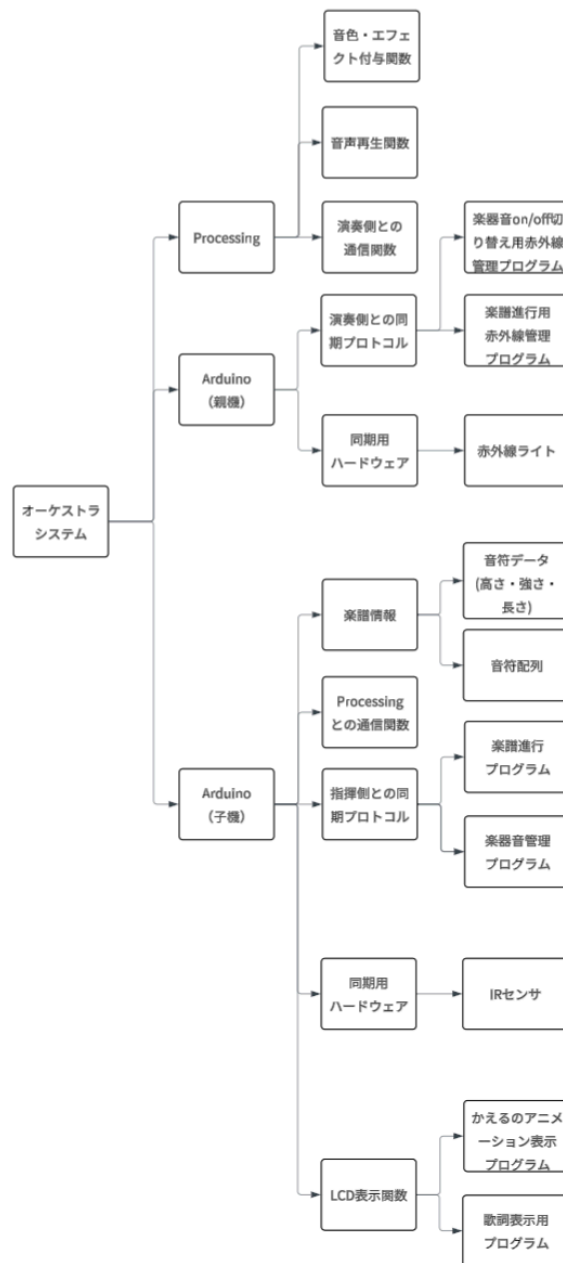


図2 製品分解構造 (PBS)

3.4 輪唱の実現と参加・退場の状態管理

輪唱は、同じ旋律を時間差で重ねることによって成立する。本システムでは、親機の再生開始後に演奏者が子機ごとのボタンを押すことで、その楽器を任意のタイミングで演奏に参加させる方式を採用した。人間の入力タイミングにはばらつきがあるため、参加の反映は次の8tick境界、すなわち拍のまとまりの先頭まで待ってから行う。こうすることで、どのタイミングでボタンを押しても各声部

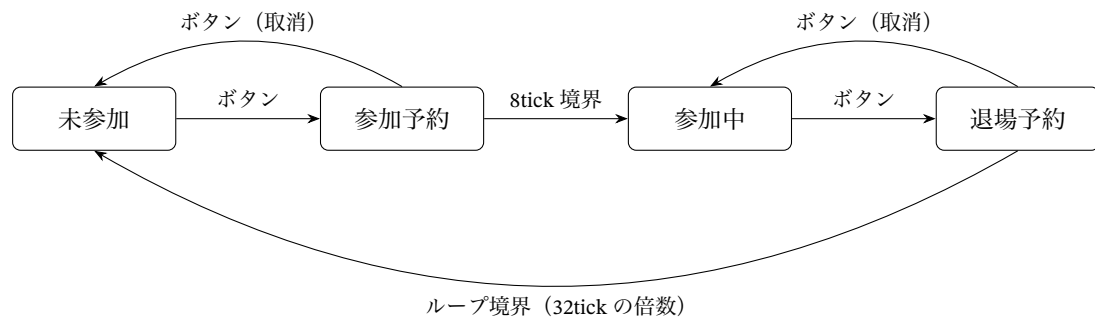


図3 子機1台分の参加状態の遷移

の入りが拍に揃い、輪唱としてかみ合う。退場の反映は、その子機が参加した時点を開始点として楽譜1周分である32tickの倍数の境界に揃える。旋律を途中で切らず、1周弾き切ってから抜けるための設計である。

親機が管理する子機1台分の状態遷移を図3に示す。各子機は未参加、参加予約、参加中、退場予約の4状態を巡回し、ボタンを押すたびに予約とその取消が交互に切り替わる。予約中にもう一度押せば取り消せるため、押し間違いにも対処できる。親機の各子機ボタンの隣には状態表示用のLEDを設け、参加の意思がある状態で点灯するようにした。どの楽器が参加予約中あるいは参加中かを物理的に確認できるので、演奏デモの操作が分かりやすくなる。

3.5 楽器音の合成

Processing側では、子機から「音高、音長、音量、譜面位置」のカンマ区切りテキストを受信し、楽器ごとの合成処理で音を再生する。ピアノは倍音の加算合成による減衰音、フルートは倍音構造とADSRエンベロープの調整による持続音、トランペットは倍音比[1.0, 0.6, 0.35, 0.2, 0.1, 0.05]と5Hzのビブラートによる金管らしい音、ドラムはキックとシンバルの2種類の打撃音である。ピアノ音源の詳細は報告者の担当分として4.6節で述べる。

3.6 LCD表示

エンターテインメント性の要素として、図4のように、子機に接続したLCDキャラクターディスプレイに、かえるのドット絵アニメーションと演奏に合わせた歌詞のスクロールを表示する。演奏の進行位置は子機が保持しているため、発音と同じタイミングで表示を更新できる。

3.7 主要な設計判断の整理

ここまで述べてきたシステム構成は、開発の過程で行った複数の設計判断の結果である。主要な判断とその根拠を表4に整理する。各判断の詳細は表中で参照する節で述べる。

このうち報告者が主体的に関与したのは、ピアノ音源の合成方式と再生方式、テンポ変更の入力方式、および輪唱の開始方式の試作である。報告者の担当部分がシステム全体の機能実現に果たす役割は次のように整理できる。ピアノは旋律を担う中心の楽器であり、その音色の再現性はMOE3(楽器音らしい音色である)に直結する。また事前レンダリング方式の採用は、発音時の処理時間

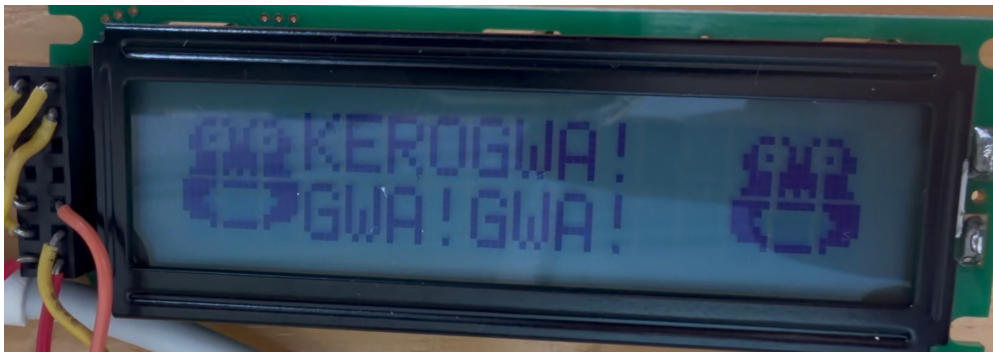


図 4 LCD に表示されたかえるのアニメーションと歌詞

を抑えることで MOE1（違和感なく聞けることができる）を支えている。テンポ管理は親機の tick 周期そのものを決める機能であり、全子機の演奏速度を一括して制御するため MOE2（可変テンポに対応している）の実現手段そのものである。さらに、4.7 節で述べる 2 件の不具合修正は、いずれも輪唱が通して成立するための前提条件であった。

4 システム実装

本章では、チームの分担と開発の経緯を述べたうえで、親機・子機・表示子機それぞれの処理を主要な関数と処理フローによって説明し、続いて報告者が担当した実装を課題ごとに述べる。報告者の担当範囲は 4.5 節に整理した 4 つの課題に対応しており、4.6 節から 4.9 節までがそれぞれの取り組みにあたる。

4.1 チーム構成と役割分担

本プロジェクトは 6 名で実施した。作業は Processing での音源作成、子機 Arduino のプログラム、親機 Arduino のプログラム、回路・表示装置の 4 系統に分け、WBS の管理番号を付けて分担した。役割分担を表 5 に示す。報告者はピアノ音源の作成とその Arduino 通信用関数、ドラムの子機プログラム、およびテンポ管理プログラムを担当した。テンポ管理は渡邊との共同である。第 9 週末までのチーム全体の作業時間は 129 時間であり、報告者の累計は 35 時間であった。

開発は、第 5 週までを計画段階、第 6 週から第 9 週までを実装段階とし、第 10 週に統合と結合テストを行う計画で進めた。WBS 第 3 層のタスク単位のスケジュールを図 5 に示す。タスク番号は表 5 の WBS 番号に対応する。

図 5 のバーは計画時の週割当てである。110: ピアノ音の作成は第 5 週の仮実装と第 7 週の音色改善の 2 期間に分けて実施した。実施ではいくつかのタスクが計画より後ろへ延びており、420: LED マトリクス作成は表示不具合の解消に時間を要して第 10 週まで作業が続いた。250: 楽譜進行、310: 赤外線管理、320: テンポ管理の 3 タスクも、第 8 週の結合テストで発覚した課題に対応するため第 9 週まで追加実装を行った。音源作成と通信用関数は計画通り第 7 週末までに完了している。

表 4 主要な設計判断と選定理由

判断項目	採用した案	検討した代替案	選定理由
同期の通信方式	赤外線ブロードキャスト	MIDI, Wi-Fi, BLE	1 対多の同時送信が本来の用途であり、通信インフラに依存しない (2.1 節)
赤外線フォーマット	Sony SIRC 12 ビット	NEC 32 ビット	フレーム長が 18.0 ms と 68.1 ms で、目標 30 ms を満たせるのは前者のみ (2.2 節)
伝送する情報	参加楽器マスク 4 ビットのみ	楽譜データの配信, tick 番号の併送	通信量を子機の台数に依存させないため、ただし位置の自己修復を失う (7.6 節)
楽譜の保持場所	各子機がプログラム内に保持	親機から毎 tick 配信	子機単体でのテストを可能にし、担当者ごとの並行開発を成立させるため
輪唱の開始方式	再生開始後の随時参加	押下順に固定の時間差を割り当てるカノン方式	任意の順・任意のタイミングで参加できる自由度を優先 (4.9 節)
参加の反映時点	次の 8tick 境界	ボタンを押した瞬間	各声部の入りを拍に揃え、輪唱としてかみ合わせるため
退場の反映時点	参加時点から 32tick の倍数境界	ボタンを押した瞬間	旋律を途中で切らず 1 周弾き切ってから抜けるため
ピアノ音源の合成	倍音の加算合成	サンプリング再生, 物理モデル合成	音源データを使わず、実時間で扱える計算量に収めるため (2.3 節)
ピアノ音源の再生	起動時の事前レンダリングと Sampler	発音のたびの実時間合成	発音時の処理を最小化し、合成の計算量が演奏タイミングへ影響しないようにするため (4.6 節)
テンポ変更の入力	可変抵抗による連続調整	ボタンによる段階的な増減	演奏中のテンポ変化を滑らかにするため (4.8 節)
親機の待機方式	millis() による経過時間の監視	delay() による待機	待機中もボタン入力と可変抵抗の読み取りを続けるため

4.2 親機 Arduino の処理

親機は、演奏の開始とリセット、子機の参加と退場の予約、およびテンポの変更を受け付けながら、全子機に共通の時間基準となる tick を刻み、赤外線フレームを送信する指揮装置である。主要な関数を表 6 に、loop() を 1 周する処理の流れを図 6 に示す。loop() では、シリアルコマンドと各ボタンの入力、および可変抵抗によるテンポ変更を毎周監視したうえで、再生中であれば前回の tick からの経過時間を調べ、式 (3) の tick 間隔 T_{tick} に達したときだけ tick() を実行する。参加ボタンとそれに対応するシリアルコマンドは toggleChild に集約されており、図 3 の状態遷移に従って

表 5 チームの役割分担

担当者	WBS 番号	主な担当
櫛濱 和輝 (報告者)	110, 120, 240, 320	ピアノ音源・通信関数, ドラム子機, テンポ管理 (共同)
家田 祐希	130, 140, 220, 310	フルート音源・通信関数・子機, 赤外線管理 (共同)
木下 遼介	150, 160, 230, 410	トランペット音源・通信関数・子機, 回路 (共同)
久家 力孔	170, 171, 180	ドラム音源 (キック・シンバル), ドラム通信関数
渡邊 乃斗	250, 310, 320	楽譜進行, 赤外線管理 (共同), テンポ管理 (共同)
手代木 巧	410, 420	回路作成, LED マトリクス・LCD 表示
全員	510, 520	輪唱機能の結合, 結合テスト

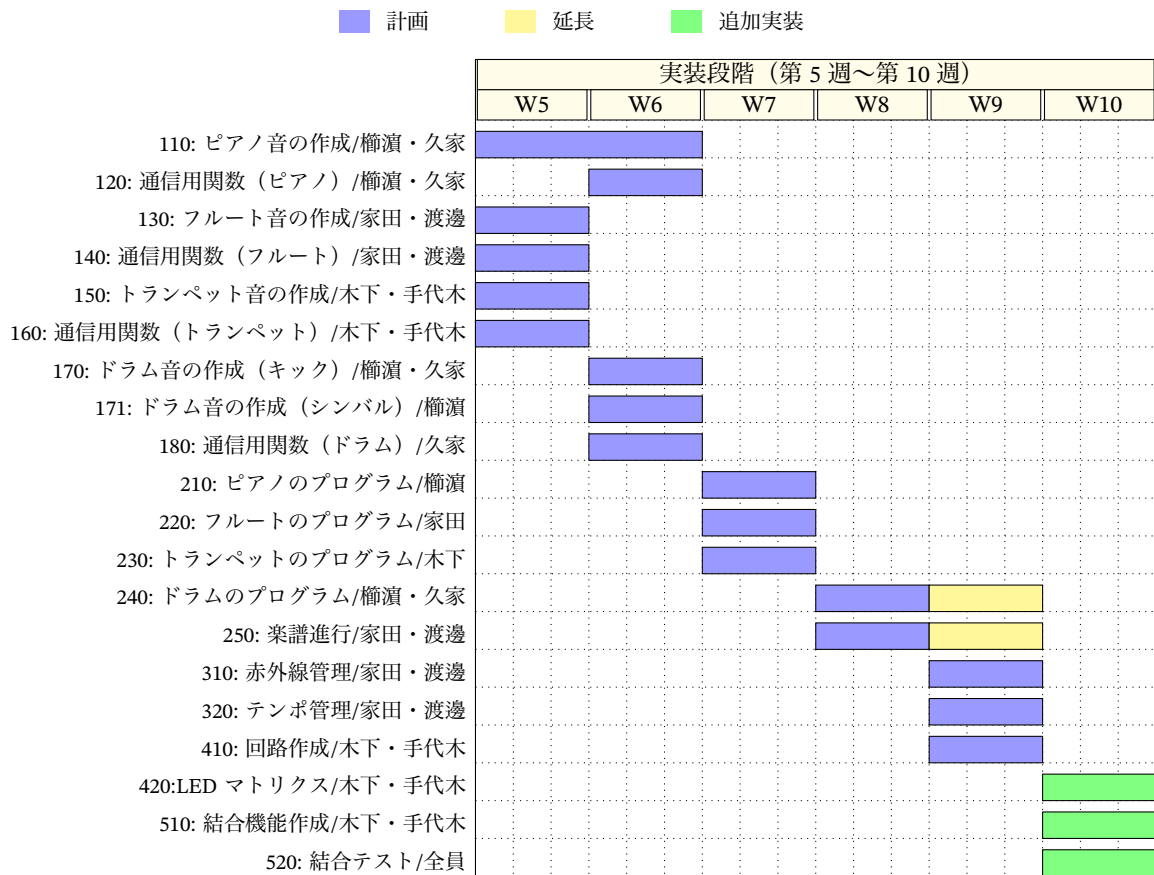


図 5 開発スケジュール

予約とその取消が切り替わる。

tick() では、まず参加予約と退場予約が発火する tick に到達したかを子機ごとに判定して参加状態を更新し、参加中の子機集合を表すマスクを組み立てる。参加中の子機が 1 台でもあるときは、遅延計測用の同期パルスを出力したのち、マスクを Sony SIRC 12 ビットの 1 フレームとして送信する。同期パルスは赤外線送信の直前に立ち上げており、子機側でこの立ち上がり時刻と受信復号の完了時刻を比較することで、6.4 節で述べる片道遅延を計測できるようにしてある。loop() の末尾では子機ボタン横の LED を参加状態と予約の有無に同期させ、予約の取消を含むどの操作でも表示が確実に切り替わるようにしている。

表 6 親機 Arduino の主要関数

関数名	役割	概要
loop	メインループ	入力の監視と tick 周期の管理を行い、子機 LED の表示を内部状態に同期させる
handleSerialCommand	シリアル入力の処理	1 文字コマンドを解釈し、参加トグルや再生開始、リセットの各関数に振り分ける
handleOrderButtons	参加ボタンの処理	子機ごとのボタンの立ち下がりを検出して toggleChild を呼び出す
toggleChild	参加と退場の予約	子機の状態に応じて参加予約、退場予約、およびそれらの取消を切り替える
handleTempoPot	テンポの調整	可変抵抗の値を平滑化して読み取り、60~150 BPM の範囲でテンポを更新する
doStart	再生の開始	全子機を未参加に初期化し、tick カウンタを 0 に戻して計時を開始する
doReset	リセット	再生を停止し、全子機の参加状態とすべての予約を初期化する
tick	同期信号の送信	予約の発火判定と参加マスクの生成を行い、赤外線フレームを送信する

4.3 子機 Arduino の処理

子機 Arduino は、親機からの赤外線フレームを受けて自分のパートを演奏する側の装置である。楽譜のデータは通信では受け取らず、各子機が自分のパートの譜面配列をプログラム内に保持している。受信したマスクのうち自分の担当ビットが立っている tick だけを自分の出番と解釈し、出番の回数を数えるカウンタ localPos を 1 つ進めたうえで、譜面配列のその位置の音符情報をシリアル通信で Processing へ送信する。親機の進行にはループの概念がないため、メロディ機は譜面の末尾に達したら localPos を 0 へ戻し、子機の側で先頭からの繰り返しを作る。ドラム機は譜面を持たず、出番のたびにキックの合図を送るだけである。子機のプログラムは 4 台で共通であり、冒頭の機体番号 childId を書き換えて各機体へ書き込むことで、メロディ機 3 台とドラム機 1 台を作り分けている。

譜面位置 24 以降は、1 tick につき 2 音を発音する倍速区間である。旋律の終盤にあたるこの区間では、1 音あたりの音長を通常の 0.40 s から 0.20 s へ半分にし、2 音目を直近の受信間隔の半分だけ遅らせて送信することで、16 分音符相当の刻みを作る。待ち時間の基準となる受信間隔は受信のたびに計測するが、演奏の停止と再開のあいだに生じる大きな間隔をそのまま使うと待ち時間が膨らむため、極端な値は捨てて更新する。また、この待ち時間の間にも次のフレームは届きうるので、受信データは復号した直後に退避して受信器をすぐに再開し、待機中も受信を続けて tick の取

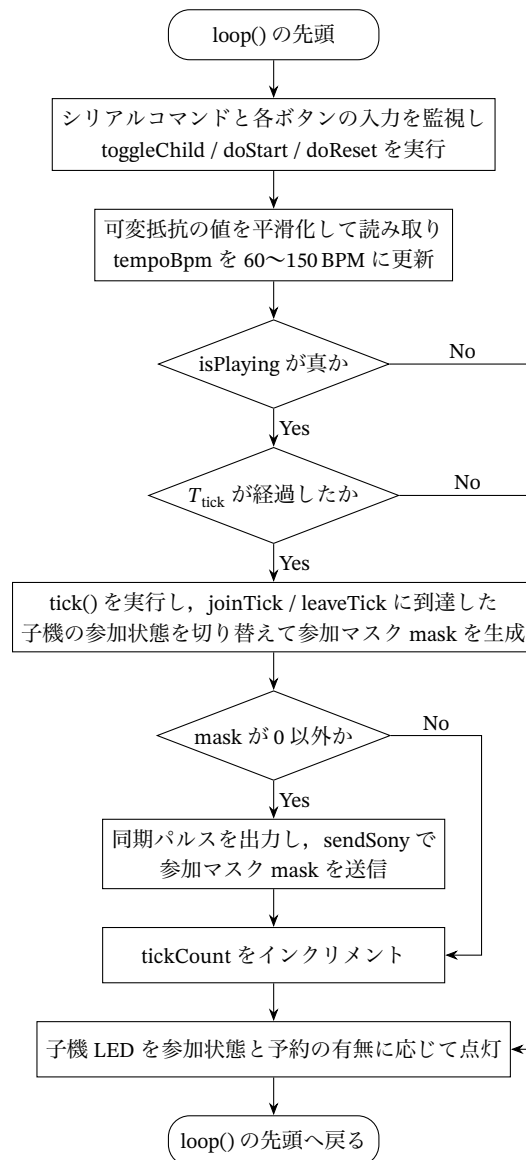


図 6 親機 Arduino の処理フロー

りこぼしを防いでいる。このほか、親機の同期パルスの立ち上がり時刻を割り込みで記録し、赤外線を受信までの遅延をログへ出力する計測の仕組みも子機側に持たせている。子機の主要な関数を表 7 に、loop の処理の流れを図 7 に示す。

4.4 LCD 表示の処理

表示用の子機は、16 文字 2 行の LCD キャラクタディスプレイ 1 枚に、左右のカエル 2 体と中央の歌詞を同時に表示する。表示例は図 4 に示したとおりである。カエルは HD44780 のカスタム文字機能で作成した 5×8 ドットのパターンを 3 列 2 行に並べて構成し、上段の 3 文字は固定の絵柄、下段の 3 文字を口の開いた絵柄と閉じた絵柄とで差し替えることで口パクを表現する。左のカエル

表 7 子機 Arduino の主要関数

関数名	役割	概要
setup	初期化	シリアル通信と赤外線受信を開始し、同期パルス入力割り込みを登録する
loop	受信監視と発音	赤外線フレームを常時監視し、自分の出番の tick で譜面位置を進めて音符情報を送信する
sendNoteToHost	音符情報の送信	音高、音長、音量、譜面位置をカンマ区切りの 1 行にまとめて Processing へ送る
onSyncRise	同期パルスの記録	親機の同期パルスの立ち上がり時刻を記録する割り込み処理で、遅延計測の基準になる
printReady	起動通知	起動時に自分の機体番号とメロディ機かドラム機かの種別をシリアルへ出力する

を列 0～2 に、右のカエルを列 13～15 に配置し、その間の列 3～12 の 2 行を歌詞表示エリアとした。HD44780 が同時に登録できるカスタム文字は 8 種類までであるため、左右のカエルで同じ 6 種類の文字を共有し、2 体の口が常に同じ形で動く構成としている。

表示の進行は親機の赤外線 tick に同期させる。受信したマスクのうち担当ビットが立った tick でのみ歌詞位置 localPos を 1 つ進め、対応する語を表示すると同時に口の開閉を反転させる。歌詞は楽譜 1 周分の 32 tick と 1 対 1 に対応する文字列テーブルとして保持している。1 語を書き込むたびに書き込み列を語長だけ進め、残り幅に収まらなくなったら下段へ改行し、下段も埋まったら歌詞エリアだけを空白で消去して先頭から書き直す。画面全体を消去せずエリアを限定して消すのは、カエルの絵柄を保持したまま歌詞だけを更新するためである。口パクの更新では、CGRAM を書き換えると表示中の文字の形も即座に変わる HD44780 の性質を利用し、カーソルを左右のカエルの下段へ移動して 6 文字を書き直すだけで 2 体の口を同時に動かしている。主要な関数を表 8 に、処理の流れを図 8 に示す。

表示子機が受信するフレームは演奏子機と同じものであり、2.2 節で述べた Sony SIRC 12 ビットへの移行にあわせて受信処理も更新している。親機は参加中の子機が 1 台でもあれば毎 tick フレームを送信するので、表示子機は演奏子機と同じ時間基準で歌詞と口パクを進められる。ただし表示子機も演奏子機と同様に受信回数で位置を数えるため、フレームを取りこぼすと歌詞の位置が 1 つ後方へずれたまま戻らない。この性質と対策は 7.6 節で演奏子機とあわせて論じる。

4.5 報告者の担当範囲と技術的課題

報告者の担当のうち、ピアノはこのシステムの旋律を担う中心の楽器である。また、テンポ管理と演奏進行の時間設計はすべての楽器に影響する。実装を進めるなかで、解決すべき課題は次の 4 点に整理された。

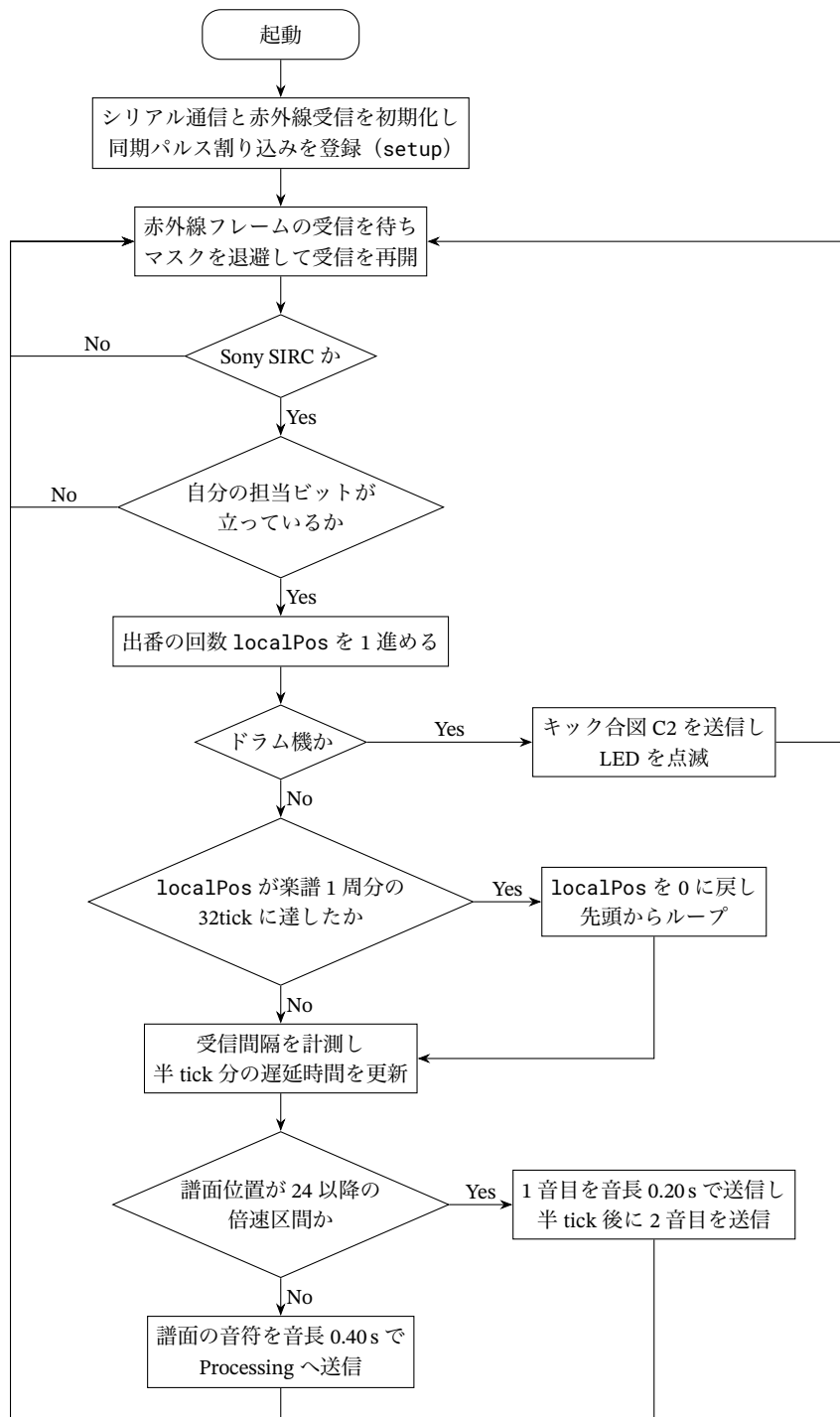


図 7 子機 Arduino の処理フロー

- 打鍵後に減衰していくピアノらしい音色を、音源データを使わずに合成で作ること
- 発音時の処理遅延を抑え、演奏のタイミングを崩さないこと
- 親機と子機の進行が食い違い、特定の音が周期的に抜けるバグを解消すること

表 8 LCD 表示の主要関数

関数名	役割	概要
setup	初期化	シリアル通信と LCD を初期化してカエルのカスタム文字 6 種を登録し、口を閉じた状態で描画したうえで赤外線受信を開始する
loop	受信と進行	赤外線フレームを受信し、担当ビットが立っていれば歌詞位置を 1 つ進めて歌詞表示と口パクを実行する
showLyric	歌詞表示	指定位置の語を歌詞エリアへ書き込み、行幅に収まらない場合は改行またはエリアの消去を行う
clearLyricArea	歌詞エリアの消去	列 3~12 の 2 行だけを空白で埋め、書き込み位置を先頭へ戻す
updateFrogs	口パクの更新	開閉に応じた口のパターンを CGRAM へ登録し直し、左右のカエル 6 文字を書き直す

- ・ 休符が休符として聞こえるように、音の余韻を制御すること

以降の節で、それぞれの課題への取り組みを順に述べる。開発にあたって利用した支援ツールと情報源、およびその検証の過程については第 5 章にまとめる。

4.6 ピアノ音源の実装

ピアノの音は、多数の倍音が異なる速さで減衰していく音である。正弦波 1 本やオルガン風の持続音では鍵盤楽器に聞こえないため、物理的な性質を模した加算合成を実装した。ピアノ弦の倍音周波数は弦の剛性により整数倍からわずかに高い方へずれることが知られており [24]、この性質はインハーモニシティと呼ばれる [26]。 h 次倍音の周波数 f_h を、基本周波数 f_s とインハーモニシティ係数 B を用いて

$$f_h = hf_s \sqrt{1 + Bh^2} \quad (4)$$

で計算する。 B は音域に応じて変える量であり、本実装では基本周波数 f_0 を用いて $B = 2 \times 10^{-4} \times (f_0/261.6)^{0.8}$ とした。すなわち中央ハ (261.6 Hz) で 2×10^{-4} である。

この値は文献の実測値と比較しておく必要がある。Young はグランドピアノとアップライトピアノ計 8 台を測定し、中央ハの第 2 倍音のずれが約 1.2 cent になると報告している [25]。これは $B \approx 3.5 \times 10^{-4}$ に相当し、本実装の値のおよそ 1.8 倍である。また音域依存についても、Young は中央ハより上では 8 半音ごとに B が倍になる、すなわち 1 オクターブあたり約 2.8 倍になるとしているのに対し、本実装の $f_0^{0.8}$ では 1 オクターブあたり約 1.7 倍にとどまる。

つまり本実装は、実際のピアノよりも非調和性を弱く、かつ音域による変化を緩やかに設定している。これは試聴を繰り返して決めた値であり、文献値から導いたものではない。合成音を単独で聴く条件では、非調和性を強めると倍音の唸りが目立って濁って聞こえたためこの値に落ち着いた。文献値に近づけた場合の聴感を比較検証することは今後の課題である。

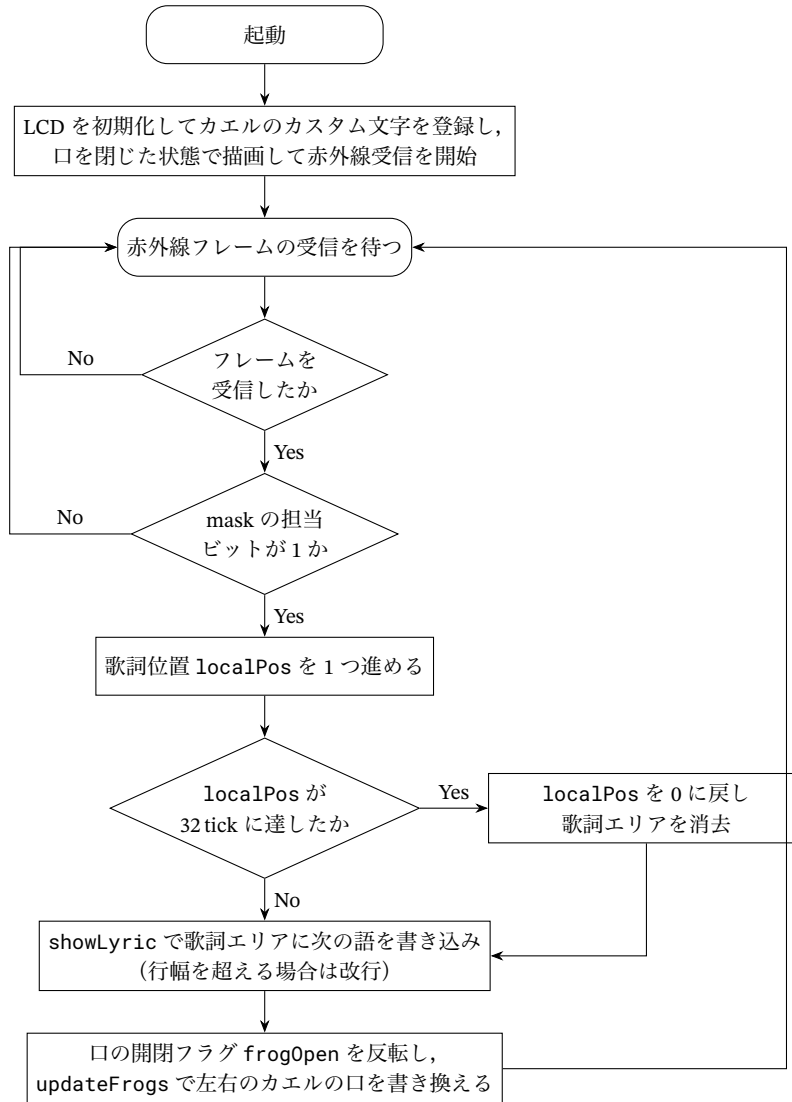


図 8 LCD 表示子機の処理フロー

倍音は 10 次まで重ね、 h 次倍音の振幅は $1/h^p$ に比例させて高次ほど弱くする。指数 p は低音域で 0.9，高音域で 1.4 程度になるように基本周波数から連続的に決めた。

減衰のエンベロープは、打鍵直後の速い減衰とその後の緩やかな減衰の 2 段階を重ねた形とし、時刻 t での振幅係数を

$$a(t) = 0.3 e^{-t/0.12} + 0.7 e^{-t/\tau_h}, \quad \tau_h = \frac{\tau_1}{1 + 0.6(h-1)} \quad (5)$$

とした。 τ_1 は基本音の減衰時定数であり、残響の長さが低音ほど長くなるように音域から決める。式 (5) の τ_h は次数とともに小さくなるので、高次倍音ほど速く消える。これは実際のピアノで打鍵直後は明るく、時間とともに丸い音色に変わっていく振る舞いに対応する。このほか、実機のピアノが 1 音あたり複数の弦を持つことを模して、音域に応じて 1～3 本の弦をセント単位でわずかにずらして重ねた。さらに打鍵の瞬間のハンマーノイズとして、最初の 15 ms だけローパスフィルタ

を通した雑音を加え、立ち上がり 3 ms のアタックと、音長経過後に時定数 0.15 s で減衰するダンパーリリースを付けている。

もう 1 つの課題は発音遅延である。上記の合成をすべて発音のたびに計算すると、波形生成に時間がかかり演奏に間に合わないおそれがある。計算量を見積もると、1 音の波形は音長 0.40 s に余韻 0.8 s を加えた 1.2 s 分であり、標準化周波数 44.1 kHz では 52,920 サンプルとなる。高音域では 3 本の弦それぞれに 10 次までの倍音を重ねるため、1 音あたり $52,920 \times 3 \times 10 \approx 1.59 \times 10^6$ 回の正弦波演算が必要になる。一方、発音の間隔は 120 BPM の tick 間隔 250 ms、倍速区間では 125 ms である。この時間内に約 159 万回の演算を確実に終える保証はない。そこで、楽譜に登場する 6 種類の音高と 2 種類の音長の組を Processing の起動時にすべて波形として生成し、Minim の Sampler にキャッシュしておく方式にした。起動時に生成する波形は 12 種類、演算回数の合計は約 1.8×10^7 回であるが、これは起動時に 1 度だけ実行すればよい。演奏中はキャッシュ済み Sampler の trigger() を呼ぶだけなので、発音時の処理は音源データの再生と同じで済む。Sampler は同時 6 声まで発音できる設定とし、前の音の余韻と次の音が自然に重なるようにした。未登録の音高が届いた場合はその場で生成してキャッシュに加える安全網も設けている。

波形生成には 2 つの処理を組み込んだ。ひとつはクリップ防止であり、生成した波形のピーク振幅が 0.98 を超える場合に全体を縮小する。倍音を 10 次まで、弦を最大 3 本重ねるため、位相が揃った瞬間に振幅が 1 を超えて歪む可能性があるためである。もうひとつは乱数シードの固定である。ハンマーノイズの生成に用いる擬似乱数に固定のシードを与えることで、同じ音高からは常に同一の波形が生成される。これにより、パラメータを変えながら試聴で比較する際に、ノイズ成分の偶然の違いが判断に混じらないようにした。音色の調整は主観的な聴感に頼らざるを得ないため、比較の条件を揃えられるようにしておくことが重要と考えた。なお、このピアノ音源は当初オルガン風の持続音で仮実装しており、第 9 週に本節で述べた合成方式へ差し替えた。差し替え時には楽譜で使う全音高について試聴による検証を行い、全ケースで楽器音として問題ないことを確認している。各波形生成部のコードは付録 C,D,E,F, に示す。

4.7 演奏進行の不具合修正

結合テストの過程で、演奏の時間管理に関わる不具合を 2 件修正した。いずれも症状の観察からログで原因を絞り込んだもので、修正自体は小さいが、輪唱の成立には欠かせない修正だった。

1 件目は、特定の位置の音が周期的に鳴らない症状である。旋律の途中、譜面位置 16 や 24 の音が数周に一度抜けて聞こえた。子機のシリアルログで譜面位置の進み方を追ったところ、親機は 32 tick を楽譜 1 周として進行しているのに対し、子機は 1 周を 33 tick と解釈していることが分かった。子機側でループ末尾の休みを表す定数を 2 tick としていたため、旋律 31 tick との合計が 33 tick になっていたのが原因である。この不整合が症状として現れる機序は次のとおりである。親機は 32 tick で楽譜 1 周を刻むのに対し、子機は 33 回の受信で 1 周とみなすため、1 周を終えるごとに子機の譜面位置が親機の意図する位置より 1 tick ずつ後方へずれていく。子機の譜面配列では末尾に休符が置かれているため、ずれが特定の位置に達した tick では本来音があるはずの箇所でも休符が参照され、音が抜けて聞こえる。ずれは 1 周あたり 1 tick ずつ増えて 32 周で一巡するので、同じ位置

の音が抜ける現象は 32 周ごとに再現する。120 BPM では 1 周が $32 \times 0.25 = 8\text{s}$ であるから、周期はおよそ 4 分 16 秒となる。症状が「数周に一度」に見え、短い試奏では気づきにくかったのはこのためである。この定数を 1 に修正して両者を 32tick に統一し、症状は解消した。第 9 週の 4 子機実機テストでも再発していない。

2 件目は、休符が休符として聞こえない問題である。本システムの音長の既定値は 0.40s であり、120 BPM での tick 間隔 0.25s より長い。前の音の余韻が次の tick まで残るため、譜面上の休符 R のタイミングでも直前の音が鳴り続け、旋律の切れ目が埋まってしまっていた。そこで Processing 側で休符 R を受信した時点で、発音中の音を即時に停止する処理を入れた。オルガン風音源の時点では `noteOff()` の即時呼び出しとして実装し、ピアノ音源への差し替え後も Sampler の `stop()` で同じ動作を維持している。音が鳴っている最中に切る処理なので不自然に聞こえる心配があったが、ピアノのダンパーで音を止めた場合と似た切れ方になり、聴感上の違和感はなかった。

4.8 テンポ管理

テンポ管理は、親機の tick 周期を演奏中に変更する機能である。当初はボタンの押下でテンポを段階的に増減する方式を実装していたが、デモで聴かせる際にテンポが跳びとびに変わるのは不自然だったため、第 9 週に可変抵抗による連続調整へ変更した。10 k Ω のシングルターン可変抵抗をアナログ入力に接続し、読み値 v (0~1023) をテンポの範囲 60~150 BPM へ線形に対応付ける。

$$BPM(v) = 60 + \left\lfloor \frac{(150 - 60)v}{1023} \right\rfloor \quad (6)$$

読み取りは 50 ms 間隔とし、読み値のノイズによるテンポの揺れを抑えるため、

$$s_n = \frac{3s_{n-1} + v_n}{4} \quad (7)$$

の指数移動平均で平滑化してから式 (6) に与える。親機の tick 周期の計算は毎ループ現在のテンポ変数を参照しているため、平滑化後の値が変われば次の tick から反映される。

この平滑化が追従時間に与える影響は式 (7) から見積もれる。 v が階段状に変化したとき、 n 回目の読み取り後の s_n は最終値の $1 - 0.75^n$ 倍まで達する。読み取り間隔は 50 ms であるから、63% に達するのは約 174 ms 後 ($n \simeq 3.5$)、95% に達するのは約 550 ms 後 ($n = 11$) である。120 BPM の 1 拍は 500 ms であり、つまみを一気に回して大きくテンポを変えた場合には、95% の反映までに 1 拍をわずかに超える計算になる。実機で 1 拍以内の追従を確認できたのは、つまみの操作が連続的で 1 回あたりの変化量が小さかったためと考えられる。追従の速さとテンポの安定性は平滑化係数を通じて背反の関係にあり、現在の値はデモでの操作感を優先して選んだものである。

シリアルモニタへのログ出力は 2 BPM 以上変化したときに限定し、微小な揺らぎでログが埋まらないようにした。

なお、親機の進行処理は `delay()` で待つのではなく、`millis()` で前回 tick からの経過時間を監視する方式である。待ち時間の間もボタン入力と可変抵抗の読み取りを続ける必要があるため、テンポ変更・参加操作・tick 送信が互いをブロックしない構造になっている。

4.9 輪唱開始方式の試作と設計変更

輪唱の開始方法については、開発の途中で設計を大きく変えた経緯がある。当初の計画では、演奏開始前にボタンで各楽器の順番を登録し、登録順に固定の時間差で追いかける方式を想定していた。報告者はこの系統の親機プログラムに対して、押下順に応じて各子機の出だしのずれを動的に割り当てる実装を試作した。子機固有の固定遅延値として持っていた配列を、押された順番 N に対する割当 $\{0, 8, 16, 20\}$ tick へ読み替えることで、どのボタンを最初に押してもその楽器が先頭声部として位置 0 から歌い出す、本来のカノンの形を実現するものである。

一方で、チーム内の検討により、輪唱の定義は「固定順で参加する」ものではなく「任意の順・任意のタイミングで参加できる」べきだという結論になり、第 7 週末にアーキテクチャを再生開始後の随時参加ボタン方式へ変更した。この方式では子機は演奏していない間も常に曲位置を共有しているため、途中から参加しても、その時点の位置から自然に演奏へ加われる。押下順カノン方式の試作は採用に至らなかったが、2 案を実機で比較したことで、随時参加方式の方が演奏デモとしての自由度が高いことを確認したうえで移行できた。最終版の親機プログラムはこの随時参加方式であり、報告者が試作で得た 8 tick 境界への参加アライメントの考え方は、そのまま参加予約の量子化として引き継がれている。

4.10 担当部分以外の実装と工夫

システムの残りの部分について、チームの実装を概観する。赤外線管理プログラムは渡邊と家田が担当し、親機からマスクと tick 番号を一齐送信する方式や、参加・退場を OFF・QUEUED・ACTIVE の 3 状態で管理する仕組みを実装した。楽譜進行プログラムは渡邊が担当し、スタート信号を受けた tick の記録と、現在の tick との比較で出力音を決める機能を追加した。

フルートは家田の担当で、倍音構造と ADSR の調整により息の混じった柔らかい音色を作り、アタックタイムを短くしてテンポへの追従性を改善した。トランペットは木下の担当で、倍音比とビブラートによる音色合成に加え、音の切り替え時に発生していたプチという雑音を、Minim の出力ラインに 0.03 s のフェードアウトを入れることで解消した。また、トランペット系では Arduino から 29 音の楽曲データを一括送信するため、シリアル通信速度を 921600 bps とし受信配列への格納を確認している。ドラム音源は久家の担当で、キックとシンバルの 2 種を合成し、Arduino から受信した整数値で再生し分ける Processing を実装した。回路と表示装置は手代木の担当で、親機・子機の回路作成と、LCD 表示プログラムの作成を進めた。

回路面では、第 6 週に赤外線受光モジュールの故障が 3 台見つかり、新品への交換と動作確認を家田が行った。こうした部品不良の切り分けができたのは、受光モジュール単体の受信テストを単体テストとして分離していたためである。

表 9 利用した開発支援ツールと情報源

種別	対象	利用目的	検証の方法
生成 AI	Claude Opus4.6	ピアノ音源の合成コードの下書き, 不具合の原因候補の列举	文献による式の裏取り, 実機での試聴, シリアルログによる動作確認
学術文献	Fletcher ら [24], Young[25], 西口 [26]	インハーモニシティと倍音減衰の物理的根拠の確認, 係数 B の実測値との照合	式 (4) として実装し, 係数を文献値と比較したうえで試聴により決定 (4.6 節)
仕様資料	SB-Projects の IR 解説 [15, 16]	NEC と Sony SIRC のフレーム長の算出	算出値と実測の片道遅延を突き合わせ (表 16)
ライブラリ文書	IRremote[29]	sendSony の引数仕様, プロトコル定数, フレーム終端判定の挙動	実機のシリアル出力で受信内容を確認
公式文書	Arduino Documentation[30]	millis, micros, attachInterrupt, analogRead の仕様確認	実機の動作と照合
データシート	OSRB38C9AA[27]	受光モジュールのピン配置, 電源電圧, キャリア周波数 38 kHz	受信テストで動作確認, 故障判定にも使用
ライブラリ文書	Minim[28]	Sampler, Oscil, ADSR の使い方	Processing 上で単体動作を確認

5 開発支援ツールと情報源の活用

本章では, 報告者が開発で利用した支援ツールと情報源, それらをどのような目的で用いたか, および生成された結果をどのように検証して採否を判断したかを述べる.

5.1 利用したツールと情報源の一覧

利用したツールと情報源を表 9 に整理する. いずれについても, 得られた内容をそのまま用いるのではなく, 一次資料との突き合わせか実機での動作確認のいずれかを経てから採用することを原則とした.

表 9 の 3 行目は, 情報源を主体的に使った例として挙げておきたい. SB-Projects の解説から読み取った Sony SIRC の単位時間とビット表現を用いて式 (1) を立て, フレーム長 18.0 ms を算出した. この値を実測の片道遅延 28.08 ms と突き合わせることで, 差分 10.1 ms が受信側の処理時間であることを特定できた (6.4 節). 仕様資料の数値を鵜呑みにして「SIRC だから速い」と述べるのではなく, 仕様から導いた予測と実測を比較することで, 遅延の内訳という新しい情報が得られている.

5.2 生成 AI の利用手順と検証の方針

生成 AI は、主にピアノ音源の実装と不具合の原因究明の 2 つの場面で利用した。いずれの場面でも、生成された内容を直接採用することはせず、次の手順を踏んだ。

第 1 に、生成されたコードや提案の根拠となる技術的な前提を、一次資料で確認する。ピアノ音源であれば、倍音周波数が整数倍からずれるという前提が正しいかを文献 [24, 26] で確認した。第 2 に、実機または Processing 上で動作させ、意図した結果が得られるかを確認する。音色であれば試聴、通信であればシリアルログでの確認である。第 3 に、得られた結果が不十分であれば、どの部分が原因かを切り分けたうえで修正する。音色を決めるパラメータ類は、この段階で試聴を繰り返しながら手動で調整した。

この手順を踏んだ理由は、生成 AI の出力が「もっともらしいが検証されていない」状態で提示されるためである。とくに音色の合成では、コードが文法的に正しく動作しても、出てくる音が目的の楽器に聞こえるとは限らない。コードが動くことと目的を達成することは別であり、前者だけを確認して採用することは避けた。

5.3 生成結果を修正して採用した事例

ピアノ音源の加算合成では、生成 AI に倍音の加算による減衰音の下書きを作らせたうえで、次の 2 点を自分で修正した。

1 点目は倍音の振幅配分である。下書きでは h 次倍音の振幅を一律に $1/h$ としていた。これを実装して試聴したところ、低音域では問題ないものの、高音域で倍音が強く残り金属的な響きになった。実際のピアノでは高音域ほど倍音が少なく基音が支配的になる。そこで振幅を $1/h^p$ とし、指数 p を低音域で 0.9、高音域で 1.4 程度になるよう基本周波数から連続的に決める形に変更した (4.6 節)。

2 点目は減衰の形である。下書きでは全倍音に共通の単一の指数減衰を用いていたが、これでは打鍵直後から音が消えるまで音色が変わらず、ピアノらしく聞こえなかった。西口が整理しているとおり、実際のピアノでは倍音ごとに減衰時定数が異なる [26]。そこで式 (5) のように、速い減衰と緩やかな減衰の 2 段階を重ねたうえで、時定数 τ_h を次数 h とともに小さくすることで、時間経過とともに音色が丸くなる振る舞いを再現した。

いずれの修正も、文献が示す物理的な性質と実機での試聴結果の両方に基づいている。生成された下書きは実装の出発点としては有用であったが、そのままでは目的とする音色に達していなかった。

5.4 生成結果を採用しなかった事例

採用を見送った提案もある。ここでは 2 件を挙げる。

1 件目は、ピアノ音源の再生方式である。初期の案は、フルートやトランペットと同様に Minim の `Oscil` と `ADSR` を組み合わせ、発音のたびに実時間で合成する構成であった。しかしピアノは 10 次までの倍音に加えて 1 音あたり最大 3 本の弦を重ねるため、1 回の発音で生成する正弦波の本

数がフルートやトランペットの数倍になる。発音のたびにこの計算を行うと、演奏のタイミングに間に合わない可能性があった。そこで実時間合成を採らず、楽譜に登場する音高と音長の組を Processing の起動時にすべて波形として生成し、Sampler にキャッシュしておく事前レンダリング方式へ変更した (4.6 節)。発音時の処理を `trigger()` の呼び出しだけに減らすことで、合成の計算量が演奏タイミングへ影響しない構成としている。

2 件目は、休符が休符として聞こえない問題への対処である。4.7 節で述べたとおり、この問題は音長の既定値 0.40 s が tick 間隔 0.25 s より長いために生じていた。対処として音長そのものを短くする案も検討したが、音長を短くすると休符でない箇所でも音が早く切れ、旋律が瘦せて聞こえる。そこで音長は保ったまま、休符 R を受信した時点で発音中の音を停止する処理を入れる方式を採った。実際のピアノでダンパーが弦の振動を止める動作に相当し、聴感上の違和感がないことを試聴で確認している。

これらの判断はいずれも、提案の内容を理解したうえで、本システムの制約 (Processing の処理能力、tick 間隔と音長の関係) に照らして下したものである。

5.5 不具合の原因究明における利用

4.7 節で述べた tick 数不整合のバグでは、「特定の位置の音が数周に一度抜ける」という症状から考えられる原因候補を生成 AI に列挙させた。挙げた候補は、赤外線フレームの取りこぼし、Processing 側のシリアルバッファの詰まり、親機と子機の定数の不一致などであった。

これらを 1 つずつシリアルログで検証した。子機の譜面位置の進み方を追ったところ、親機が 32 tick を楽譜 1 周としているのに対し子機が 33 tick と解釈していることが判明し、定数の不一致が原因であると特定できた。候補の列挙は切り分けの出发点として有用であったが、どの候補が正しいかを決めたのはログという実データである。

なお、このとき否定した「赤外線フレームの取りこぼし」という候補は、当時の症状の原因ではなかったものの、後の定量計測 (6.4 節) で 0.26% の頻度で実在することが分かった。症状の原因ではないことと、現象が存在しないことは別である。この点は、原因候補を消去する際に「今回の症状を説明できるか」と「そもそも起きているか」を区別すべきだという教訓として残った。

6 評価方法と結果

本章では、まず評価に用いる指標の定義を計画段階に立ち返って整理する。とくに演奏タイミング誤差については、計測した量と指標が定義する量が異なるため、両者の対応を明確にしたうえで結果を述べる。続いて単体テストと結合テスト・システムテストの結果を示し、赤外線同期については 10,323 tick 分のログに基づく統計と、遅延の内訳・外れ値・欠落の原因を示す。音色と LCD 表示についても結果と、その判定に伴う不確かさを述べる。得られた結果の解釈と限界の議論は第 7 章で行う。

表 10 有効性指標 (MOE) の定義

MOE	有効性指標
MOE1	違和感なく聞くことができる
MOE2	可変テンポに対応している
MOE3	楽器音らしい音色である
MOE4	歌詞・アニメーションが演奏に対応している

表 11 合奏および電子楽器における遅延の基準値

遅延	内容	出典
約 6 ms	等時的な系列における発音タイミングのずれの弁別閾。これを下回るずれは聴き手に検知されない	[31]
10 ms	電子楽器において、演奏者の操作から音が出るまでに許容される上限。ジッタは 1 ms 以内とすべきとされる	[32]
25 ms	遠隔合奏システムが満たすべき端点間遅延の上限 (Ensemble Performance Threshold)	[33]
30 ms	本システムの MOP1 の目標値	[13]
20~60 ms	この範囲の片道遅延であれば合奏は成立するとされる範囲	[34]
100 ms 以上	二重奏が明確に破綻する遅延	[35]

6.1 評価方法

本システムの評価は、計画段階で定義した有効性指標 (MOE)、性能指標 (MOP)、技術性能指標 (TPM) に基づいて行う。有効性指標 MOE を表 10 に示す。

各 MOE に対応する性能指標 MOP を表 12 に定めた。MOE1 には MOP1 (演奏タイミング誤差) と MOP2 (同期信号の受信成功率) が、MOE2 には MOP3 (テンポ変更への追従時間) が、MOE3 には MOP4 (音色の再現性) が、MOE4 には MOP5 (歌詞・アニメーション同期遅延) がそれぞれ対応する。

ここで MOP1 の定義に注意が必要である。計画段階での MOP1 は「各演奏側 Arduino 間の音符開始タイミングのずれ」、すなわち機器間の相対的なずれとして定義した。親機の送信から子機の受信までに要する片道の時間は MOP1 ではなく、プロジェクト中に監視する技術性能指標 TPM1 (赤外線通信遅延) として別に定義している。この 2 つは物理的に異なる量であり、混同すると評価の意味が変わってしまう。本報告書では両者を区別し、TPM1 として計測した片道遅延の統計から、MOP1 が満たされているかを論証する形をとる。MOP と TPM の対応を表 12 および表 13 に示す。

MOP1 の目標値 30 ms は、実演奏のアンサンブルにおける奏者間の発音タイミングのずれがおおよそ 30~50 ms の範囲に収まるという Rasch の報告 [13] を根拠とする。この値は、合奏における遅延の許容範囲に関する既存研究の中でどこに位置づけられるだろうか。表 11 に、関連する報告を遅延の小さい順に整理する。

表 11 が示すとおり、目標値 30 ms は、聴き手が検知できる下限 (6 ms) を上回るが、遠隔合奏システムの一般的な基準 (25 ms) と同じ桁にあり、合奏が成立するとされる範囲 (20~60 ms) に収

表 12 性能指標 MOP と目標値

MOP	性能指標	目標値（計画段階での定義）	計測方法
MOP1	演奏タイミング誤差	演奏側 Arduino 間の音符開始タイミングの差が 30 ms 以内	TPM1 の統計から上界を評価
MOP2	同期信号の受信成功率	送信に対する正しい受信の割合が 95 % 以上	送信回数と受信回数の比較
MOP3	テンポ変更への追従時間	全演奏側が 1 拍以内に追従	シリアルモニタでの確認
MOP4	音色の再現性	聴衆の過半数が対応する楽器の音と判断	5 段階のアンケート
MOP5	歌詞・アニメーション同期遅延	演奏と LCD 表示の差が 55 ms 以内	演奏の動画からのフレーム差分

表 13 監視した技術性能指標 TPM と本報告書での扱い

TPM	監視項目	本報告書での扱い
TPM1	赤外線通信遅延	親機の送信から子機の受信復号完了までの片道遅延として計測し、6.4 節で統計を示す
TPM2	同期信号受信率	MOP2 の評価にそのまま用いる
TPM3	Processing 音声再生遅延	事前レンダリング方式の採用により発音時の処理を最小化（4.6 節）、定量計測は未実施
TPM4	LCD 表示遅延	MOP5 の評価にそのまま用いる
TPM5	CPU 負荷・メモリ使用量	コンパイル時のメモリ使用量で監視、定量計測は未実施

まり、破綻する水準（100 ms）の 1/3 以下である。すなわち「ずれは検知されうるが合奏としては成立する」という位置づけの目標値であり、本プロジェクトの規模に対して妥当な設定であったといえる。

MOP5 の目標値 55 ms は、音と映像のずれの知覚に関する報告 [36] と、アンサンブル演奏における音の遅延条件の検討 [37] に基づいて定めた。音と映像の同期については視聴者が同時と感じる範囲を概ね 100 ms 以内とする整理もある [38] が、本システムでは口パクと発音が 1 対 1 に対応し、ずれが検知されやすいと判断してより厳しい 55 ms を採用した。

検証は単体テスト、結合テスト、システムテストの 3 段階で計画した。単体テストでは各音源と通信機能を個別に確認し、結合テストでは親機・子機間の同期と子機・Processing 間の接続を確認する。システムテストでは全構成を接続して課題曲の通し演奏を行う。

6.2 単体テストの結果

第 5 週から第 8 週にかけて実施した単体テストの結果を表 14 に示す。音源系では、フルートの音色聴感テスト、トランペットの音色合成テストと切り替えノイズテスト、ドラムの単体再生テストがそれぞれ完了した。トランペットの切り替えノイズは、フェードアウト処理の追加によってテスト上で雑音が確認されなくなり、MOP4（音色の再現性）に関連する問題点として管理していたものが解消された。通信系では、921600 bps でのシリアル通信テストで 29 音の楽曲データが欠落

表 14 テストの実施結果

実施日	テスト内容	結果	備考
5/20～5/25	フルート音色聴感テスト	成功	倍音構造・ADSR 調整完了
5/26	トランペット音色合成テスト	成功	倍音比とビブラートの確定
5/26	音切り替えノイズテスト	成功	0.03s フェードアウトで雑音解消
5/26	シリアル通信テスト (921600 bps)	成功	29 音を欠落なく受信
6 月初旬	ドラム音色単体テスト	成功	キック・シンバルの再生確認
6/17	受光モジュール交換後の通信テスト	成功	故障 3 台を交換
6/19	3 子機の輪唱テスト	部分成功	ドラムを除く 3 楽器で動作
6/23	4 子機の輪唱実機テスト	成功	音抜け・休符抜けの再発なし
7 月中旬	全構成でのシステムテスト	成功	表示子機を含む全モジュールが同期動作

なく受信・格納されることを確認した。また、赤外線受光モジュールの故障 3 台の交換後に受信テストを再実施し、回路の正常動作を確認している。ピアノ音源は差し替えの際に、楽譜で使う全音高と 2 種類の音長の組について試聴で確認した。

6.3 結合テストとシステムテストの結果

結合テストは 2 段階で行った。第 7 週末の 3 子機テストでは、ピアノ・フルート・トランペットの 3 楽器での輪唱開始を実機で確認した。この時点ではドラムが未結合であり、部分成功という扱いである。続く第 9 週の 6 月 23 日に、親機 1 台と子機 4 台の全構成による輪唱の実機テストを実施した。ボタンの押下順にピアノ・フルート・トランペット・ドラムが順次参加するカノン演奏を通して行い、聴感上の破綻なく演奏できることを全員で確認した。4.7 節で述べた位置抜けと休符抜けは、このテストで再発していない。参加・退場のトグル操作についても、予約とその取消、途中退場後の再参加が状態表示 LED の点灯と一致して動作した。

システムテストは、最終発表でのデモンストレーションとして実施した。親機 1 台、演奏子機 4 台、表示子機、および楽器音を再生する PC 4 台を接続した全構成で「かえるのうた」の輪唱を通して演奏し、ボタン操作による楽器の参加と退場、可変抵抗によるテンポの変更、LCD の歌詞表示と口パクアニメーションのすべてが正常に動作することを確認した。図 1 に示した構成のうち、結合テストの段階では未接続だった表示子機を含め、全モジュールが同一の赤外線 tick に同期して動作したことになる。MOE1 から MOE4 までの有効性指標は、このシステムテストをもって達成と判断した。

テンポ管理については、可変抵抗のつまみ操作で演奏中のテンポが 60～150 BPM の範囲で連続的に変わることを確認した。式 (3) の tick 周期は毎ループ現在のテンポを参照するため、追従の遅れはつまみ読み取りの平滑化に伴う数百 ms 程度であり、1 拍以内という MOP3 の目標を満たす動作である。ただしこの追従時間もログからの定量値としては算出しておらず、シリアルモニタ上の値の変化と聴感での確認にとどまっている。

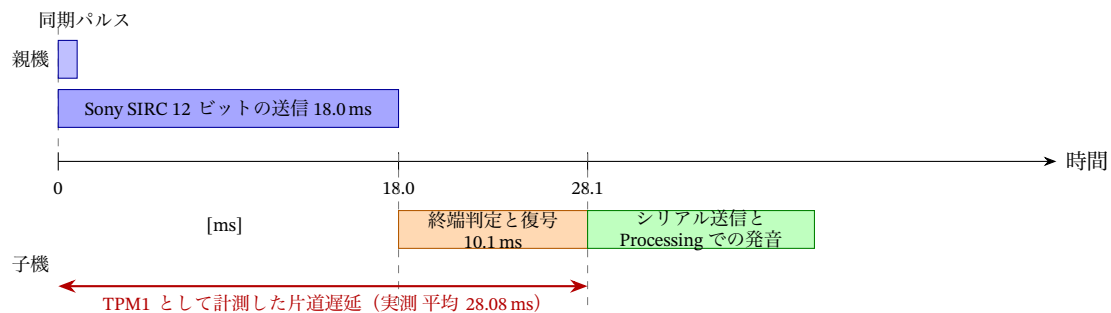


図 9 1tick における時間の流れと計測区間の対応

表 15 片道遅延 (TPM1) の統計 ($n = 10,323$)

統計量	値 [ms]	統計量	値 [ms]
平均	28.08	最小値	27.98
中央値	28.07	最大値	34.69
標準偏差	0.13	第 99 百分位	28.12
外れ値を除く平均	28.07	外れ値を除く標準偏差	0.02

6.4 赤外線同期の定量評価

6.4.1 計測方法

赤外線同期の性能を計測するため、親機と子機の双方に計測用の機構を実装した。親機は赤外線フレームを送信する直前に同期パルス用の端子を立ち上げる。子機はこのパルスの立ち上がりを外部割り込みで捉えて `micros()` による時刻を記録し、赤外線フレームの復号が完了した時点との差を片道遅延としてシリアルへ出力する。この片道遅延が TPM1 (赤外線通信遅延) にあたる。

7 月 13 日に、この機構を有効にしたまま親機 1 台と子機 1 台を接続し、演奏状態のまま tick を刻み続けて 10,323 tick 分のログを取得した。計測時の tick 間隔は、限られた時間で試行回数を確保するため実演奏時の 250 ms (120 BPM) より短い 50 ms に設定している。総計測時間は 520.9 s であった。時刻の記録に用いた `micros()` の分解能は $4\mu\text{s}$ であり、以下に示す統計量の桁数はこの分解能に対して十分である。図 9 に、1 回の tick における時間の流れと計測区間の対応を示す。

6.4.2 片道遅延の統計

取得した 10,323 件の片道遅延の統計を表 15 に、分布を図 10 に示す。平均は 28.08 ms、標準偏差は 0.13 ms であり、全体の 99.88% が 28.2 ms 以下に収まった。

図 10 が示すとおり、遅延は 28.0~28.2 ms の幅 0.2 ms の範囲にほぼすべてが集中している。28.2 ms を超えたのは 12 回 (0.116%) であり、このうち 1 回は起動直後の初回受信 (34.69 ms) である。残る 11 回は 30.8~31.9 ms の範囲に集まっている。これら 12 件を除いた 10,311 件では標準偏差が 0.02 ms となり、これは時刻記録に用いた `micros()` の分解能 $4\mu\text{s}$ の数倍にすぎない。すなわち本システムの片道遅延は、測定分解能に近い精度で一定値とみなせる。

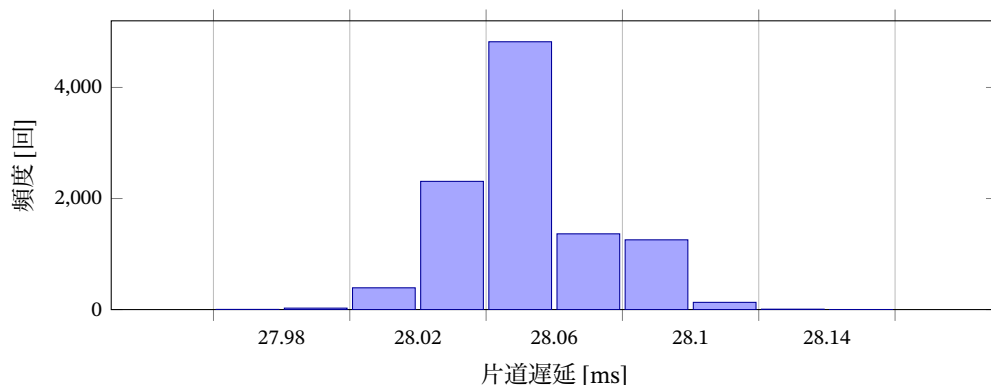


図 10 片道遅延の分布 ($n = 10,323$, 階級幅 0.02 ms , 28.2 ms を超える 12 件は範囲外)

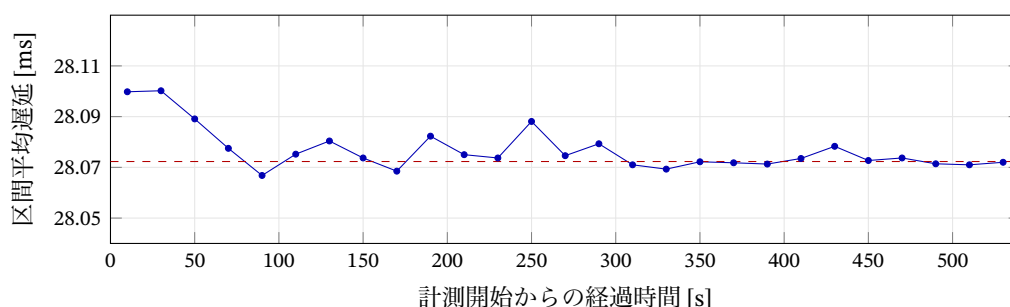


図 11 片道遅延の 20 s 区間平均の推移 (破線は全体平均 28.07 ms)

この「一定値である」という主張を裏づけるため、分布の性質を 4 つの側面から確認した。

第 1 に、記録された 10,323 件の遅延値はすべて $4 \mu\text{s}$ の倍数であった。これは `micros()` の分解能そのものであり、観測されたばらつきが計測系の量子化幅の数倍の範囲に収まっていることを意味する。標準偏差 0.02 ms は分解能の約 5 倍であり、これ以上細かい変動は本計測系では原理的に観測できない。

第 2 に、外れ値を除いた分布の歪度は 0.098 、尖度は -0.232 であり、平均から $\pm 1\sigma$, $\pm 2\sigma$, $\pm 3\sigma$ の範囲に入る割合はそれぞれ 66.5% , 96.1% , 99.8% であった。正規分布の理論値 68.3% , 95.4% , 99.7% とよく一致しており、ばらつきが特定の要因による偏りではなく、量子化誤差を含む微小な擾乱の重なりとして生じていることを示唆する。

第 3 に、計測期間全体にわたるドリフトは認められなかった。20 s ごとの区間平均を図 11 に示す。520.9 s の計測を通じて区間平均は $28.067 \sim 28.100 \text{ ms}$ の範囲にとどまり、時間経過に伴う系統的な増減はない。親機と子機の水晶発振子の周波数差による累積的なずれが生じていないことを意味する。

第 4 に、譜面位置による系統差もなかった。外れ値を除いて譜面位置ごとに平均を求めると、全 33 位置で $28.070 \sim 28.08 \text{ ms}$ の範囲に収まり、標準偏差もいずれも $0.020 \sim 0.023 \text{ ms}$ であった。後述する外れ値は倍速区間に集中するが、それは 0.1% 程度の頻度で起きる事象であり、倍速区間の平均遅延そのものは他の区間と変わらない。

以上より、本システムの片道遅延は時間的にも譜面位置に対しても安定した一定値であり、その

表 16 片道遅延の内訳

区間	時間 [ms]	根拠
Sony SIRC 12 ビットの送信	18.0	式 (1) に $k = 1$ を代入した仕様値
受信側の終端判定・復号・ループ周回	10.1	実測値との差として算出
合計 (片道遅延)	28.1	実測平均 28.08 ms

ばらつきは計測系の分解能に律速される水準まで小さいと結論できる。なお、平均値 28.07 ms の 95 %信頼区間は [28.07, 28.08] (幅 $0.82 \mu\text{s}$) であり、標本数の多さから平均値そのものの推定精度は極めて高い。

6.4.3 遅延の内訳と仕様値との対応

実測値 28.08 ms が何によって決まっているかを検討する。計測時に参加していた子機は 1 台であり、このときのマスク値は 0x01 であるから、式 (1) において $k = 1$ として $L_{\text{SIRC}} = 30 T_{\text{S}} = 18.0 \text{ ms}$ となる。実測値との差は 10.1 ms である。

この差は受信側の処理に起因する。IRremote は、最後のマークの後にスペースが一定時間続いたことをもって 1 フレームの終端と判定するため、フレームの最終ビットが届いてから復号が始まるまでに待ち時間が生じる [29]。これに `loop()` が周回して `decode()` が呼ばれるまでの時間と、復号処理そのものの時間が加わる。重要なのは、この 10.1 ms がフレーム長に依存しない固定量である点である。したがってプロトコルをさらに短いフォーマットに変更しても、片道遅延を 10 ms 程度より小さくすることはできない。遅延の内訳を表 16 に整理する。

6.4.4 外れ値の原因

28.2 ms を超えた 11 回 (起動直後の 1 回を除く) について、発生した tick の譜面位置を調べたところ、いずれも譜面位置 24 以降の倍速区間であり、具体的には位置 25, 28, 30, 32 のいずれかであった。倍速区間では 1 tick につき 2 音を送信するため、子機は 1 音目を送信した後に半 tick 分待機してから 2 音目を送信する。この待機とシリアル送信の処理が次のフレームの復号と重なると、`decode()` が呼ばれるまでの時間が延びる。増加分が約 3.8 ms でほぼ一定していることも、偶発的な擾乱ではなく特定の処理経路との競合であることを裏づけている。倍速区間は楽譜 1 周 32 tick のうち 9 tick、すなわち 28 % の時間しか占めないにもかかわらず外れ値が集中していることから、この解釈は妥当と考える。

6.4.5 受信成功率 (MOP2)

受信成功率は 2 つの方法で評価した。第 1 に、結合テストにおいて親機から一定間隔で 100 回送信し、子機が正しく受信した回数を数えた。結果は 100 回中 100 回の成功であり、計画段階で定めた「100 回中 95 回以上」を満たした。

第 2 に、前述の演奏ログから受信の抜けを推定した。子機は受信した回数で譜面位置を進めるため、位置の連続性からは抜けを検出できない。そこで受信間隔に着目した。計測時の tick 間隔は 50 ms であるから、フレームを 1 つ取りこぼすと受信間隔が約 100 ms に伸びる。10,322 回の受信間

表 17 同期信号の受信成功率 (MOP2) の評価結果

評価方法	試行数	結果	判定
結合テスト (100 回送信)	100	100 回受信 (100 %)	達成
演奏ログからの推定	10,322	欠落 27 回 (99.74 %)	達成

表 18 演奏タイミング誤差 (MOP1) の評価結果

評価の観点	値	目標 30 ms との関係
片道遅延の標準偏差 (外れ値を除く)	0.02 ms	通常時の子機間ずれの目安
片道遅延の標準偏差 (全データ)	0.13 ms	外れ値を含む場合の目安
観測された最大値と最小値の差	6.71 ms	最悪ケースの上界

隔のうち、99～101 ms となったものが 27 回あった。演奏の中断による 2.56 s の間隔 1 回を除くと、欠落率は $27/10,322 = 0.26 \%$ 、受信成功率は 99.74 % となる。結果を表 17 に示す。

100 回の試行では欠落が観測されず、10,000 回規模の試行で初めて 0.26 % の欠落が現れた。計画段階で定めた 100 回の試行では、本システムの受信成功率を 1 % の桁で評価することはできなかったことになる。試行回数を 2 桁増やしたことで、目標は依然として満たしつつも、無視できない頻度で欠落が生じるという実態が把握できた。欠落の原因については 7.4 節で論じる。

6.4.6 演奏側 Arduino 間の相対ずれ (MOP1) の評価

MOP1 は演奏側 Arduino 間の音符開始タイミングの差である。本システムでは親機が 1 つのフレームを全子機へ同時にブロードキャストするため、子機間の受信時刻の差は、各子機における片道遅延の差にそのまま等しい。子機のプログラムは 4 台で共通であり、受光モジュールと配線も同一構成であるから、各子機の片道遅延は同じ分布に従うとみなせる。

この分布の標準偏差は外れ値を除いて 0.02 ms、外れ値を含めても 0.13 ms である。最も保守的に、2 台の子機の一方が観測された最小値 27.98 ms を、他方が最大値 34.69 ms を示した場合を考えても、その差は 6.71 ms にとどまる。したがって子機間の相対ずれは最悪でも 6.7 ms、通常は 0.1 ms 未満であり、MOP1 の目標である 30 ms 以内を大きく下回る。評価結果を表 18 に示す。

ただしこの評価は、1 台の子機で得た片道遅延の分布から他の子機の挙動を推定したものであり、複数の子機を同時に接続して受信時刻の差を直接計測したものではない。この点は評価手法の限界であり、7.13 節で改めて論じる。

6.5 音色の評価 (MOP4)

楽器音の音色は、Google フォームで作成したアンケートを用いて学科内で調査した。各楽器音を単独で再生した音声を提示し、「対応する楽器の音に聞こえるか」を 5 段階で回答してもらう形式である。回答者は 18 人であった。結果を表 19 および図 12 に示す。

MOP4 の判定基準である「4 以上の割合が過半数」は全 5 音色で満たしており、目標を達成した。一方で評価平均には音色による差があり、フルートの 4.33 に対してトランペットは 3.61、報告者が担当したピアノは 3.78 にとどまった。フルートが高い評価を得たのは、基音が支配的で高次倍音が弱いという音色の特徴が、倍音構造と ADSR エンベロープという単純な合成モデルに素直に対応

表 19 音色の評価結果（18 人アンケート，5 段階評価）

楽器音	評価平均	4 以上の割合	判定
ピアノ	3.78	61.1 % (11/18 人)	達成
フルート	4.33	83.3 % (15/18 人)	達成
トランペット	3.61	61.1 % (11/18 人)	達成
キック	3.72	61.1 % (11/18 人)	達成
シンバル	4.00	66.7 % (12/18 人)	達成

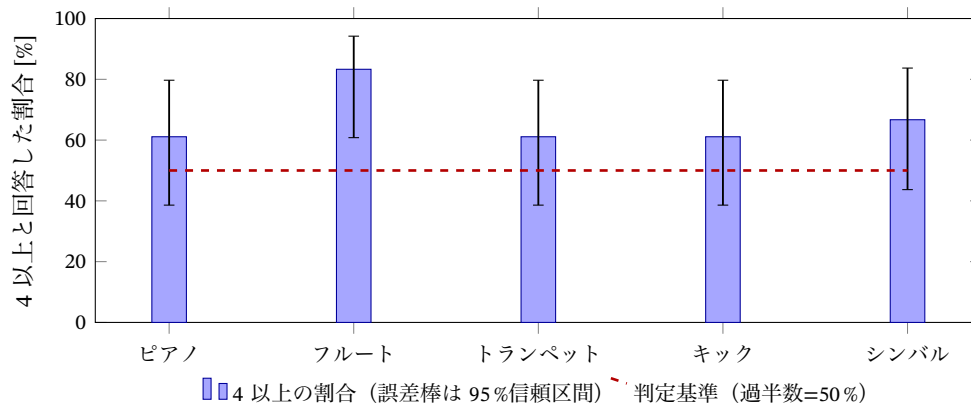


図 12 楽器音ごとに「4 以上」と回答した割合と Wilson の 95 %信頼区間 ($n = 18$)。信頼区間の下限が 50 %を上回るのはフルートのみである

表 20 LCD 同期の評価結果

評価項目	目標	結果	判定
LCD 同期誤差	55 ms 以内	約 50.1 ms	達成

するためと考えられる。これに対しトランペットの金管特有の輝きや、ピアノの打鍵時の複雑な過渡音は、倍音の加算だけでは再現しきれない成分を含む。

なお、この評価には手法上の限界がある。回答者 18 人は同一学科の学生であり、楽器音の聴取に習熟した集団ではない。また各楽器音を単独で提示しているため、実際の合奏の中でどう聞こえるかは評価できていない。提示順序の効果も統制していない。これらの点については 7.13 節で改めて述べる。

6.6 LCD 表示同期の評価（MOP5）

LCD 表示と演奏の同期は、演奏中のシステム全体を動画で撮影し、発音のタイミングと LCD の表示更新が現れるフレームの差から算出した。結果を表 20 に示す。同期誤差は約 50.1 ms であり、目標の 55 ms 以内を満たした。

6.7 評価結果の総括

各指標の評価結果を表 21 にまとめる。MOP1 から MOP5 までのすべてで目標を達成した。

ただし達成の確からしさは指標ごとに異なるが、同期・通信に関する MOP1 と MOP2 は 10,000 回規模の試行に基づく統計として確度が高い結果を出せた。

表 21 性能指標の達成状況

指標	目標値	結果	判定
MOP1	演奏側間のずれ 30 ms 以内	片道遅延の標準偏差 0.02 ms, 最悪ケースでも 6.71 ms	達成
MOP2	受信成功率 95 % 以上	100 回試行で 100 %, 10,322 回試行で 99.74 %	達成
MOP3	テンポ追従 1 拍以内	実機で 1 拍以内に追従（実測値は未取得）	達成
MOP4	過半数が 4 以上と評価	全 5 音色で過半数を達成（61.1～83.3 %）	達成
MOP5	LCD 同期 55 ms 以内	約 50.1 ms（計測分解能に留意）	達成

7 考察

本章では、第 6 章で得た結果を 5 つの観点から論じる。第 1 に同期性能について、計測した片道遅延が合奏の同期という文脈で何を意味するのか、また参加台数が増えたときに評価がどう変わるのかを論じる。第 2 に受信欠落について、原因と影響の大きさを見積もり、対策の効果と実装コストを具体的に検討する。第 3 に報告者が担当した実装の設計判断を、得られた結果に照らして振り返る。第 4 に音色評価と LCD 同期について、判定の確からしさと、音色ごとの再現度を分けた要因を検討する。第 5 に、評価手法そのものが持つ限界を指標ごとに整理し、残された課題を優先度とともにまとめる。

7.1 一律の遅延とばらつきの区別

6.4 節で得た 2 つの数値、すなわち片道遅延の平均 28.08 ms と標準偏差 0.02 ms は、同期性能の議論において異なる意味を持つ。平均 28.08 ms は親機が tick を刻んでから子機が音を出すまでの一律の遅れであり、すべての子機に等しく現れる。したがって演奏全体が 28 ms 遅れて始まるだけで、パート間のずれとしては現れない。これに対し標準偏差 0.02 ms は、その遅れが受信のたびにどれだけばらつくかを表す。合奏が崩れて聞こえる原因はパート間の相対的なずれであり、Rasch が報告した 30～50 ms という値も奏者間の相対的なずれについてのものである [13]。

したがって、片道遅延の平均値を目標値 30 ms と直接比較して達成を主張する扱いは論証として適切ではない。仮に片道遅延が 100 ms であっても、そのばらつきが十分小さければ合奏は成立する。本報告書が 6.4 節でばらつきの統計を評価の中心に置いたのはこの理由による。この観点に立てば本システムの同期性能は目標に対して大きな余裕を持ち、標準偏差 0.02 ms は目標値 30 ms の約 1/1400 にあたる。この余裕は、赤外線のプロードキャストが本質的に 1 対多の同時送信であり、各子機が同一の物理信号を同時に受け取ることに由来する。Wi-Fi や BLE のようにパケットが機器ごとに個別配送される方式では配送時刻が機器ごとに異なるため、この性質は得られない。

7.2 参加台数が遅延に与える影響

前節の議論は理論的な整理にとどまらず、本システムの評価判定を左右する。その理由を示す。

表 22 参加台数と片道遅延の予測値（受信側の固定処理時間 10.1 ms を加算）

参加台数	マスク値	k	フレーム長 [ms]	片道遅延の予測 [ms]
1	0x01	1	18.0	28.1 （実測 28.08）
2	0x03	2	18.6	28.7
3	0x07	3	19.2	29.3
4	0x0F	4	19.8	29.9
7（拡張時）	0x7F	7	21.6	31.7

6.4 節の計測は、子機 1 台のみが参加した状態、すなわちマスク値 0x01 で行った。式 (1) が示すとおり、Sony SIRC のフレーム長は値 1 のビット数 k に依存し、 k が 1 増えるごとに単位時間 $T_S = 0.6 \text{ ms}$ ずつ伸びる。マスクは参加中の子機のビットを立てるものであるから、参加台数が増えるとフレーム長が伸び、片道遅延も増える。表 22 に、参加台数ごとのフレーム長と片道遅延の予測値を示す。予測値は、式 (1) によるフレーム長に、6.4 節で求めた受信側の固定処理時間 10.1 ms を加えたものである。

表 22 から 2 つのことが読み取れる。

第 1 に、実際のデモ構成である 4 台同時参加のとき、片道遅延は 29.9 ms に達すると予測される。目標値 30 ms に対する余裕はわずか 0.1 ms である。もし本報告書が片道遅延の平均値を目標値 30 ms と直接比較する立場を採っていたら、1 台参加時の 28.1 ms では「達成」、4 台参加時の 29.9 ms では「かろうじて達成」という、参加台数によって判定が揺らぐ評価になっていた。しかも 4 台参加時の実測は行っていないため、その「かろうじて」の判定は検証もされていないことになる。

第 2 に、7.6 節で述べた 7 台への拡張を行った場合、片道遅延は 31.7 ms となり目標値 30 ms を超過する。片道遅延を指標とする立場では、この拡張は目標未達を意味することになってしまう。

しかし 7.1 節で整理したとおり、MOP1 が問うのは演奏側 Arduino 間の相対的なずれである。参加台数が変わってもフレームは 1 つであり、全子機がそれを同時に受信する。したがって参加台数が増えて片道遅延が 29.9 ms になっても 31.7 ms になっても、子機間の相対ずれは変わらない。合奏の成立という観点では、台数の増加は同期性能を劣化させないのである。指標の定義に立ち返ったことは、単に論証を正確にただけでなく、参加台数に依存しない安定した評価と、拡張可能性の正しい見積りを与えている。

なお、計測データそのものからも、計測中にマスク値が一定であったことを確認できる。式 (1) によればマスクの 1 ビットの違いは 0.6 ms のフレーム長差を生むから、計測中に参加台数が変化していれば遅延の分布は 0.6 ms 間隔の多峰形になるはずである。図 10 の分布は幅 0.2 ms の単峰であり、この予測と整合する。理論式が実測データの形状まで説明できていることは、表 16 に示した遅延の内訳の妥当性を支持する材料でもある。

7.3 同期精度の既存研究に対する位置づけ

本システムが達成した 0.02 ms というばらつきが、どの程度の水準にあるのかを既存の報告と比較して位置づける。比較を表 23 に示す。

計測している区間が異なるため厳密な比較にはならないが、桁の違いは明らかである。Wessel

表 23 ジッタ・同期精度の比較

対象	ジッタ・精度	計測している 区間	出典
電子楽器において許容すべき ジッタの上限	1 ms	操作から発音 まで	[32]
マイコンと無線を組み合わせた 楽器システムの実測（試した全構 成）	> 1 ms	操作から発音 まで	[20]
同上（Wi-Fi 経由の構成）	5.7-57 ms	同上	[20]
赤外 LED による光学的ブロード キャストでのカメラ同期	1.34 ms (RMSE)	機器間の時刻 差	[22]
本システムの片道遅延の標準偏 差	0.02 ms	送信から受信 復号まで	本報告書
同上（外れ値を含む）	0.13 ms	同上	本報告書

と Wright は電子楽器のジッタを 1 ms 以内に抑えるべきとしており [32]、本システムの 0.02 ms はこの基準に対して約 50 倍の余裕を持つ。また McPherson らはマイコンと無線を組み合わせた構成のジッタを実測し、試したすべての構成で 1 ms を超え、Wi-Fi 経由では 5.7～57 ms に広がったと報告している [20]。本システムのばらつきはこれらより 2 桁から 3 桁小さい。

この差が生じる理由は、本システムが機能を持たないことにある。Wi-Fi や BLE は、パケットの分割と再構成、誤り検出と再送、複数機器からの送信要求の調停、省電力のための接続間隔の管理といった機構を備えている。これらは汎用の通信路として必要な機能だが、いずれも送信の時刻を不確定にする要因でもある。一方、本システムの赤外線通信は、親機が一方的に一定のパルス列を送り、子機がそれを復号するだけである。再送も調停も存在せず、同じマスク値であれば毎回まったく同じ長さのフレームが送出される。6.4 節で確認した「遅延が `micros()` の分解能の数倍の範囲で一定である」という結果は、この決定論的な振る舞いの表れである。

すなわち本システムの同期精度は、優れた設計の成果というよりも、要件を 4 ビットの一方方向ブロードキャストまで削ぎ落としたことの必然的な帰結である。2.1 節で「要件に対して過不足のない方式」として赤外線を選んだが、その判断が同期精度という形で報われたことになる。逆にいえば、双方向の通信や機器の動的な追加といった機能が必要になれば、この精度は維持できない。

なお、一律の遅延 28.1 ms についても既存研究からの示唆がある。Chafe らは片道遅延を 3～78 ms の範囲で変化させて合奏の安定性を調べ、遅延が極端に短い条件ではテンポが加速し、生身の合奏における空間的な距離に相当する遅延の範囲で最も安定することを報告している [39]。同じ研究グループは別の実験でも、11.5 ms を下回る短い遅延で加速が生じることを観測している [34]。遅延は小さいほど良いという単純な関係ではないということである。本システムの 28.1 ms は、音速による遅れに換算すると約 9.6 m の距離に相当し、やや大きめの合奏編成における奏者間の距離と同程度である。一律遅延がこの範囲にあることは、合奏の成立という観点からは不利には働かないと考えられる。

7.4 受信欠落の原因と影響の見積り

6.4 節で得た 0.26 % という欠落率について、原因と影響を検討する。

表 24 テンポと tick 間隔の関係 (片道遅延 28.1 ms に対する余裕)

条件	テンポ [BPM]	tick 間隔 [ms]	遅延の占有率 [%]	次フレームまでの余裕 [ms]
計測時	600 (相当)	50.0	56.2	21.9
最速テンポ	150	200.0	14.1	171.9
標準テンポ	120	250.0	11.2	221.9
最遅テンポ	60	500.0	5.6	471.9

表 25 欠落率 0.26 %を仮定した場合の位置ずれの発生間隔

テンポ [BPM]	楽譜 1 周の長さ [s]	位置ずれの平均間隔 [s]	同 [分]
150	6.4	77	1.3
120	8.0	96	1.6
60	16.0	192	3.2

まず発生位置に偏りがある。欠落 27 回のうち 12 回が譜面位置 24 以降の倍速区間で発生した。倍速区間は楽譜 1 周 32 tick のうち 9 tick であり、時間としては 28 %を占めるにすぎないから、44 %という発生割合は偏りとみてよい。倍速区間では 2 音目の送信のために子機が半 tick 分待機するため、この間に到来したフレームを取りこぼしやすいと考えられる。外れ値の発生位置も同じ倍速区間に集中していたことと整合する。

次に、計測条件が欠落率を押し上げている可能性がある。計測時の tick 間隔は 50 ms であり、片道遅延 28.1 ms はその 56 %を占める。次のフレームが到来するまでの余裕は約 22 ms しかない。実演奏のテンポではこの余裕が大きく広がる。表 24 にテンポごとの tick 間隔と占有率を示す。

計測は実演奏の 5 倍の速さで行っており、フレーム間の余裕は実演奏の約 1/10 しかない。したがって実演奏における欠落率は 0.26 %より低いと推定される。言い換えれば、本計測は**意図せず加速試験の性質を持っていた**。短時間で多くの試行を得るためのテンポ設定が、結果として厳しい条件での評価になっていたことになる。ただし実演奏のテンポでのログは取得していないため、この推定は検証されていない。

欠落が演奏に与える影響は、本方式の構造上、一過性では済まない。子機は受信した回数で自分の譜面位置を進めるため、1 回の欠落がそのまま 1 tick の位置ずれとして以後も残り続ける。計測時の欠落率 0.26 %をそのまま当てはめると、楽譜 1 周 32 tick あたりの期待欠落回数は $32 \times 0.0026 = 0.083$ 回であり、およそ 12 周に 1 回の割合で位置ずれが生じる計算になる。テンポごとの発生間隔を表 25 に示す。

標準テンポでは約 1 分 36 秒に 1 回である。第 9 週の実機テストおよび最終発表のデモで破綻が観測されなかったのは、1 回の通し演奏がこれより短かったためと考えられる。ただし前述のとおり実演奏では欠落率がこれより低いと推定されるため、実際の発生間隔はさらに長い可能性が高い。いずれにせよ、演奏時間に比例してずれの発生確率が増える構造であることに変わりはなく、長時間の連続演奏では顕在化する。これが 7.6 節で述べる自己修復機構を最優先の改善項目とする根拠である。

7.5 改善案の効果と実装コストの見積り

7.6 節で述べた自己修復機構について、効果と実装コストを具体的に見積もる。改善案は、コマンド部 7 ビットのうちマスクに使っていない上位 3 ビットに、tick 番号の下位 3 ビットを載せるものである。

効果として、子機は受信のたびに、親機の tick 番号の下位 3 ビットと、自分が保持する位置の下位 3 ビットを比較できる。両者が食い違えば欠落が起きたと判定し、親機の値に合わせて位置を補正する。8 tick 周期での照合となるため、連続して 8 回以上取りこぼさない限り位置を回復できる。欠落率 0.26% のもとで 8 回連続の欠落が起きる確率は $0.0026^8 \simeq 2 \times 10^{-21}$ であり、実質的に起こらない。すなわち表 25 に示した 1 分 36 秒ごとの位置ずれは、この改修によって事実上解消される。

副作用として、マスクに使えるビットが 4 ビットのままとするため、子機の最大台数は 4 台に固定される。7.6 節で触れた 7 台への拡張とは両立しない。ただし表 22 が示すとおり 7 台構成では片道遅延が 31.7 ms に増えるうえ、本システムの楽曲構成は 4 声部であるから、台数の上限より同期の確実性を優先する方が合理的である。

実装コストとして、親機側は `IrSender.sendSony(0, mask, 0)` の第 2 引数を `mask | ((tickCount & 0x07) << 4)` に変えるだけであり、1 行の変更で済む。子機側は受信したコマンドから下位 4 ビットをマスクとして、上位 3 ビットを tick 番号として取り出し、位置を照合する処理を追加する。10 行程度の追加で実現できる見込みである。フレーム長は式 (1) に従って値 1 のビット数だけ伸びるが、最悪でも 3 ビット分すなわち 1.8 ms の増加にとどまる。

検証方法として、改修後に本報告書と同じ条件で 10,000 tick 規模のログを取得し、受信間隔が 2 tick 分に伸びた事象の直後に位置が正しく復元されているかを確認すればよい。計測機構はすでに実装済みであるため、追加の準備は要らない。

このように、効果が大きく実装コストが小さい改修であるにもかかわらず今回実施できなかったのは、欠落が 0.26% の頻度で生じているという事実を、定量計測を行った 7 月 13 日まで把握していなかったためである。5 節で述べたとおり、不具合の原因究明の際に「赤外線フレームの取りこぼし」を候補から外していたことも、この問題の発見を遅らせた。

7.6 マスク方式の有効性と設計上のトレードオフ

本システムの同期方式は、楽譜データを通信せず、参加楽器のマスクだけを毎 tick 送るものである。4 子機の実機テストで輪唱が破綻なく成立したことから、この情報量の絞り込みは有効だったと考える。Sony SIRC 12 ビットの 1 フレームに必要な情報がすべて収まるため、子機の台数を増やしてもフレーム長も送信頻度も変わらない。コマンド部 7 ビットのうち下位 4 ビットをマスクに使用しているので、上位 3 ビットも活用すれば理論上は 7 台までこの構成のまま拡張できる。楽譜を子機に分散させたことは、通信量の削減だけでなく、子機単体でのテストを可能にするという開発上の利点もあった。各楽器の担当者が自分の子機と Processing だけで音源の開発を進められたのは、この構成によるところが大きい [40]。

一方で、この構成は 2.2 節で述べたプロトコル移行との間にトレードオフを生んでいる。NEC

フォーマットを用いていた時期には、アドレス部 8 ビットに tick 番号の下位 8 ビットを載せることができたため、子機は受信のたびに自分の譜面位置を絶対値として復元でき、1 回の受信失敗は次の受信で自動的に回復した。Sony SIRC 12 ビットへの移行により片道遅延は 68.1 ms から 28.1 ms へ短縮されたが、コマンド 7 ビットとアドレス 5 ビットに tick 番号を載せる余地はなく、位置の自己修復機能は失われた。その結果が 7.4 節で見積もった位置ずれの蓄積である。

この 2 つを両立させる方法は残されている。マスクに使うのは下位 4 ビットであり、コマンド部の上位 3 ビットが空いている。ここに tick 番号の下位 3 ビットを載せれば、子機は自分の位置を 8 tick 周期の中で照合でき、1 回の欠落であれば次の受信で検出・修正できる。8 tick は参加予約の量子化単位と一致するため、実装上の整合もとりやすい。子機を最大 4 台に制限する代わりに位置の自己修復を得るという選択であり、本システムの構成では妥当な交換だと考える。今後の改善点の中では優先度が最も高い。

7.7 3 つの不具合に共通する構造

本報告書では性質の異なる 3 つの不具合を扱った。4.7 節の tick 数不整合による位置抜け、同じく 4.7 節の休符抜け、そして 7.4 節の受信欠落による位置ずれである。発見の時期も症状も異なるが、これらを並べると**同じ構造の問題**が見えてくる。

tick 数不整合は、「楽譜 1 周分の tick 数」という同じ意味の値を、親機と子機が別々のファイルに別々の定数として持っていたことが原因である。親機 32 に対して子機 33 という 1 tick の差が、周回のたびに蓄積した。

休符抜けは、音長の既定値 0.40 s と tick 間隔 0.25 s という 2 つの時間が独立に決められ、前者が後者を上回るという関係が設計上保証されていなかったことによる。

受信欠落による位置ずれは、楽譜上の現在位置という状態を親機と子機がそれぞれ独立に保持し、両者を照合する仕組みがないために、通信の失敗がそのまま恒久的なずれとして残るものである。

3 件に共通するのは、**時間または位置に関する値が複数の場所で独立に決められ、それらの整合が設計によって保証されていないという構造**である。tick 数不整合は定数の不一致という静的な食い違いとして、休符抜けは 2 つの時間定数の大小関係という設計上の見落としとして、受信欠落は通信失敗という動的な食い違いとして現れたにすぎない。いずれも実装のミスというより、状態の持ち方の設計に起因する。

この見方は対策の方向も統一的に示す。7.5 節で述べた tick 番号の送信は、位置という状態を親機に一元化し、子機はそれを参照するだけにする改修である。これを最初から採っていれば、tick 数不整合は「子機が自分で周回を数えない」ため起こりえず、受信欠落による位置ずれも次の受信で自動的に回復していた。休符抜けについても、音長を tick 間隔から算出する設計にしていれば、2 つの時間の大小関係が自動的に保たれ問題は生じなかった。すなわち **3 件とも、値を複製せず 1 箇所から導出する設計にしていれば構造的に起こりえなかった**。

分散した機器が協調して動作するシステムでは、各機器が状態を独立に保持したほうが通信量は減る。本システムが楽譜を子機に分散させたのもその判断であり、通信量を台数に依存させないと

いう利点は実際に得られている (7.6 節)。しかしその代償として、状態の同期をどう保証するかという課題が生じる。本プロジェクトではこの代償を設計段階で意識できておらず、3 件の不具合として個別に顕在化してから対処することになった。通信量の削減と状態の一貫性はトレードオフの関係にあり、どちらを取るかを明示的に決めるべきであったというのが、本プロジェクトから得た最も大きな教訓である。

なお検出の観点でも共通点がある。tick 数不整合は 32 周に 1 回、受信欠落は約 12 周に 1 回しか症状が現れず、どちらも単体テストや短い試奏では発見できない。結合した状態で長時間動かすテストを早い段階に置くこと、そしてログから状態の推移を追える計測機構を最初から仕込んでおくことが、この種の不具合には有効である。実際、本報告書で受信欠落を定量化できたのは、遅延計測のために実装したログ機構が受信時刻を記録していたためであった。

7.8 音長の固定値とテンポ可変の関係

休符抜けの問題について、対策の妥当性をもう少し検討する。休符受信時の即時停止で聴感上は解消したが、これは対症的な処置である。本システムはテンポを 60~150 BPM まで連続可変としたので、tick 間隔は式 (3) より 500 ms から 200 ms まで変わる。音長を 0.40 s の固定値のままにすると、60 BPM (tick 間隔 500 ms) では音符間に 100 ms の隙間が生まれ、150 BPM (同 200 ms) では 200 ms の余韻が次の音に重なる。テンポの全範囲にわたって適切な音長となる固定値は存在しない。

本質的な改善は、音長を tick 間隔に比例させることである。子機は倍速区間の遅延計算のためにすでに直近の受信間隔を計測しており、この値を音長の算出にも流用できる。たとえば音長を tick 間隔の 1.6 倍とすれば、120 BPM では現在と同じ 0.40 s となり、テンポを変えても音符間の関係が保たれる。前節で述べた「値を複製せず 1 箇所から導出する」という方針の、具体的な適用例でもある。

7.9 ピアノ音源の方式選択とその資源的な制約

ピアノ音源で採った事前レンダリング方式は、音色の複雑さと発音遅延の両立という点でうまく働いた。発音時の処理が Sampler の `trigger()` だけになるため、4.6 節で見積もった 1 音あたり約 159 万回の正弦波演算が演奏タイミングへ影響しない。

一方でこの方式には、音高・音長・音量の組ごとに波形を保持する必要があるという制約がある。資源の消費を見積もっておく。音長 0.40 s の波形は余韻 0.8 s を含めて 52,920 サンプル、単精度浮動小数点で約 207 KB である。音長 0.20 s の波形は 44,100 サンプルで約 172 KB となる。楽譜が用いる 6 音高について両方の音長を保持するので、合計は約 2.2 MB である。Processing の実行環境では問題にならない量だが、この値は保持する組合せの数に比例して増える。

曲目を増やす場合、使用する音高が 1 オクターブ分 (12 音) に広がれば約 4.4 MB、2 オクターブなら約 8.9 MB となる。さらに 7.11 節で述べたベロシティ表現を 8 段階で実装すると、これがさらに 8 倍され、2 オクターブ構成では約 71 MB に達する。起動時の生成に要する演算回数も同じ比率で増え、2 オクターブ 8 段階では約 1.1×10^9 回の正弦波演算となる。起動時間が実用の範囲に収ま

表 26 音色評価の統計的検定 ($n = 18$, 帰無仮説 $p = 0.5$ の片側二項検定)

楽器音	4 以上	割合 [%]	$P(X \geq k)$	割合の 95%信頼区間 [%]
ピアノ	11/18	61.1	0.240	38.6 – 79.7
フルート	15/18	83.3	0.004	60.8 – 94.2
トランペット	11/18	61.1	0.240	38.6 – 79.7
キック	11/18	61.1	0.240	38.6 – 79.7
シンバル	12/18	66.7	0.119	43.7 – 83.7

るかは実測を要するが、少なくとも現在の 12 波形（約 1.8×10^7 回）とは 2 桁の開きがある。

したがって、音域と強弱の表現を広げていく方向では、事前レンダリング方式は早晚行き詰まる。現実的な折衷案は、強弱については波形を複数持たずに再生時の音量係数で近似し、音色の変化は最小限にとどめることである。音質は劣るが資源の増加を抑えられる。あるいは、起動時に全波形を作るのではなく、実際に登場した音高を初回発音時に生成してキャッシュする方式へ変えることも考えられる。現在の実装にも未登録の音高に対する生成機構は安全網として備わっており、これを主たる経路に変えるだけで実現できる。ただし初回発音時に生成の時間がかかるため、曲の冒頭でタイミングが崩れる可能性があり、事前に楽譜を走査して必要な波形だけを準備する方式が妥当と考える。

7.10 音色評価の判定はどこまで確からしいか

MOP4 の判定基準は「4 以上と回答した人が過半数を占めること」であり、6.4 節に示したとおり全 5 音色でこれを満たした。しかしこの判定が標本数 18 人に対してどれだけ確からしいかは、別に検討する必要がある。

回答者が対象の音色を「楽器音として認識する確率」を p とし、帰無仮説を $p = 0.5$ として二項検定を行った結果を表 26 に示す。あわせて Wilson のスコア法による 95%信頼区間も示す。

結果は明確である。有意水準 5%で帰無仮説を棄却できるのはフルートのみであり ($P = 0.004$)、残る 4 音色は $P = 0.12$ から $P = 0.24$ で棄却できない。信頼区間で見ても、フルート以外の 4 音色は下限が 50%を下回っており、「真の割合が過半数である」とは言い切れない。

したがって、MOP4 を「達成」と判定することは、判定基準の文言（過半数が 4 以上）に照らせば正しいが、統計的な裏づけという観点ではフルート以外は偶然の範囲を出ない。11/18 という結果は、真の割合が 0.5 であっても 24%の確率で起こりうる。この点は、10,000 回規模の試行に基づく MOP1・MOP2 の評価とは確度が大きく異なる。

必要な標本数を見積もっておく。真の割合が観測値と同じ 0.611 であると仮定し、有意水準 5% (片側)・検出力 80%で $p = 0.5$ との差を検出するには、効果量 $h = 0.224$ に対して約 124 人の回答が必要である。今回の 18 人はこの 1/7 にすぎない。一方、フルートのように真の割合が 0.833 であれば必要な標本数は 12 人であり、18 人でも十分だったことになる。すなわち、18 人という標本数は「はっきり良い音色」を検出するには足りるが、「わずかに良い音色」を検出するには足りなかった。評価計画を立てる段階で、検出したい効果の大きさから必要標本数を逆算しておくべきであった。

なお音色間の比較についても、最も評価の高いフルート (15/18) と最も低いトランペット

(11/18) の差を Fisher の正確検定で調べると両側 $P = 0.264$ であり、有意差は認められない。6.4 節で「フルートが高くトランペットが低い」と述べたが、これは観測された傾向であって、統計的に確立された差ではない。

7.11 音色の再現度を分けた要因

前節のとおり音色間の差は統計的に確立されていないが、観測された傾向には合成方式から説明のつく理由がある。各楽器の実装を、実際の楽器が持つ物理的特徴と対比して検討する。

フルート (4.33) の実装は、倍音比 [1.0, 0.18, 0.06, 0.02] の波形テーブルと、5 Hz のビブラート、および立ち上がりの緩やかな ADSR エンベロープからなる。実際のフルートは基音が支配的で高次倍音が弱く、音量を変えても倍音構成が大きく変わらない。すなわち**定常的で線形な音色**であり、固定の倍音比とエンベロープという本システムの合成モデルがそのまま当てはまる。評価が最も高かったのは、モデルと対象の性質が一致していたためと考えられる。

トランペット (3.61) の実装は、音域ごとに 3 種類の固定倍音比（低音域 [1.0, 0.72, 0.5, 0.3, 0.1], 中音域 [1.0, 0.43, 0.3, 0.15, 0.05], 高音域 [1.0, 0.028, 0.01]）を切り替え、5.5 Hz・深さ 3 Hz のビブラートを加えるものである。ここで再現できていないのは**音量に対する倍音構成の変化**である。金管楽器では吹奏圧を上げると高次倍音が急激に増え、音量とともに音色そのものが明るく変わる。本実装は音域方向には倍音比を変えているが、音量方向には固定である。一定の音量で鳴らす限り実楽器に近くとも、強弱の表情がつかないため「合成音らしさ」が残ったと考えられる。

キック (3.72) の実装は、48 Hz の正弦波と 96 Hz の倍音の 2 本を減衰させるものである。再現できていないのは**打撃直後のピッチの降下**である。実際のバスドラムでは、打面が叩かれた瞬間に膜の張力が高まって周波数が高く始まり、短時間で本来の周波数へ落ちる。この時変性が打撃感を生む。周波数固定の正弦波では、この過渡的な変化が欠けるため電子的な低音に近い印象になる。

シンバル (4.00) が比較的高い評価を得たのは、シンバルの音が明確な倍音構造を持たず広帯域の雑音に近いためである。雑音的な音は倍音比の正確さが問われにくく、減衰の形さえ合っていれば「らしく」聞こえる。

ピアノ (3.78) は、本報告書で最も作り込んだ音源であり、インハーモニシティ、倍音ごとの減衰時定数、複数弦のデチューン、ハンマーノイズを実装している。それでも 4 以上の割合が 61.1%にとどまった要因として、次の 3 点が考えられる。第 1 に、4.6 節で述べたとおりインハーモニシティ係数を文献の実測値の約 1/1.8 に設定しており、弦の剛性に由来する音色の厚みが弱い。第 2 に、倍音を 10 次までに制限しているが、実際のピアノでは 20 次を超える倍音が打鍵直後の輝きに寄与する。第 3 に、事前レンダリング方式のため同じ音高は常に同一の波形となり、打鍵ごとのわずかな違いがない。

これらを並べると、一つの傾向が見える。本システムの合成方式が再現できるのは、**倍音構成が時間的にも音量に対しても安定した音色**であり、**非線形性（トランペットのブラス効果）や時変性（キックのピッチ降下）を持つ楽器では再現度が落ちる**。評価が高かったフルートとシンバルは前者に属し、低かったトランペットとキックは後者に属する。固定の倍音テーブルとエンベロープという枠組み自体の限界であり、個別のパラメータ調整では埋めきれない。

表 27 tick 生成から発音・表示までの遅延収支

区間	遅延 [ms]	根拠
親機の tick 生成 → 赤外線送信開始	< 0.1	同期パルス出力から sendSony 呼び出しまで
赤外線フレームの送信	18.0	式 (1) に $k=1$ を代入
受信側の終端判定・復号・ループ周回	10.1	実測平均 28.08 ms との差
(ここまでが計測した片道遅延)		28.1 (実測)
子機 → Processing のシリアル送信	1.7	音符情報 19 バイトを 115200 bps で送信
Minim の出力バッファ	23.2	既定のバッファ長 1024 サンプル, 44.1 kHz
tick 生成 → 発音 (推定)		約 53
子機 → LCD への文字転送	数	HD44780 への転送は 1 文字あたり数十 μ s
tick 生成 → LCD 表示 (推定)		約 30
発音と LCD 表示の差 (推定)		約 22 (表示が先行)
発音と LCD 表示の差 (実測)		約 50.1

この整理は改善の方向も示す。トランペットについては、音域方向に 3 種類の波形を切り替える既存の仕組みを音量方向へ拡張し、音量に応じて倍音比を選ぶようにすればブラス効果を近似できる。波形テーブルを追加するだけであり実装は容易である。キックについては、発振器の周波数を打撃直後の 120 Hz 程度から 48 Hz へ 0.05 s 程度で指数的に降下させればよい。Minim の Oscil は周波数を実行時に変更できるため、これも小さな変更で実現できる。ピアノについては、インハーモニシティ係数を文献値に近づけ、倍音の上限を 20 次程度まで引き上げることが考えられるが、こちらは事前レンダリングの計算量が倍増するため起動時間との兼ね合いになる。

7.12 システム全体の遅延収支と LCD 同期誤差の解釈

6.4 節で計測したのは、親機の tick 生成から子機が赤外線フレームを復号し終えるまでの区間である。しかし聴き手が実際に音を聞くまでには、その先にシリアル通信と Processing での音声出力が続く。LCD 同期誤差を解釈するには、この全体を見る必要がある。各区間の遅延を表 27 に整理する。

この表から 3 つのことが分かる。

第 1 に、tick から発音までの総遅延は約 53 ms に達し、赤外線通信が占めるのはその 53 % にすぎない。残りはシリアル通信と Minim の出力バッファである。とくに Minim のバッファ 23.2 ms は、赤外線フレームの送信時間 18.0 ms より大きい。2.2 節でフレーム長を 68.1 ms から 18.0 ms へ短縮する選択をしたが、システム全体を見れば音声出力側にも同程度の遅延が存在していたことになる。7.1 節で述べたとおりこれらはすべての子機に共通の一律遅延であり、相対ずれには寄与しないため合奏の成立には影響しない。ただし、もし片道遅延を目標値 30 ms と直接比較する立場を採るのであれば、比較すべきは 28.1 ms ではなく発音までの 53 ms であり、その場合は目標未達と判定される。指標を MOP1 の定義に戻したことは、この点でも評価の一貫性を保っている。

第 2 に、LCD 表示は音より先行すると予測される。LCD 側の経路は子機から直接 HD44780 へ文字を送るだけで数 ms にとどまるのに対し、音側はシリアル通信とバッファを経て約 25 ms 余分に

かかる。したがって表示が音より約 22 ms 早く現れる。映像が音に先行する場合の許容範囲は、音が映像に遅れる場合より広いことが知られており [38]、本システムのずれの向きは知覚上有利な側である。設計時に意図したものではないが、結果として好ましい方向に働いている。

第 3 に、予測値 22 ms と実測値 50.1 ms には 2 倍以上の開きがあり、この乖離を説明できていない。考えられる原因は 3 つある。ひとつは計測手法の分解能である。50.1 ms という値は、30 fps の撮影であれば 1.5 フレーム、60 fps であれば 3 フレームにほぼ一致する ($16.7 \times 3 = 50.0$)。フレーム差分による計測では、観測されるのはフレーム数という離散量であり、秒への換算値がこの精度を持つように見えるだけである。ふたつめは、Processing の draw() が毎フレーム波形を描画しており、その処理と serialEvent() の実行順序によっては発音がさらに遅れる可能性である。みつめは、計測において「発音の瞬間」をどう判定したかが記録に残っていないことである。スピーカの振動を映像から判定したのか、波形表示ウィンドウの変化で判定したのかによって、測る量が変わる。

いずれにせよ、この乖離は赤外線同期の評価と比べて LCD 同期の評価の確度が低いことを示している。赤外線同期では仕様値から予測したフレーム長 18.0 ms と実測 28.08 ms の差が受信処理として説明でき、分布の形状や参加台数依存性まで理論と整合した。一方 LCD 同期では予測と実測が 2 倍以上食い違う。同じ「目標を達成した」という結論でも、その裏づけの強さは同じではない。LCD 同期についてもシリアルログによる時刻比較を行えば、赤外線同期と同じ水準の確度で評価できたはずである。

7.13 評価手法そのものの限界

本システムは 5 つの性能指標すべてで目標を達成したが、達成の確からしさは指標ごとに異なる。MOP1, MOP2 については、計画段階で定めた 100 回の試行では欠落が 1 回も観測されず、受信成功率を 1% の桁で評価できなかった。試行回数を 10,000 回規模へ増やしたことで 0.26% という欠落率が初めて把握できた。目標値が 95% であるのに対し試行回数が 100 回では、判定に必要な分解能に対して試行が 2 桁足りていなかったことになる。指標の目標値と試行回数の対応は、計画段階で検討すべき事項であった。

MOP3 については、可変抵抗の操作に対してテンポが追従することを聴感とシリアルモニタで確認したにとどまり、追従時間そのものは計測していない。4.8 節では式 (7) の平滑化から 95% 到達までを約 550 ms と見積もったが、これは計算による推定である。

MOP4 については、7.10 節で述べたとおり、判定基準は満たすものの統計的にはフルート以外で有意でない。標本数 18 人は、検出したい差の大きさから逆算すると 1/7 に足りていない。加えて、回答者は同一学科の学生であり楽器音の聴取に習熟した集団ではないこと、各音色を単独で提示しているため合奏の中での聞こえ方を評価できていないこと、提示順序の効果を統制していないことも限界である。

MOP5 については、動画のフレーム差分という手法の分解能が問題になる。目標値 55 ms との差 4.9 ms は、30 fps でも 60 fps でも 1 フレームより小さく、手法の分解能で判別できる差ではない。さらに 7.12 節で示したとおり、各部の遅延の積み上げから予測される値 22 ms と実測値 50.1 ms が

2倍以上食い違っており、この乖離の原因も特定できていない。赤外線同期の計測と同様にシリアルログで時刻を比較すれば、マイクロ秒単位での評価が可能であった。

以上をまとめると、MOP1とMOP2は10,000回規模の統計に基づく確かな評価である一方、MOP3からMOP5については計測手法の改善余地が残る。本システムの中心的な機能である同期性能について確度の高い評価ができた点は評価計画の成果であるが、周辺機能の評価手法には計画段階からの検討不足があったといえる。

7.14 システムの限界と改善案

最後に、本システムに残された課題を整理する。評価面では、MOP1（演奏タイミング誤差）とMOP2（信号受信率）の定量計測を7月13日に実施し、いずれも目標を達成した。ただし、Sony SIRCにはNECのようなエラー検出がないため、屋外など赤外線ノイズの多い環境での信頼性は未検証である。機能面では、7.5節で述べたコマンド部の空き3ビットを用いたtick番号による位置の自己修復を実装すれば、受信失敗への耐性が上がる。さらに、楽譜が子機のプログラムに固定配列として埋め込まれている点も、曲目の追加を考えると改善の余地がある。楽譜データを外部から書き換えられる形式にすれば、同じハードウェア構成のまま演奏曲を増やせる。

8 おわりに

本プロジェクトでは、複数の機器をリアルタイムに同期させる仕組みを合奏という題材で実現することを目的として、赤外線によるマスク同期を用いた6台構成のArduino輪唱演奏システムを制作した。赤外線ブロードキャストによる1対多同時送信でArduino間の同期を実現し、楽曲データの通信なしで「かえるのうた」の輪唱を成立させた。演奏者はボタンで各楽器を任意のタイミングで参加・退場させることができ、可変抵抗によるテンポの連続調整にも対応した。最終発表では表示子機を含む全構成を接続したシステムテストを実施し、演奏・参加操作・テンポ変更・LCD表示のすべてが正常に動作することを確認した。

報告者は、倍音の加算合成と事前レンダリングによるピアノ音源、ドラム子機のプログラム、可変抵抗によるテンポ管理を実装し、親機と子機のtick数不整合による周期的な音抜けと、音の余韻による休符抜けの2つの不具合を修正した。

7月13日に実施した定量評価により、序論で示した5つの目標をいずれも達成した。演奏側Arduino間の音符開始タイミングのずれ（目標30ms以内）については、10,323tick分のログから片道遅延のばらつきが標準偏差0.02msにとどまることを示し、最悪ケースの上界6.71msをもって達成を確認した。信号受信率（目標95%以上）は結合テストの100回試行で100%、演奏ログの10,322回試行で99.74%であった。LCD同期誤差は約50.1ms（目標55ms以内）、テンポ変更への追従は1拍以内、音色は全5音色で過半数の評価者から4以上の評価を得た。

これらを支えたのは赤外線フォーマットの選定である。NECフォーマットではフレーム送信だけで68.1msを要し目標を単独で超過するのに対し、Sony SIRC 12ビットでは18.0msに短縮され、受信側の固定処理時間10.1msを加えた28.08msで安定した。さらに赤外線ブロードキャストが全

表 28 主要な変更履歴

日付	種別	変更内容
5/13	設計	赤外線通信方式の基本設計を策定
5/20	実装	フルート音色の聴感テスト完了
5/26	実装	トランペット音色合成テスト完了
5/26	修正	トランペット音切り替えノイズを解消 (0.03s フェードアウト)
5/26	実装	シリアル通信テスト (921600 bps) 完了
6 月初旬	実装	ドラム音色 (キック・シンバル) 単体テスト完了
6/17	修正	赤外線受光モジュール故障 3 台を交換・動作確認
6/19	テスト	3 子機での輪唱テスト (部分成功)
6/23	テスト	4 子機の輪唱実機テスト成功
6 月下旬	修正	tick 数不整合バグを修正 (子機 33tick → 32tick)
6 月下旬	修正	休符が聞こえない問題を修正 (即時停止処理追加)
6 月下旬	設計変更	輪唱開始方式を押下順カノンから随時参加方式へ変更
7 月初旬	実装	ピアノ音源をオルガン風から加算合成方式へ差し替え
7 月初旬	実装	テンポ管理をボタン式から可変抵抗式へ変更
7 月初旬	実装	LCD 表示プログラムの統合
7/8	計測	音色アンケート実施 (18 人, 5 段階評価)
7/8	計測	LCD 同期誤差の計測 (約 50.1 ms, 動画フレーム差分法)
7/13	計測	同期・通信の定量評価実施 (片道遅延 平均 28.08 ms, 標準偏差 0.13 ms, 10,323 tick 分のログ)
7 月中旬	テスト	最終発表で全構成のシステムテストを実施 (表示子機を含め正常動作)

子機へ同一の物理信号を同時に届ける性質により、遅延のばらつきが測定分解能に近い水準に収まった。合奏の同期で問題になるのは一律の遅れではなく機器間の相対的なずれであるという 7.1 節の整理に立てば、この性質こそが本方式の本質的な利点である。

今後の課題として、コマンド部の空き 3 ビットを用いた譜面位置の自己修復、トランペットとキックの音色改善 (倍音構造の調整)、MOP1 を複数子機の同時計測で直接検証すること、およびテンポに追従する音長制御が挙げられる。

9 変更履歴

本報告書の主要な変更履歴を表 28 に示す。

参考文献

- [1] 別所正博, 坂村健, “INIAD : IoT 時代の新しい学び舎,” 情報の科学と技術, Vol.67, No.11, pp.582–588, 2017. https://doi.org/10.18919/jkg.67.11_582
- [2] 三井一希, 佐藤和紀, 萩原丈博, 竹内慎一, 堀田龍也, “IoT ブロックを活用したプログラミング教育の試行,” 日本デジタル教科書学会発表予稿集, Vol.7, pp.27–28, 2018. https://doi.org/10.20755/jsdtp.7.0_27
- [3] 山崎貞登, “STEM, STEAM, エンジニアリング教育概念の比較教育からの論点整理,” 日本産業技術教育学会誌, Vol.62, No.3, pp.197–207, 2020. https://doi.org/10.32309/jjste.62.3_197
- [4] 森健一郎, 芳賀均, 青木隆, “学校教育における「ものづくり」を中心とした STEAM 教育実践—NHK E テレ「ツ克蘭カー」の活用—,” 日本科学教育学会研究会研究報告, Vol.37, No.3, pp.25–28, 2022. https://doi.org/10.14935/jsr.37.3_25
- [5] 木戸冬子, 平本健二, “大学におけるハッカソン実施の試み—ハッカソン研究会の活動—,” 研究・技術計画学会年次学術大会講演要旨集, Vol.30, pp.138–139, 2015. https://doi.org/10.20801/randi.30.0_138
- [6] 吉川祐樹, 川勝望, 林和彦, “全学科全学年に横断した PBL 型授業による学生の主体性の向上について,” 工学教育, Vol.69, No.2, pp.2_2–2_10, 2021. https://doi.org/10.4307/jsee.69.2_2
- [7] 小俣海斗, 今井慎一, “小学校における IoT 教材を活用した PBL 型授業に関する探索的研究,” 日本教育工学会論文誌, Vol.44, No.3, pp.305–314, 2020. <https://doi.org/10.15077/jjet.44025>
- [8] 井垣宏, 武元貴一, 上田悠貴, “基礎的な IoT 教育のためのチーム開発を重視した PBL 授業の提案,” コンピュータソフトウェア, Vol.35, No.1, pp.1_54–1_66, 2018.
- [9] 橋本武彦, “データサイエンティストの採用・育成におけるハッカソンの活用,” 日本ソーシャルデータサイエンス学会論文誌, Vol.3, No.1, pp.8–16, 2019. <https://doi.org/10.69423/jsdss.25>
- [10] 松本慎平, 中島亨輔, Azelin Mohamed Noor, “オンラインコミュニケーションツールを用いた英語によるプログラミングハッカソンの実践—広島工業大学とペトロナス工科大学との国際交流—,” コンピュータ&エデュケーション, Vol.51, pp.75–80, 2021. <https://doi.org/10.14949/konpyutariyoukyouiku.51.75>
- [11] 野口孝文, 布施泉, 千田和範, 稲守栄, “IoT 機能を持つロボットを用いた協調学習環境,” 教育システム情報学会誌, Vol.37, No.2, pp.106–119, 2020. <https://doi.org/10.14926/jsise.37.106>
- [12] 日下菜穂子, 末宗佳倫, 下村篤子, 上田信行, “プログラミング教育を介した多世代が教えあう協調学習の生涯発達における可能性,” 発達心理学研究, Vol.31, No.4, pp.201–212, 2020. <https://doi.org/10.11201/jjdp.31.201>
- [13] Rasch, R. A., “Synchronization in performed ensemble music,” *Acustica*, Vol.43, No.2, pp.121–131, 1979.
- [14] 一般社団法人音楽電子事業協会 (AMEI), “MIDI 規格委員会,” <https://amei.or.jp/midistandardcommittee/>, 参照 2026 年 6 月.

- [15] SB-Projects, “IR Remote Control Theory: NEC Protocol,” <https://www.sbprojects.net/knowledge/ir/nec.php>, 参照 2026 年 5 月.
- [16] SB-Projects, “IR Remote Control Theory: Sony SIRC,” <https://www.sbprojects.net/knowledge/ir/sirc.php>, 参照 2026 年 7 月.
- [17] Mitchell, T., Madgwick, S., Rankine, S., Hilton, G., Freed, A. and Nix, A., “Making the Most of Wi-Fi: Optimisations for Robust Wireless Live Music Performance,” Proc. International Conference on New Interfaces for Musical Expression (NIME 2014), London, UK, pp.251–256, 2014.
- [18] Rubenstein, M., Ahler, C. and Nagpal, R., “Kilobot: A Low Cost Scalable Robot System for Collective Behaviors,” Proc. 2012 IEEE International Conference on Robotics and Automation (ICRA), pp.3293–3298, 2012. <https://doi.org/10.1109/ICRA.2012.6224638>
- [19] Tosi, J., Taffoni, F., Santacatterina, M., Sannino, R. and Formica, D., “Performance Evaluation of Bluetooth Low Energy: A Systematic Review,” Sensors, Vol.17, No.12, 2898, 2017. <https://doi.org/10.3390/s17122898>
- [20] McPherson, A. P., Jack, R. H. and Moro, G., “Action-Sound Latency: Are Our Tools Fast Enough?,” Proc. International Conference on New Interfaces for Musical Expression (NIME 2016), Brisbane, Australia, pp.20–25, 2016.
- [21] Gomez, C., Oller, J. and Paradells, J., “Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology,” Sensors, Vol.12, No.9, pp.11734–11753, 2012. <https://doi.org/10.3390/s120911734>
- [22] Meyer, J., Giraud, F., Wüthrich, J., Pollefeys, M., Färnstahl, P. and Calvet, L., “RocSync: Millisecond-Accurate Temporal Synchronization for Heterogeneous Camera Systems,” Sensors, Vol.26, No.3, 1036, 2026. <https://doi.org/10.3390/s26031036>
- [23] Blanco, R., Gavilanes, L., Garcia-Ordeig, R. and Oreggioni, J., “A Configurable Infrared Communication Platform for Custom Protocol Development, Prototyping and Testing,” IEEE Embedded Systems Letters, early access, 2026. <https://doi.org/10.1109/LES.2026.3663986>
- [24] Fletcher, H., Blackham, E. D. and Stratton, R., “Quality of Piano Tones,” The Journal of the Acoustical Society of America, Vol.34, No.6, pp.749–761, 1962. <https://doi.org/10.1121/1.1918192>
- [25] Young, R. W., “Inharmonicity of Plain Wire Piano Strings,” The Journal of the Acoustical Society of America, Vol.24, No.3, pp.267–273, 1952. <https://doi.org/10.1121/1.1906888>
- [26] 西口磯春, “ピアノの音響とその物理モデル,” RIST ニュース, 一般財団法人高度情報科学技術研究機構, No.56, pp.25–35, 2014. <https://www.rist.or.jp/rnews/56/56s4.pdf>
- [27] OptoSupply International Ltd., “7.4×6.2×5.25 mm Infrared Receiver Module OSRB38C9AA データシート,” <https://akizukidenshi.com/goodsaffix/OSRB38C9AA.pdf>, 参照 2026 年 6 月.
- [28] Di Fede, D., “Minim: An audio library for Processing,” <http://code.compartmental.net/minim/>, 参照 2026 年 5 月.
- [29] Shirriff, K. (原作), Joachimsmeier, A. (保守), “IRremote Arduino Library,” <https://github.com/Arduino-IRremote/Arduino-IRremote>, 参照 2026 年 5 月.

- [30] Arduino, “Arduino Documentation,” <https://docs.arduino.cc/>, 参照 2026 年 5 月.
- [31] Friberg, A. and Sundberg, J., “Time discrimination in a monotonic, isochronous sequence,” *The Journal of the Acoustical Society of America*, Vol.98, No.5, pp.2524–2531, 1995. <https://doi.org/10.1121/1.413218>
- [32] Wessel, D. and Wright, M., “Problems and Prospects for Intimate Musical Control of Computers,” *Computer Music Journal*, Vol.26, No.3, pp.11–22, 2002. <https://doi.org/10.1162/014892602320582945>
- [33] Lakiotakis, E., Liaskos, C. and Dimitropoulos, X., “Improving networked music performance systems using application-network collaboration,” *Concurrency and Computation: Practice and Experience*, Vol.31, No.24, e4730, 2019. <https://doi.org/10.1002/cpe.4730>
- [34] Chafe, C., Gurevich, M., Leslie, G. and Tyan, S., “Effect of Time Delay on Ensemble Accuracy,” *Proc. International Symposium on Musical Acoustics (ISMA2004)*, Nara, Japan, 2004.
- [35] Bartlette, C., Headlam, D., Bocko, M. and Velikic, G., “Effect of Network Latency on Interactive Musical Performance,” *Music Perception*, Vol.24, No.1, pp.49–62, 2006. <https://doi.org/10.1525/mp.2006.24.1.49>
- [36] Arrighi, R., Alais, D. and Burr, D., “Perceptual synchrony of audiovisual streams for natural and artificial motion sequences,” *Journal of Vision*, Vol.6, No.3, Article 6, pp.260–268, 2006. <https://doi.org/10.1167/6.3.6>
- [37] 長尾翼 他, “音の遅延条件がアンサンブル演奏に与える影響に関する検討,” *日本音響学会研究発表会講演論文集*, pp.997–998, 2012 年 3 月.
- [38] 桑原圭裕, “映像と音の同期 — 「動画先行の原則」 の根拠と応用,” *映像学*, Vol.102, pp.54–74, 2019. https://doi.org/10.18917/eizogaku.102.0_54
- [39] Chafe, C., Cáceres, J.-P. and Gurevich, M., “Effect of Temporal Separation on Synchronization in Rhythmic Performance,” *Perception*, Vol.39, No.7, pp.982–992, 2010. <https://doi.org/10.1068/p6465>
- [40] 小林敬太, 佐藤賢文, 神戸英利, 三井浩康, “学生を対象とした組込みソフトウェア技術者育成のための HW/SW トレードオフ実験方式,” *工学教育*, Vol.61, No.4, pp.4_36–4_42, 2013. https://doi.org/10.4307/jsee.61.4_36

A 親機 Arduino のスケッチ

親機のスケッチを以下に示す。報告者の担当箇所は、可変抵抗によるテンポ管理を行う `handleTempoPot()` と、子機の状態表示 LED の制御である。

```
1  #define DECODE_SONY // Sony SIRCプロトコルのデコードを有効化
2  #define IR_SEND_PIN 3 // 赤外線LEDの送信ピンをD3に設定
3  #include <IRremote.hpp> // 赤外線送受信ライブラリを読み込む
4
5  const int NUM_CHILDREN = 4; // 子機の台数(ピアノ/フルート/トランペット/ドラム)
6
7  const int PIN_BTN_ORDER[NUM_CHILDREN] = {4, 5, 6, 7}; // 子機ごとの「参加/退場」トグルボタン
8  // 各子機ボタン横のLED (D4→A0, D5→A1, D6→A2, D7→A3)。
9  // ボタン押下毎に必ずトグル (参加予約/取消/退場予約/取消 のいずれでも反転)。
10 const int PIN_LED_CHILD[NUM_CHILDREN] = {A0, A1, A2, A3}; // 参加状態表示LEDのピン配列(上コメント参照)
11 const int PIN_BTN_START = 8; // 演奏開始ボタンのピン(D8)
12 const int PIN_POT_TEMPO = A5; // BPM可変抵抗器 (uxcell 10kΩ シングルターン)。回転で 60~150 BPM
13 const int PIN_BTN_RESET = 10; // リセットボタンのピン(D10)
14 const int LED_INDICATOR = 13; // 動作確認用の内蔵LED(D13)
15 const int PIN_SYNC_OUT = 9; // 子機D3への同期パルス出力 (遅延計測用)
16
17 const int QUANTIZE_TICKS = 8; // 参加予約のアライメント(0,8,16,...)
18 // 子機の TOTAL_TICKS (=楽譜1周分のIR tick数) と一致させる必要がある。
19 // 退場予約はこの倍数(子機の譜面ループ末尾)にアライメントする。
20 const int SCORE_TICKS = 32; // 子機の楽譜1周分のtick数(退場アライメントの基準)
21 const int TEMPO_MIN_BPM = 60; // テンポ下限(可変抵抗の最小側)
22 const int TEMPO_MAX_BPM = 150; // テンポ上限(可変抵抗の最大側)
23 const unsigned long POT_READ_INTERVAL_MS = 50; // 可変抵抗の読み取り間隔(50ms)
24 // Sony SIRC 12bit: address(5bit) + command(7bit)。mask は command に載せる
25
26 bool isActive[NUM_CHILDREN] = {false, false, false, false}; // 参加済みフラグ
27 long joinTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 参加予定tick (未予約は-1)
28 long leaveTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 退場予定tick (未予約は-1)
29 long joinedAtTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 実際に参加発火したtick(退場アライメント計算用)
30
31 bool isPlaying = false; // 演奏中かどうかのフラグ
32 int tempoBpm = 120; // 現在のテンポ(BPM)。初期値は120
33 long tickCount = 0; // 演奏開始からの累積tick数
34 unsigned long lastTickMs = 0; // 前回tickを打った時刻(ms)
```

```

35
36 int prevBtnOrder[NUM_CHILDREN] = {HIGH, HIGH, HIGH, HIGH}; // 参加ボタンの前回値(立
    下り検出用)
37 int prevBtnStart = HIGH; // 開始ボタンの前回値(立下り検出用)
38 int prevBtnReset = HIGH; // リセットボタンの前回値(立下り検出用)
39
40 // ポテンショメータ読み取りの平滑化と更新間引き用
41 unsigned long lastPotReadMs = 0; // 前回可変抵抗を読んだ時刻(ms)
42 int smoothedPotRaw = -1; // 平滑化後のADC生値(-1は未初期化)
43 int lastReportedBpm = -1; // 前回ログ出力したBPM(変化検出用)
44
45 void setup() { // 起動時に一度だけ実行される初期化関数
46     Serial.begin(115200); // PCとのシリアル通信を115200bpsで開始
47     IrSender.begin(IR_SEND_PIN); // 赤外線送信をD3ピンで初期化
48
49     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機分のピンを順に設定
50         pinMode(PIN_BTN_ORDER[i], INPUT_PULLUP); // 参加ボタンを内部プルアップ入力に設定
51         pinMode(PIN_LED_CHILD[i], OUTPUT); // 状態表示LEDを出力に設定
52         digitalWrite(PIN_LED_CHILD[i], LOW); // 起動時はLEDを消灯
53     }
54     pinMode(PIN_BTN_START, INPUT_PULLUP); // 開始ボタンをプルアップ入力に設定
55     pinMode(PIN_BTN_RESET, INPUT_PULLUP); // リセットボタンをプルアップ入力に設定
56     // PIN_POT_TEMPO はアナログ入力なので pinMode 不要 (pull-up は付けない)
57     pinMode(LED_INDICATOR, OUTPUT); // インジケータLEDを出力に設定
58     pinMode(PIN_SYNC_OUT, OUTPUT); // 同期パルス出力ピンを出力に設定
59     digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを初期LOWにしておく
60
61     printHelp(); // 操作方法をシリアルに表示
62 }
63
64 void loop() { // メインループ(起動後ずっと繰り返す)
65     handleSerialCommand(); // PCからのシリアルコマンドを処理
66     handleOrderButtons(); // 各子機の参加/退場ボタンを処理
67     handleResetButton(); // リセットボタンを処理
68     handleTempoPot(); // 可変抵抗によるテンポ調整を処理
69     handleStartButton(); // 開始ボタンを処理
70
71     if (isPlaying) { // 演奏中のときだけtick処理を行う
72         unsigned long now = millis(); // 現在時刻(ms)を取得
73         unsigned long interval = 60000UL / (unsigned long)tempoBpm / 2UL; // 8分音符1つ
            分の時間間隔を計算
74         if (now - lastTickMs >= interval) { // 前回tickから間隔が経過したか判定
75             lastTickMs = now; // tick時刻を更新
76             tick(); // 1tick分の処理(マスク送信等)を実行
77             tickCount++; // 累積tick数を1増やす
78         }
79     }
80

```

```

81 // 子機LEDを内部状態に同期。isActive XOR (joinTick≠-1 || leaveTick≠-1) で
82 // 「参加意思あり=点灯」となり、ボタン押下毎に確実にトグルする。
83 // 4状態の遷移は tick() で join/leave 反映後も継続する:
84 // 未参加+予約なし(OFF) → 未参加+joinTick(ON) → tickでjoin → 参加+予約なし(ON)
85 // → 参加+leaveTick(OFF) → tickでleave → 未参加+予約なし(OFF)
86 for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機のLED状態を更新
87     bool on = isActive[i] != ((joinTick[i] != -1) || (leaveTick[i] != -1)); // 参加
        意思があれば点灯(XORでトグル表現)
88     digitalWrite(PIN_LED_CHILD[i], on ? HIGH : LOW); // 計算結果に従いLEDを点灯/消灯
89 }
90 }
91
92 // 参加ボタン (子機ごと) のトグル処理 (ボタン・シリアル共有)。
93 // 未参加 → 次の区切りから参加予約
94 // 参加予約中 → その予約を取消 (参加しない)
95 // 参加中 → 次の区切りから退場予約
96 // 退場予約中 → その予約を取消 (残留)
97 void toggleChild(int i) { // 子機iの参加/退場状態をトグルする
98     if (i < 0 || i >= NUM_CHILDREN) return; // 範囲外の子機番号は無視
99     if (!isPlaying) { // 停止中は操作を受け付けない
100         Serial.println("[!]_再生中のみ操作できます"); // 再生中のみ操作可能と警告表示
101         return; // 処理を中断
102     }
103
104     long joinBoundary = ((tickCount / QUANTIZE_TICKS) + 1) * QUANTIZE_TICKS; // 次の区
        切り(8tick境界)を参加tickに計算

105
106     if (isActive[i]) { // すでに参加中の場合
107         if (leaveTick[i] != -1) { // 退場予約済みなら
108             leaveTick[i] = -1; // 退場予約を取り消す
109             Serial.print("child_"); // ログ出力(子機番号の前置き)
110             Serial.print(i); // 子機番号を出力
111             Serial.println("_leave_cancelled_(stay)"); // 退場取消(残留)を通知
112         } else { // 退場予約がなければ新規に退場予約
113             // 子機が現在のループを最後まで弾き切ってから抜けるよう、
114             // joinedAtTick から数えて次の SCORE_TICKS 境界に揃える。
115             long ticksSinceJoin = tickCount - joinedAtTick[i]; // 参加してからの経過tick数
116             long loopsCompleted = ticksSinceJoin / SCORE_TICKS; // 完了した楽譜ループ数
117             leaveTick[i] = joinedAtTick[i] + (loopsCompleted + 1) * SCORE_TICKS; // 次の
                ループ末尾を退場tickに設定
118             Serial.print("child_"); // ログ出力
119             Serial.print(i); // 子機番号を出力
120             Serial.print("_will_leave_at_tick_"); // 退場予定tickの前置き
121             Serial.print(leaveTick[i]); // 退場予定tickを出力
122             Serial.print("_end_of_loop_"); // 何ループ目の末尾かの前置き
123             Serial.print(loopsCompleted + 1); // ループ番号を出力
124             Serial.println(""); // 行を閉じる

```



```

125     }
126 } else { // 未参加の場合
127     if (joinTick[i] != -1) { // 参加予約済みなら
128         joinTick[i] = -1; // 参加予約を取り消す
129         Serial.print("child_"); // ログ出力
130         Serial.print(i); // 子機番号を出力
131         Serial.println("_join_cancelled"); // 参加取消を通知
132     } else { // 参加予約がなければ新規に参加予約
133         joinTick[i] = joinBoundary; // 次の区切りを参加tickに設定
134         Serial.print("child_"); // ログ出力
135         Serial.print(i); // 子機番号を出力
136         Serial.print("_will_join_at_tick_"); // 参加予定tickの前置き
137         Serial.println(joinTick[i]); // 参加予定tickを出力
138     }
139 }
140 }
141
142 // 参加ボタン：押すごとに 参加予約⇔退場予約 をトグルする（リアルタイム、次の区切りに
    反映）
143 void handleOrderButtons() { // 参加/退場ボタンの押下を検出する関数
144     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機のボタンを順に確認
145         int s = digitalRead(PIN_BTN_ORDER[i]); // 現在のボタン状態を読む
146         if (prevBtnOrder[i] == HIGH && s == LOW) { // HIGH→LOWの立下り(押した瞬間)を検
            出
147             toggleChild(i); // 該当子機の状態をトグル
148             delay(30); // チャタリング防止に30ms待つ
149         }
150         prevBtnOrder[i] = s; // 今回の状態を次回比較用に保存
151     }
152 }
153
154 void doReset() { // 全状態を初期化して停止する関数
155     isPlaying = false; // 再生を停止
156     tickCount = 0; // tickカウンタを0に戻す
157     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機分の状態を初期化
158         isActive[i] = false; // 参加状態を解除
159         joinTick[i] = -1; // 参加予約をクリア
160         leaveTick[i] = -1; // 退場予約をクリア
161         joinedAtTick[i] = -1; // 参加発火tickをクリア
162     }
163     digitalWrite(LED_INDICATOR, LOW); // インジケータLEDを消灯
164     Serial.println("reset"); // リセット完了をログ出力
165 }
166
167 void handleResetButton() { // リセットボタンの押下を検出する関数
168     int s = digitalRead(PIN_BTN_RESET); // ボタン状態を読む
169     if (prevBtnReset == HIGH && s == LOW) { // 立下り(押下)を検出
170         doReset(); // リセット処理を実行

```

```

171     delay(30); // チャタリング防止
172 }
173 prevBtnReset = s; // 状態を保存
174 }
175
176 // 可変抵抗 (A5) の値を 60~150 BPM に線形マップしてリアルタイム反映する。
177 // 50ms間隔で読み取り、指数移動平均で軽くノイズを抑える。
178 // 既存の tick 周期計算は毎ループ tempoBpm を参照しているため、即座に反映される。
179 void handleTempoPot() { // 可変抵抗を読んでテンポに反映する関数
180     unsigned long now = millis(); // 現在時刻を取得
181     if (now - lastPotReadMs < POT_READ_INTERVAL_MS) return; // 前回から50ms未満なら読
        ますに戻る
182     lastPotReadMs = now; // 読み取り時刻を更新
183
184     int raw = analogRead(PIN_POT_TEMPO); // 可変抵抗のADC値(0~1023)を読む
185     if (smoothedPotRaw < 0) smoothedPotRaw = raw; // 初回はそのまま初期値にする
186     else smoothedPotRaw = (smoothedPotRaw * 3 + raw) / 4; // 指数移動平均でノイズを平
        滑化
187
188     int newBpm = map(smoothedPotRaw, 0, 1023, TEMPO_MIN_BPM, TEMPO_MAX_BPM); // ADC値
        を60~150BPMへ線形変換
189     newBpm = constrain(newBpm, TEMPO_MIN_BPM, TEMPO_MAX_BPM); // 範囲外に出ないようにク
        ランプ
190     tempoBpm = newBpm; // テンポを更新(次tickから即反映)
191
192     // 微小変動でシリアルが埋まらないよう、2BPM以上変わったときだけログ
193     if (lastReportedBpm < 0 || abs(newBpm - lastReportedBpm) >= 2) { // 2BPM以上変化し
        たときだけログ
194         Serial.print("tempo="); // テンポ値の前置きを出力
195         Serial.println(tempoBpm); // 現在のテンポを出力
196         lastReportedBpm = newBpm; // 報告済みBPMを更新
197     }
198 }
199
200 // 再生開始：誰も参加していない状態から開始する (参加はボタンで都度行う)
201 void doStart() { // 演奏を開始する関数
202     if (isPlaying) { // すでに再生中なら
203         Serial.println("[!]_already_playing"); // 二重開始を警告
204         return; // 何もせず戻る
205     }
206     isPlaying = true; // 再生中フラグを立てる
207     tickCount = 0; // tickカウンタを初期化
208     lastTickMs = millis() - (60000UL / (unsigned long)tempoBpm / 2UL); // すぐ最初の
        tickが出るよう時刻を過去にずらす
209     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機の状態を初期化
210         isActive[i] = false; // 参加状態を解除
211         joinTick[i] = -1; // 参加予約をクリア

```

```

212     leaveTick[i] = -1; // 退場予約をクリア
213     joinedAtTick[i] = -1; // 参加発火tickをクリア
214 }
215 Serial.println("start"); // 開始をログ出力
216 }
217
218 void handleStartButton() { // 開始ボタンの押下を検出する関数
219     int s = digitalRead(PIN_BTN_START); // ボタン状態を読む
220     if (prevBtnStart == HIGH && s == LOW) { // 立下り(押下)を検出
221         doStart(); // 演奏開始処理を実行
222         delay(30); // チャタリング防止
223     }
224     prevBtnStart = s; // 状態を保存
225 }
226
227 void doTest() { // 通信品質計測用のテスト送信関数
228     const long TEST_COUNT = 10000; // 送信回数(1万回)
229     const int SEND_INTERVAL_MS = 30; // 送信間隔(30ms)
230     Serial.println("[TEST]_start_10000_sends_(mask=0x0F,_interval=30ms)"); // テスト開
        始をログ出力
231     for (long i = 0; i < TEST_COUNT; i++) { // 1万回のループ
232         digitalWrite(PIN_SYNC_OUT, HIGH); // 同期パルスを立てる(計測起点)
233         delayMicroseconds(50); // 50us保持
234         digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを下げる
235         IrSender.sendSony(0, 0x0F, 0); // 全子機宛(mask=0x0F)でSony信号を送信
236         if (i % 1000 == 0) { // 1000回ごとに
237             Serial.print("[TEST]_"); // 進捗の前置き
238             Serial.print(i); // 現在の送信回数
239             Serial.println("/10000"); // 総数を出力
240         }
241         delay(SEND_INTERVAL_MS); // 次の送信まで30ms待つ
242     }
243     Serial.println("[TEST]_done_10000/10000"); // テスト完了をログ出力
244 }
245
246 void handleSerialCommand() { // シリアル入力コマンドを処理する関数
247     while (Serial.available()) { // 受信バッファがある間繰り返す
248         char c = (char)Serial.read(); // 1文字読み取る
249         if (c == '\r' || c == '\n' || c == '_') continue; // 改行・空白は無視
250
251         if (c >= '0' && c <= ('0' + NUM_CHILDREN - 1)) { // '0'~'3'なら子機番号として扱
            う
252             toggleChild(c - '0'); // 対応子機をトグル
253         } else if (c == 's' || c == 'S') { // 's'なら
254             doStart(); // 演奏開始
255         } else if (c == 'r' || c == 'R') { // 'r'なら
256             doReset(); // リセット
257         } else if (c == 't' || c == 'T') { // 't'なら

```

```

258     doTest(); // テスト送信
259 } else if (c == '?') { // '?'なら
260     printHelp(); // ヘルプ表示
261 }
262 }
263 }
264
265 void printHelp() { // 操作方法をシリアルに表示する関数
266     Serial.println("===_hack_oya_kai_help_==="); // ヘルプの見出しを出力
267     Serial.println("0..3:_子機の参加/退場トグル_(D4..D7_ボタンと同じ動作)"); // 数字
        キーの説明
268     Serial.println("_____未参加→参加予約_/_参加中→退場予約_/_予約中に再度押すと取消
        "); // トグル遷移の説明
269     Serial.println("s_____:_start"); // sキーの説明
270     Serial.println("r_____:_reset_(即時の強制停止)"); // rキーの説明
271     Serial.println("?_____:_help"); // ?キーの説明
272     Serial.println("(BPMは_A5_のポテンショメータで_60-150_を連続調整)"); // テンポ調
        整方法の説明
273     Serial.print("Current_BPM="); // 現在BPMの前置き
274     Serial.println(tempoBpm); // 現在のテンポを出力
275 }
276
277 void tick() { // 1tickごとにマスクを送信する中核関数
278     digitalWrite(LED_INDICATOR, (tickCount % 2) ? HIGH : LOW); // tickごとにインジケー
        タLEDを点滅

279
280     uint8_t mask = 0; // 送信する参加楽器マスクを初期化
281     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機について
282         // 参加予約していたtickに到達したら有効化
283         if (!isActive[i] && joinTick[i] != -1 && tickCount >= joinTick[i]) { // 参加予約
            tickに到達したら
284             isActive[i] = true; // 参加状態にする
285             joinTick[i] = -1; // 参加予約をクリア
286             joinedAtTick[i] = tickCount; // 退場アライメント計算の起点
287             Serial.print("child_"); // ログ出力
288             Serial.print(i); // 子機番号を出力
289             Serial.println("_joined"); // 参加をログ出力
290         }
291         // 退場予約していたtickに到達したら無効化 (このtickからは送らない)
292         if (isActive[i] && leaveTick[i] != -1 && tickCount >= leaveTick[i]) { // 退場予
            約tickに到達したら
293             isActive[i] = false; // 参加状態を解除
294             leaveTick[i] = -1; // 退場予約をクリア
295             joinedAtTick[i] = -1; // 参加発火tickをクリア
296             Serial.print("child_"); // ログ出力
297             Serial.print(i); // 子機番号を出力
298             Serial.println("_left"); // 退場をログ出力

```

```

299     }
300     if (isActive[i]) { // 参加中の子機は
301         mask |= (uint8_t)(1 << i); // マスクの該当ビットを立てる
302     }
303 }
304
305 if (mask != 0) { // 参加中の子機が1つでもあれば
306     // 子機D3への同期パルス。IR送信直前に立てて、子機ISRのmicros()と
307     // 本ループのsendSony完了までの遅延を子機側で計測できるようにする。
308     digitalWrite(PIN_SYNC_OUT, HIGH); // 送信直前に同期パルスを立てる(遅延計測用)
309     delayMicroseconds(50); // 50us保持
310     digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを下げる
311
312     IrSender.sendSony(0, mask, 0); // 参加マスクをSony SIRCで全子機へ送信
313 }
314 }

```

B 子機 Arduino のスケッチ

ピアノ・フルート・トランペット・ドラムの4子機で共通の Arduino スケッチを以下に示す。書き込み時に childId を機体ごとに変更する。

```

1  #define DECODE_SONY // Sony SIRCプロトコルのデコードを有効化
2  #include <IRremote.hpp> // 赤外線受信ライブラリを読み込む
3
4  // hack_oya_kai.ino 用プロトコル対応版:
5  // IrSender.sendSony(0, mask, 0) を受信 (Sony SIRC 12bit, ~17ms)。
6  // mask の bit(childId) が立っているtickだけ「自分の出番」として
7  // localPos をカウントアップし、その位置の音をホストへ送信する。
8  // 親機側にループの概念がないが、メロディ機 (childId != DRUM_CHILD_ID)
9  // は楽譜の最後まで行ったら localPos を 0 に戻し、自力でループする。
10 // (ドラム機は元々楽譜を持たず鳴らし続けるだけなので、この対象外)
11
12 // 書き込む機体ごとに値を変えて Upload する: 0,1,2 = メロディ機, 3 = ドラム機。
13 int childId = 0; // 自分の担当ビット番号(機体ごとに0~3へ変更)
14 const int DRUM_CHILD_ID = 3; // ドラム機のID(3番はドラム扱い)
15 const int PIN_IR_RECV = 2; // 赤外線受光モジュールの入力ピン(D2)
16 const int PIN_SYNC_IN = 3; // 親機D9からの同期パルス入力(遅延計測用)
17 const int LED_INDICATOR = 13; // 動作確認用の内蔵LED(D13)
18
19 const int SCORE_LENGTH = 40; // 楽譜配列の要素数(音符数)
20 const String myScore[SCORE_LENGTH] = { // この子機が演奏する楽譜(音名の並び)
21     "C4", "D4", "E4", "F4", "E4", "D4", "C4", "R", // 1~8音目(冒頭ドレミファミレド)
22     "E4", "F4", "G4", "A4", "G4", "F4", "E4", "R", // 9~16音目(ミファソラソファミ)
23     "C4", "R", "C4", "R", "C4", "R", "C4", "R", // 17~24音目(ドを休符を挟んで4回)
24     "C4", "C4", "D4", "D4", "E4", "E4", "F4", "F4", // 25~32音目(各音2回、以降16分音
    符)

```

```

25  "E4", "R",  "D4", "R",  "C4", "R" , "R" , "R"  // 33~40音目(ミレドと休符で締め)
26  };
27
28  // index 24 以降は1IR=2音(16分音符相当)で発音する。
29  const int DOUBLE_START = 24;  // この位置以降は1tickで2音(16分音符)鳴らす
30
31  // 1周(ループ)に必要なtick数。
32  //   通常区間: 0 .. DOUBLE_START-1 (1tick=1音)
33  //   倍速区間: DOUBLE_START .. (1tick=2音) なので tick数は半分になる
34  // 例) DOUBLE_START=24, SCORE_LENGTH=40 → 24 + (40-24)/2 = 32
35  const int TOTAL_TICKS = DOUBLE_START + (SCORE_LENGTH - DOUBLE_START) / 2;  // 楽譜1周
    に要するtick数(=32)

36
37  // Processing への通信フォーマット: "ピッチ名,duration秒,amplitude,localPos\n"
38  const float NOTE_DURATION_DEFAULT = 0.40f;  // 通常音符の長さ(秒)
39  const float NOTE_DURATION_HALF    = 0.20f;  // 倍速区間の音符長(秒)
40  const float NOTE_AMPLITUDE        = 0.60f;  // 音量(0~1)
41
42  // ドラム機が drum.pde に送るキック合図 (drum.pde は "C2" 単独行を期待)。
43  const String DRUM_NOTE = "C2";  // ドラム機がキックとして送る合図
44
45  // 自分の出番が来た回数 (= 親機のtickCountに相当)。受信したtickごとに進む。
46  long localPos = -1;  // -1 = まだ一度も自分の出番が来ていない
47
48  // テストモード: 't' で ON。受信カウント+レイテンシCSV出力のみ行う
49  bool testMode = false;  // テストモードのON/OFF
50  long testRecvCount = 0;  // テスト中の受信回数
51  unsigned long testLatencyMin = 0xFFFFFFFF;  // 遅延の最小値(初期値は最大に)
52  unsigned long testLatencyMax = 0;  // 遅延の最大値
53  unsigned long testLatencySum = 0;  // 遅延の合計(平均算出用)
54  unsigned long lastTestRecvMs = 0;  // 最後にテスト受信した時刻
55
56  unsigned long lastReceiveMs = 0;  // 前回IRを受信した時刻(ms)
57  unsigned long lastIntervalMs = 250;  // 双子音化区間の半tick遅延に使う
58
59  // 遅延計測: 親機の同期パルス立ち上がり時刻をISRで記録
60  volatile unsigned long syncRiseUs = 0;  // 同期パルス立ち上がり時刻(ISRで更新)
61
62  void onSyncRise() {  // 同期パルス割り込み時に呼ばれるISR
63    syncRiseUs = micros();  // 立ち上がり時刻をusで記録
64  }
65
66  void sendNoteToHost(const String& pitch, float duration, float amplitude, long pos) {
    // Processingへ音符情報を送る関数
67    Serial.print(pitch);  // 音名を送信
68    Serial.print(',');  // 区切りカンマ
69    Serial.print(duration, 3);  // 音の長さを小数3桁で送信

```

```

70 Serial.print(','); // 区切りカンマ
71 Serial.print(amplitude, 3); // 音量を小数3桁で送信
72 Serial.print(','); // 区切りカンマ
73 Serial.println(pos); // localPosを送信して改行
74 }
75
76 void printReady() { // 起動完了メッセージを出す関数
77     Serial.print("hack-ko_ready_CHILD_ID="); // 準備完了とID前置き
78     Serial.print(childId); // 自分のIDを出力
79     Serial.println(childId == DRUM_CHILD_ID ? "_(DRUM)" : "_(MELODY)"); // ドラム機か
        メロディ機かを表示
80 }
81
82 void setup() { // 起動時の初期化関数
83     Serial.begin(115200); // シリアル通信を115200bpsで開始
84
85     // UNO R4 等の native USB CDC は Serial.begin() 直後はホスト未接続のことがある。
86     // 最大3秒だけCDC接続完了を待つ (タイムアウトしてもそのまま進む)。
87     unsigned long boot_t = millis(); // 起動時刻を記録
88     while (!Serial && (millis() - boot_t) < 3000) { ; } // 最大3秒だけUSB接続を待つ
89
90     IrReceiver.begin(PIN_IR_RECV); // 赤外線受信をD2で開始
91     pinMode(PIN_SYNC_IN, INPUT); // 同期パルス入力ピンを入力に設定
92     attachInterrupt(digitalPinToInterrupt(PIN_SYNC_IN), onSyncRise, RISING); // 立ち上
        がりでISRを呼ぶよう割り込み設定
93     pinMode(LED_INDICATOR, OUTPUT); // インジケータLEDを出力に設定
94
95     delay(100); // 安定待ちに100ms
96     printReady(); // 起動完了を通知
97 }
98
99 void handleChildSerial() { // PCからのコマンド(テスト切替)を処理
100     while (Serial.available()) { // 受信がある間繰り返す
101         char c = (char)Serial.read(); // 1文字読む
102         if (c == '\r' || c == '\n' || c == '_') continue; // 改行・空白は無視
103         if (c == 't' || c == 'T') { // 't'ならテストモードを切替
104             testMode = !testMode; // ON/OFFを反転
105             testRecvCount = 0; // 受信カウントをリセット
106             testLatencyMin = 0xFFFFFFFF; // 最小遅延をリセット
107             testLatencyMax = 0; // 最大遅延をリセット
108             testLatencySum = 0; // 遅延合計をリセット
109             lastTestRecvMs = 0; // 最終受信時刻をリセット
110             if (testMode) { // テストON時は
111                 Serial.println("[TEST]_receive_mode_ON_—_waiting_for_IR..."); // 待機メッ
                    セージを出力
112                 Serial.println("seq,latency_us"); // CSVヘッダを出力
113             } else { // テストOFF時は
114                 Serial.println("[TEST]_receive_mode_OFF"); // 終了メッセージを出力

```



```

115     }
116 }
117 }
118 }
119
120 void loop() { // メインループ
121     handleChildSerial(); // シリアルコマンドを処理
122
123     // テスト完了検知: 最後の受信から3秒経ったらサマリーを出力して終了
124     if (testMode && testRecvCount > 0 && (millis() - lastTestRecvMs > 3000)) { // テス
        ト中で3秒受信が途絶えたら集計
125         Serial.println("---"); // 区切り線を出力
126         Serial.print("[TEST]_received:"); // 受信数の前置き
127         Serial.print(testRecvCount); // 受信回数を出力
128         Serial.println("/10000"); // 総数を出力
129         Serial.print("[TEST]_min="); // 最小遅延の前置き
130         Serial.print(testLatencyMin); // 最小遅延を出力
131         Serial.print("us_max="); // 最大遅延の前置き
132         Serial.print(testLatencyMax); // 最大遅延を出力
133         Serial.print("us_avg="); // 平均遅延の前置き
134         Serial.print(testLatencySum / testRecvCount); // 平均遅延を計算して出力
135         Serial.println("us"); // 単位を出力
136         long lost = 10000 - testRecvCount; // 取りこぼした回数を計算
137         Serial.print("[TEST]_lost:"); // ロスト数の前置き
138         Serial.println(lost); // ロスト数を出力
139         testMode = false; // テストモードを終了
140     }
141
142     if (!IrReceiver.decode()) return; // IR信号を受信していなければ戻る
143
144     // decode直後にデータを退避し、即座にresumeする。
145     // delay()中もIR受信を継続させ、tick取りこぼしを防ぐ。
146     uint8_t protocol = IrReceiver.decodedIRData.protocol; // 受信プロトコルを退避
147     uint8_t mask = (protocol == SONY) ? IrReceiver.decodedIRData.command : 0; // Sony
        ならコマンド(マスク)を取り出す
148     IrReceiver.resume(); // 次の受信に備えて即座に再開
149
150     if (protocol == SONY) { // 受信がSony信号のときだけ処理
151         if (testMode) { // テストモード中は計測だけ行う
152             testRecvCount++; // 受信回数を1増やす
153             lastTestRecvMs = millis(); // 最終受信時刻を更新
154             unsigned long recvUs = micros(); // 受信時刻をusで取得
155             unsigned long riseUs = syncRiseUs; // ISRが記録した同期時刻を取得
156             unsigned long lat = (riseUs != 0) ? (recvUs - riseUs) : 0; // 同期からの遅延を
                計算
157             if (lat > 0) { // 遅延が計測できたら
158                 if (lat < testLatencyMin) testLatencyMin = lat; // 最小値を更新
159                 if (lat > testLatencyMax) testLatencyMax = lat; // 最大値を更新

```

```

160     testLatencySum += lat; // 合計に加算
161 }
162 Serial.print(testRecvCount); // シーケンス番号を出力
163 Serial.print(','); // 区切りカンマ
164 Serial.println(lat); // 遅延値を出力(CSV)
165 return; // テスト中は発音せず戻る
166 }
167
168 if (mask & (1 << childId)) { // 自分の担当ビットが立っていれば出番
169     unsigned long recvUs = micros(); // 受信時刻を取得
170     unsigned long riseUs = syncRiseUs; // ISRで記録された値をコピー
171     Serial.print("[RECV]_millis="); // 受信ログの前置き
172     Serial.print(millis()); // 受信時刻(ms)を出力
173     Serial.print("_child="); // 子機番号の前置き
174     Serial.print(childId); // 自分のIDを出力
175     Serial.print("_localPos="); // 楽譜位置の前置き
176     Serial.print(localPos + 1); // 次の楽譜位置を出力
177     if (riseUs != 0) { // 同期時刻が取れていれば
178         Serial.print("_latency="); // 遅延の前置き
179         Serial.print(recvUs - riseUs); // 遅延値を出力
180         Serial.print("us"); // 単位を出力
181     }
182     Serial.println(); // ログ行を閉じる
183     localPos++; // 楽譜位置を1進める
184
185     if (childId == DRUM_CHILD_ID) { // ドラム機の場合
186         Serial.println(DRUM_NOTE); // キック合図(C2)を送信
187         digitalWrite(LED_INDICATOR, HIGH); // LEDを点灯
188         delay(15); // 15ms点灯
189         digitalWrite(LED_INDICATOR, LOW); // LEDを消灯
190     } else { // メロディ機の場合
191         if (localPos >= TOTAL_TICKS) { // 楽譜末尾を超えたら
192             localPos = 0; // 先頭へ戻り自力でループ
193         }
194
195         unsigned long now = millis(); // 現在時刻を取得
196         if (lastReceiveMs != 0) { // 前回受信時刻があれば
197             unsigned long interval = now - lastReceiveMs; // tick間隔を計算
198             // 演奏停止→再開時の大きなギャップでdelayが膨張するのを防ぐ
199             if (interval >= 50 && interval <= 2000 && interval <= lastIntervalMs * 3) {
200                 // 妥当な間隔のときだけ採用
201                 lastIntervalMs = interval; // 直近のtick間隔を更新
202             }
203         }
204         lastReceiveMs = now; // 受信時刻を保存
205
206         if (localPos >= 0 && localPos < DOUBLE_START) { // 通常区間(1tick=1音)なら
207             sendNoteToHost(myScore[localPos], NOTE_DURATION_DEFAULT, NOTE_AMPLITUDE,

```

```

207         localPos); // 該当音符をProcessingへ送信
digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW); // 位置に応じLED
        を点滅
208     } else if (localPos >= DOUBLE_START) { // 倍速区間(1tick=2音)なら
209         int scoreIdx = DOUBLE_START + 2 * (int)(localPos - DOUBLE_START); // 楽譜
            配列の実インデックスを算出
210         if (scoreIdx < SCORE_LENGTH) { // 範囲内なら1音目を
211             sendNoteToHost(myScore[scoreIdx], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
                localPos); // 1音目を送信
212             digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW); // LEDを点滅
213         }
214         if (scoreIdx + 1 < SCORE_LENGTH) { // 2音目が範囲内なら
215             unsigned long halfDelay = min(lastIntervalMs / 2, 200UL); // 半tick分の
                待ち時間を計算
216             delay(halfDelay); // 半tick待つ
217             sendNoteToHost(myScore[scoreIdx + 1], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
                localPos); // 2音目を送信
218         }
219     }
220 }
221 }
222 }
223 }

```

C ピアノ音再生用 Processing プログラム

ピアノ音の合成と再生を行う Processing のプログラムを以下に示す。

```

1  import processing.serial.*; // シリアル通信ライブラリを読み込む
2  Serial port; // Arduinoと通信するシリアルポート
3  import ddf.minim.*; // 音声合成ライブラリMinimを読み込む
4  import ddf.minim.ugens.*; // Minimの音源生成ユニット群を読み込む
5  import java.util.HashMap; // キャッシュ用のHashMapを読み込む
6  Minim minim; // Minim本体のインスタンス
7  AudioOutput out; // 音声出力先
8  String currentNote = ""; // 現在表示中の音名(画面表示用)
9  int currentPos = -1; // 親機が割り当ててきた localPos (デバッグ表示用)
10
11 // ---- グランドピアノ音源 (音源部のみ差し替え。シリアル受信の仕組みは既存のまま)
    ----
12 // hack_koP2saisei_pianotest.pde と同一の合成アルゴリズム・同一パラメータ (音響検証7
    /7合格)。
13 // 方式: 既知ピッチは起動時にプリレンダしてSamplerにキャッシュ、発音はtrigger()のみ。
14 HashMap<String, Sampler> pianoCache = new HashMap<String, Sampler>(); // 生成済みピ
    アノ音のキャッシュ
15 Sampler lastSampler = null; // 直前に鳴らした音。R受信時にstop()で即打ち切る (旧
    noteOff()と同じ役割)

```

```

16 final float SAMPLE_RATE = 44100f; // サンプリング周波数(44.1kHz)
17 final float RELEASE_TAIL = 0.8f; // ノート長に加えてレンダリングする余韻(秒)
18
19 void setup() { // 起動時の初期化関数
20     size(512, 200); // 波形表示ウィンドウのサイズ
21     minim = new Minim(this); // Minimを初期化
22     out = minim.getLineOut(); // ライン出力を取得
23
24     // 楽譜(hack-koP2.ino)で使うピッチ×チ2種の長さを起動時にプリレンダリング。
25     // 未知のピッチ/長さ/音量が来ても playNote 側の安全網で都度生成される。
26     // ポートを開く前に済ませ、プリレンダ中のシリアル取りこぼしを避ける。
27     String[] knownPitches = { "C4", "D4", "E4", "F4", "G4", "A4" }; // 事前生成する音
        名の一覧
28     float[] knownDurs = { 0.40f, 0.20f }; // 事前生成する音の長さ2種
29     long t0 = millis(); // プリレンダ開始時刻を記録
30     println("ピアノ音源をプリレンダリング中..."); // 進捗メッセージを表示
31     for (String pitch : knownPitches) { // 各音名について
32         for (float dur : knownDurs) { // 各長さについて
33             String key = cacheKey(pitch, dur, 0.60f); // キャッシュキーを生成
34             pianoCache.put(key, buildSampler(pitch, dur, 0.60f)); // 音を生成してキャッ
                シュに登録
35             println("_" + key + "_OK"); // 生成完了を表示
36         }
37     }
38     println("プリレンダ完了:" + pianoCache.size() + "音_" + (millis() - t0) + "ms");
        // 生成数と所要時間を表示

39
40     // portは各自変えて。
41     port = new Serial(this, "/dev/cu.usbserial-A5069RR4", 115200); // シリアルポートを
        開く
42     port.clear(); // 受信バッファをクリア
43     port.bufferUntil('\n'); // 改行までを1メッセージとして受信
44 }
45
46 void draw() { // 毎フレームの描画関数
47     background(0); // 背景を黒で塗る
48     stroke(255); // 描画色を白に
49     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
        て
50         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
            描画
51     }
52     fill(255); // 文字色を白に
53     text("note:" + currentNote, 10, 170); // 現在の音名を表示
54     text("pos:" + currentPos, 10, 190); // 現在の楽譜位置を表示
55 }
56

```

```

57 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
58     String inString = p.readStringUntil('\n'); // 1行読み取る
59     if (inString == null) return; // 空なら戻る
60     inString = trim(inString); // 前後の空白を除去
61     if (inString.length() == 0) return; // 空行なら戻る
62     currentNote = inString; // 受信文字列を表示用に保存
63
64     // ino側のフォーマット: "ピッチ名,duration秒,amplitude,localPos"
65     // localPos は親機が決めた楽譜index。発音には使わずデバッグ表示専用。
66     // 旧3列フォーマット(pitch,duration,amplitude)も互換のため受け付ける。
67     // それ以外(起動メッセージなど)はパース不能としてスキップ。
68     String[] parts = split(inString, ','); // カンマ区切りで分解
69     if (parts.length < 3) { // 3要素未満なら音符でない
70         println("non-note:_" + inString); // 音符以外としてログ
71         return; // 戻る
72     }
73     String pitch = parts[0]; // 1列目=音名
74     float duration = float(parts[1]); // 2列目=長さ(秒)
75     float amplitude = float(parts[2]); // 3列目=音量
76     if (parts.length >= 4) { // 4列目があれば
77         currentPos = int(parts[3]); // localPosを表示用に保存
78     }
79     // R は休符: 直前の音の余韻が次のtickまで残らないよう、即stop()で打ち切る。
80     // (NOTE_DURATION_DEFAULT=0.40s は tick間隔(120BPMで0.25s)より長いので、
81     // R が来た時点で前音がまだ鳴っているのを止めないと休符に聞こえない。)
82     if (pitch.equals("R")) { // Rは休符なので
83         if (lastSampler != null) { // 直前の音が鳴っていれば
84             lastSampler.stop(); // 即座に止めて余韻を切る
85             lastSampler = null; // 参照をクリア
86         }
87         return; // 発音せず戻る
88     }
89     playNote(pitch, duration, amplitude); // 音名に対応する音を再生
90 }
91
92 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
93     数
94     String key = cacheKey(pitch, duration, amplitude); // キャッシュキーを生成
95     Sampler smp = pianoCache.get(key); // キャッシュから音を取得
96     if (smp == null) { // 安全網: 未キャッシュの組み合わせは都度生成してキャッシュ
97         smp = buildSampler(pitch, duration, amplitude); // 未生成なら新たに合成
98         pianoCache.put(key, smp); // キャッシュに登録
99     }
100     smp.trigger(); // 音を発音
101     lastSampler = smp; // 直近の音として保存
102 }
103
104 String cacheKey(String pitch, float duration, float amplitude) { // キャッシュキー文

```

```

104     字列を作る関数
105     return pitch + "_" + nf(duration, 0, 2) + "_" + nf(amplitude, 0, 2); // 音名・長
106     さ・音量を連結
107 }
108 // ---- グランドピアノ音源 (pianotestから無改変移植。amplitudeのみ引数化) -----
109 Sampler buildSampler(String pitch, float dur, float amp) { // ピアノ音を合成して
110     Samplerにする関数
111     float f0 = Frequency.ofPitch(pitch).asHz(); // 音名から基本周波数を求める
112     float[] data = renderPianoNote(f0, dur, amp); // 波形データを生成
113     MultiChannelBuffer mcb = new MultiChannelBuffer(data.length, 1); // 1chのバッファ
114     を用意
115     mcb.setChannel(0, data); // 波形データを書き込む
116     Sampler smp = new Sampler(mcb, SAMPLE_RATE, 6); // 最大6声 (余韻の重なり用)
117     smp.patch(out); // 出力に接続
118     return smp; // 生成したSamplerを返す
119 }
120 float[] renderPianoNote(float f0, float dur, float amp) { // ピアノ波形を数値計算で
121     生成する関数
122     int n = (int)((dur + RELEASE_TAIL) * SAMPLE_RATE); // 余韻を含む総サンプル数
123     float[] buf = new float[n]; // 波形バッファを確保
124
125     // 複弦構成 (実ピアノ: 高音ほど弦が多い)。デチューンはcent単位
126     float[] cents; // 各弦のデチューン量(cent)
127     if (f0 >= 350f) cents = new float[] { 0f, 1.2f, -0.8f }; // 高音域は3本弦
128     else if (f0 >= 150f) cents = new float[] { 0f, 1.0f }; // 中音域は2本弦
129     else cents = new float[] { 0f }; // 低音域は1本弦
130
131     float p = 0.9f + 0.5f * constrain((f0 - 110f) / 770f, 0f, 1f); // 倍音ロール
132     オフ
133     float B = 0.0002f * pow(f0 / 261.6f, 0.8f); // インハーモ
134     ニシティ
135     float T60 = constrain(6.0f * pow(261.6f / f0, 0.7f), 0.8f, 8.0f); // 残響長(音
136     域依存)
137     float tauBase = T60 / 6.9f; // 基準となる減衰時定数
138     float stringAmp = amp / cents.length; // 弦数で割った1本あたりの振幅
139
140     for (int s = 0; s < cents.length; s++) { // 各弦について
141         float fs = f0 * pow(2f, cents[s] / 1200f); // デチューンを反映した弦の周波数
142         for (int h = 1; h <= 10; h++) { // 第1~10倍音について
143             float fh = h * fs * sqrt(1f + B * h * h); // インハーモニシティ補正周波数
144             if (fh > SAMPLE_RATE * 0.45f) break; // ナイキスト付近を超える倍音は打ち切り
145             float ah = stringAmp / pow(h, p); // 倍音次数に応じ振幅を減衰
146             float tauH = tauBase / (1f + 0.6f * (h - 1)); // 高次倍音ほど速く減衰
147             float w = TWO_PI * fh / SAMPLE_RATE; // 1サンプルあたりの位相増分
148             for (int i = 0; i < n; i++) { // 全サンプルについて
149                 float t = i / SAMPLE_RATE; // 経過時間(秒)

```

```

144     float env = 0.3f * exp(-t / 0.12f) + 0.7f * exp(-t / tauH); // 2段階減衰
145     buf[i] += ah * env * sin(w * i); // 倍音成分を波形に加算
146 }
147 }
148 }
149
150 // ハンマーノイズ: 最初の15ms、1次LPF(fc=f0*8上限8k)、指数減衰
151 float fc = min(f0 * 8f, 8000f); // ノイズLPFのカットオフ周波数
152 float a = 1f - exp(-TWO_PI * fc / SAMPLE_RATE); // 1次LPFの係数
153 float lp = 0f; // LPFの状態変数
154 int nNoise = (int)(0.015f * SAMPLE_RATE); // ノイズを乗せる長さ(15ms分)
155 randomSeed(12345); // 打鍵音の再現性
156 for (int i = 0; i < nNoise && i < n; i++) { // ノイズ区間の各サンプルについて
157     float t = i / SAMPLE_RATE; // 経過時間
158     float x = random(-1f, 1f); // 白色ノイズを生成
159     lp += a * (x - lp); // LPFを通す
160     buf[i] += amp * 0.15f * lp * exp(-t / 0.008f); // 減衰させて打鍵音として加算
161 }
162
163 // アタック(3ms)とダンパーリリース(dur経過後  $\tau=0.15s$ )
164 for (int i = 0; i < n; i++) { // 全サンプルについて
165     float t = i / SAMPLE_RATE; // 経過時間
166     buf[i] *= (1f - exp(-t / 0.003f)); // 3msのソフトアタックを掛ける
167     if (t > dur) buf[i] *= exp(-(t - dur) / 0.15f); // 音符長経過後はダンパーで減衰
168 }
169
170 // クリップ防止
171 float peak = 0f; // 最大振幅を求める変数
172 for (int i = 0; i < n; i++) peak = max(peak, abs(buf[i])); // 波形のピークを探索
173 if (peak > 0.98f) { // ピークが0.98を超えたら
174     float g = 0.98f / peak; // 収める倍率を計算
175     for (int i = 0; i < n; i++) buf[i] *= g; // 全体を縮小してクリップ防止
176 }
177 return buf; // 完成した波形を返す
178 }

```

D フルート音再生用 Processing プログラム

フルート音の合成と再生を行う Processing のプログラムを以下に示す。

```

1 import processing.serial.*; // シリアル通信ライブラリを読み込む
2 Serial port; // シリアルポート
3 import ddf.minim.*; // 音声合成ライブラリMinim
4 import ddf.minim.ugens.*; // Minimの音源ユニット群
5 Waveform currentWaveform; // フルート用の波形テーブル
6 Minim minim; // Minim本体
7 AudioOutput out; // 音声出力先

```



```

8 String currentNote = ""; // 現在の音名(表示用)
9 int currentPos = -1; // 親機が割り当ててきた localPos (デバッグ表示用)
10
11 void setup() { // 起動時の初期化関数
12     size(512, 200); // 波形表示ウィンドウのサイズ
13     minim = new Minim(this); // Minimを初期化
14     out = minim.getLineOut(); // ライン出力を取得
15     out.setTempo(120); // 基準テンポを120に設定
16     // フルーツは基音が支配的で高次倍音が弱い。第2倍音を少しだけ加える。
17     currentWaveform = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.18f, 0.06f,
18         0.02f }); // 基音中心の倍音構成で波形を生成
19     // portは各自変えて。
20     port = new Serial(this, "/dev/cu.usbmodem64E8335D22342", 115200); // シリアルポ
21     // ートを開く
22     port.clear(); // 受信バッファをクリア
23     port.bufferUntil('\n'); // 改行までを1メッセージとして受信
24 }
25
26 void draw() { // 毎フレームの描画関数
27     background(0); // 背景を黒で塗る
28     stroke(255); // 描画色を白に
29     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
30         て
31         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
32         描画
33     }
34     fill(255); // 文字色を白に
35     text("note:_ " + currentNote, 10, 170); // 現在の音名を表示
36     text("pos:_ " + currentPos, 10, 190); // 現在の楽譜位置を表示
37 }
38
39 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
40     String inString = p.readStringUntil('\n'); // 1行読み取る
41     if (inString == null) return; // 空なら戻る
42     inString = trim(inString); // 前後の空白を除去
43     if (inString.length() == 0) return; // 空行なら戻る
44     currentNote = inString; // 表示用に保存
45
46     // ino側のフォーマット: "ピッチ名,duration秒,amplitude,localPos"
47     // localPos は親機が決めた楽譜index。発音には使わずデバッグ表示専用。
48     // 旧3列フォーマット(pitch,duration,amplitude)も互換のため受け付ける。
49     // それ以外(起動メッセージなど)はパース不能としてスキップ。
50     String[] parts = split(inString, ','); // カンマ区切りで分解
51     if (parts.length < 3) { // 3要素未満なら音符でない
52         println("non-note:_ " + inString); // 音符以外としてログ
53         return; // 戻る
54     }
55     String pitch = parts[0]; // 1列目=音名

```

```

52 float duration = float(parts[1]); // 2列目=長さ(秒)
53 float amplitude = float(parts[2]); // 3列目=音量
54 if (parts.length >= 4) { // 4列目があれば
55     currentPos = int(parts[3]); // localPosを表示用に保存
56 }
57 if (pitch.equals("R")) return; // 休符なら発音せず戻る
58 playNote(pitch, duration, amplitude); // 音を再生
59 }
60
61 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
    数
62     float freq = Frequency.ofPitch(pitch).asHz(); // 音名から周波数を求める
63     out.playNote(0, duration, new FluteInstrument(freq, amplitude, currentWaveform));
        // フルート楽器を生成して演奏
64 }
65
66 class FluteInstrument implements Instrument { // フルート音を定義する楽器クラス
67     Oscil wave; // 主音を出す発振器
68     ADSR ampEnv; // 振幅のエンベロープ
69     Oscil vib; // ビブラート用の発振器
70     Summer freqSum; // 基音とビブラートを合成する加算器
71     Constant baseFreq; // 基準周波数を出す定数源
72     float maxAmp; // 最大振幅
73
74     FluteInstrument(float frequency, float maxAmp, Waveform wf) { // コンストラクタ(音
        を組み立てる)
75         this.maxAmp = maxAmp; // 最大振幅を保存
76         freqSum = new Summer(); // 周波数加算器を生成
77         baseFreq = new Constant(frequency); // 基準周波数を設定
78         // フルートらしい穏やかなビブラート (約±5Hz、周波数の深さ)。
79         vib = new Oscil(5, frequency * 0.006f, Waves.SINE); // 約5Hzのビブラートを生成
80         baseFreq.patch(freqSum); // 基準周波数を加算器へ
81         vib.patch(freqSum); // ビブラートを加算器へ
82         // 振幅はADSR側でかけるので、ここでは 1.0 にしておく (0だと無音になる)。
83         wave = new Oscil(frequency, 1.0f, wf); // 主発振器を生成(振幅は後段で制御)
84         freqSum.patch(wave.frequency); // 合成周波数を発振器へ入力
85         // ピアノ的な瞬時アタック→減衰ではなく、ソフトに立ち上がり保持するエンベロープ。
86         // ADSR(maxAmp, attack, decay, sustainLevel, release)
87         ampEnv = new ADSR(this.maxAmp, 0.08f, 0.05f, this.maxAmp * 0.9f, 0.12f); // 柔ら
            かい立ち上りのADSRを設定
88         wave.patch(ampEnv); // 発振器をエンベロープへ接続
89     }
90
91     void noteOn(float duration) { // 発音開始時の処理
92         ampEnv.noteOn(); // エンベロープを立ち上げる
93         ampEnv.patch(out); // 出力に接続
94     }
95

```

```

96 void noteOff() { // 発音終了時の処理
97     ampEnv.noteOff(); // エンベロープを解放
98     ampEnv.unpatchAfterRelease(out); // 減衰後に出力から切断
99 }
100 }

```

E トランペット音再生用 Processing プログラム

トランペット音の合成と再生を行う Processing のプログラムを以下に示す。

```

1 import processing.serial.*; // シリアル通信ライブラリを読み込む
2 Serial port; // シリアルポート
3 import ddf.minim.*; // 音声合成ライブラリ Minim
4 import ddf.minim.ugens.*; // Minimの音源ユニット群
5
6 // 1. 変数の宣言
7 Waveform lowWave; // 低音域用の波形テーブル
8 Waveform midWave; // 中音域用の波形テーブル
9 Waveform highWave; // 高音域用の波形テーブル
10
11 Minim minim; // Minim本体
12 AudioOutput out; // 音声出力先
13 String currentNote = ""; // 現在の音名(表示用)
14
15 void setup() { // 起動時の初期化関数
16     size(512, 200); // 波形表示ウィンドウのサイズ
17     pixelDensity(1); // ピクセル密度を1に固定
18     minim = new Minim(this); // Minimを初期化
19     out = minim.getLineOut(); // ライン出力を取得
20     out.setTempo(120); // 基準テンポを120に設定
21
22
23     lowWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.72f, 0.5f, 0.3f,
24         0.1f }); // 低音域は倍音を豊かに
25     midWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.43f, 0.3f, 0.15f,
26         0.05f }); // 中音域は中程度の倍音
27     highWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.028f, 0.01f });
28     // 高音域はほぼ基音のみ
29
30
31 // シリアルポート設定
32 port = new Serial(this, "/dev/cu.usbmodem64E83364FFA82", 115200); // シリアルポ
    トを開く
33 port.clear(); // 受信バッファをクリア
34 port.bufferUntil('\n'); // 改行までを1メッセージとして受信
35 }

```

```

33 void draw() { // 毎フレームの描画関数
34     background(0); // 背景を黒で塗る
35     stroke(255); // 描画色を白に
36     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
        て
37         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
            描画
38     }
39     fill(255); // 文字色を白に
40     text("note:_" + currentNote, 10, 180); // 現在の音名を表示
41 }
42
43 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
44     String inString = p.readStringUntil('\n'); // 1行読み取る
45     if (inString == null) return; // 空なら戻る
46     inString = trim(inString); // 前後の空白を除去
47     if (inString.length() == 0) return; // 空行なら戻る
48     currentNote = inString; // 表示用に保存
49
50     String[] parts = split(inString, ','); // カンマ区切りで分解
51
52     if (parts.length < 3) { // 3要素未満なら音符でない
53         if (inString.equals("C2")) { // C2ならドラムのキック合図
54             println("🥁_ドラム音(C2)を受信"); // 受信をログ表示
55         }
56         return; // 音符でないので戻る
57     }
58
59     String pitch = parts[0]; // 1列目=音名
60     if (pitch.equals("R")) { // 休符なら
61         println("♪_[_休符]"); // 休符をログ表示
62         return; // 発音せず戻る
63     }
64
65     float duration = float(parts[1]); // 2列目=長さ(秒)
66     float amplitude = float(parts[2]); // 3列目=音量
67
68     playNote(pitch, duration, amplitude); // 音を再生
69 }
70
71 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
    数
72     try { // 不正な音名に備えて例外処理
73         float freq = Frequency.ofPitch(pitch).asHz(); // 音名から周波数を求める
74
75         // 3. 周波数に応じて3つのウェーブテーブルの値を正しく出し分け
76         Waveform selectedWave = midWave; // 既定は中音域波形
77         if (freq < 550) selectedWave = lowWave; // 550Hz未満は低音域波形

```

```

78     else if (freq > 900) selectedWave = highWave; // 900Hz超は高音域波形
79
80     out.playNote(0, duration, new TrumpetInstrument(freq, amplitude, selectedWave));
81     // トランペット楽器で演奏
82 } catch (Exception e) { // 例外がでてく
83     // 安全弁
84 }
85
86 class TrumpetInstrument implements Instrument { // トランペット音を定義する楽器クラ
87     // ス
88     Oscil wave; // 主音を出す発振器
89     Line ampEnv; // 振幅の直線エンベロープ
90     Oscil vib; // ビブラート用の発振器
91     Summer freqSum; // 基音とビブラートを合成する加算器
92     Constant baseFreq; // 基準周波数を出す定数源
93     float maxAmp; // 最大振幅
94
95     TrumpetInstrument(float frequency, float maxAmp, Waveform wf) { // コンストラクタ(
96         // 音を組み立てる)
97         this.maxAmp = maxAmp; // 最大振幅を保存
98         freqSum = new Summer(); // 周波数加算器を生成
99         baseFreq = new Constant(frequency); // 基準周波数を設定
100
101         vib = new Oscil(5.5f, 3.0f, Waves.SINE); // 5.5Hz・深さ3Hzのビブラート
102         baseFreq.patch(freqSum); // 基準周波数を加算器へ
103         vib.patch(freqSum); // ビブラートを加算器へ
104
105         wave = new Oscil(frequency, 0, wf); // 主発振器(振幅0で開始)
106         freqSum.patch(wave.frequency); // 合成周波数を発振器へ入力
107
108         ampEnv = new Line(); // 振幅制御用の直線を生成
109         ampEnv.patch(wave.amplitude); // 発振器の振幅に接続
110     }
111
112     void noteOn(float duration) { // 発音開始時の処理
113         float attackTime = 0.05f; // 立上り時間(50ms)
114         ampEnv.activate(attackTime, 0, this.maxAmp); // 0から最大振幅へ立ち上げる
115         wave.patch(out); // 発振器を出力に接続
116     }
117
118     void noteOff() { // 発音終了時の処理
119         float fadeOutTime = 0.03f; // フェードアウト時間(30ms)
120         ampEnv.activate(fadeOutTime, this.maxAmp, 0); // 最大振幅から0へ下げる
121
122         // 音が消えるのを待ってから安全に切断
123         out.setNoteOffset(fadeOutTime); // フェードアウト分だけ切断を遅らせる
124         wave.unpatch(out); // 発振器を出力から切断

```

```
123 }  
124 }
```

F ドラム音再生用 Processing プログラム

ドラム音の合成と再生を行う Processing のプログラムを以下に示す。

```
1 import ddf.minim.*; // 音声合成ライブラリ Minim  
2 import ddf.minim.ugens.*; // Minimの音源ユニット群  
3 import processing.serial.*; // シリアル通信ライブラリ  
4  
5 Minim minim; // Minim本体  
6 AudioOutput out; // 音声出力先  
7 Serial arduino; // Arduinoと通信するシリアルポート  
8  
9 // =====  
10 // シリアル設定  
11 // =====  
12  
13 final int SERIAL_PORT_INDEX = 0; // 環境に合わせて変更  
14 final int BAUD_RATE = 115200; // シリアル通信速度  
15  
16 // =====  
17 // キック  
18 // =====  
19  
20 Oscil kickOsc; // キックの基音発振器  
21 Multiplier kickAmp; // キック基音の音量制御  
22  
23 Oscil kickHarmonic; // キックの倍音発振器  
24 Multiplier kickHarmonicAmp; // キック倍音の音量制御  
25  
26 // =====  
27 // シンバル  
28 // =====  
29  
30 Noise cymNoise; // シンバルのノイズ源  
31 Multiplier cymNoiseAmp; // シンバルノイズの音量制御  
32  
33 Oscil cymMetal1; // シンバル金属倍音1  
34 Oscil cymMetal2; // シンバル金属倍音2  
35 Oscil cymMetal3; // シンバル金属倍音3  
36  
37 Multiplier cymMetalAmp1; // 金属倍音1の音量制御  
38 Multiplier cymMetalAmp2; // 金属倍音2の音量制御  
39 Multiplier cymMetalAmp3; // 金属倍音3の音量制御  
40
```

```

41 // =====
42 // 自動演奏
43 // =====
44
45 boolean autoPlay = false; // 自動演奏のON/OFF
46 int lastTriggerTime = 0; // 前回自動発音した時刻
47 int autoStep = 0; // 自動演奏のステップ位置
48
49 // =====
50 // 初期設定
51 // =====
52
53 void setup() { // 起動時の初期化関数
54     size(700, 260); // ウィンドウサイズ
55
56     minim = new Minim(this); // Minimを初期化
57     out = minim.getLineOut(Minim.MONO, 512); // モノラルのライン出力を取得
58
59     setupKick(); // キック音源を構築
60     setupCymbal(); // シンバル音源を構築
61     setupSerial(); // シリアル通信を初期化
62
63     textSize(18); // 画面文字サイズを設定
64 }
65
66 void setupKick() { // キック音源を組み立てる関数
67     kickOsc = new Oscil(48.0, 1.0, Waves.SINE); // 48Hzの正弦波で基音を作る
68     kickAmp = new Multiplier(0.0); // 音量制御(初期は無音)
69     kickOsc.patch(kickAmp).patch(out); // 発振器→音量→出力へ接続
70
71     kickHarmonic = new Oscil(96.0, 1.0, Waves.SINE); // 96Hz(倍音)の発振器
72     kickHarmonicAmp = new Multiplier(0.0); // 倍音の音量制御
73     kickHarmonic.patch(kickHarmonicAmp).patch(out); // 倍音→音量→出力へ接続
74 }
75
76 void setupCymbal() { // シンバル音源を組み立てる関数
77     cymNoise = new Noise(1.0, Noise.Tint.WHITE); // 白色ノイズを生成
78     cymNoiseAmp = new Multiplier(0.0); // ノイズの音量制御
79     cymNoise.patch(cymNoiseAmp).patch(out); // ノイズ→音量→出力へ接続
80
81     cymMetal1 = new Oscil(431.0, 1.0, Waves.SQUARE); // 431Hzの矩形波(金属倍音1)
82     cymMetal2 = new Oscil(647.0, 1.0, Waves.SQUARE); // 647Hzの矩形波(金属倍音2)
83     cymMetal3 = new Oscil(913.0, 1.0, Waves.SQUARE); // 913Hzの矩形波(金属倍音3)
84
85     cymMetalAmp1 = new Multiplier(0.0); // 金属倍音1の音量制御
86     cymMetalAmp2 = new Multiplier(0.0); // 金属倍音2の音量制御
87     cymMetalAmp3 = new Multiplier(0.0); // 金属倍音3の音量制御
88

```



```

89   cymMetal1.patch(cymMetalAmp1).patch(out); // 倍音1→音量→出力へ接続
90   cymMetal2.patch(cymMetalAmp2).patch(out); // 倍音2→音量→出力へ接続
91   cymMetal3.patch(cymMetalAmp3).patch(out); // 倍音3→音量→出力へ接続
92 }
93
94 void setupSerial() { // シリアル通信を初期化する関数
95   println("===_Serial_Ports_==="); // ポート一覧の見出しを表示
96   printArray(Serial.list()); // 利用可能なポートを列挙
97
98   final String SERIAL_PORT_NAME = "/dev/cu.usbmodem34B7DA64C6002"; // 使用するポート
      名
99
100   arduino = new Serial(this, SERIAL_PORT_NAME, BAUD_RATE); // シリアルポートを開く
101   arduino.bufferUntil('\n'); // 改行までを1メッセージとして受信
102 }
103
104 // =====
105 // 画面描画
106 // =====
107
108 void draw() { // 毎フレームの描画関数
109   background(245); // 背景を明るい灰色で塗る
110   fill(0); // 文字色を黒に
111
112   text("Keyboard_test:", 30, 40); // 操作説明を表示
113   text("1:_Kick__2:_Cymbal", 30, 70); // キー割り当てを表示
114   text("A:_Auto_play_ON/OFF", 30, 100); // 自動演奏キーの説明
115   text("Arduino_serial:_1=_Kick,_2=_Cymbal", 30, 130); // シリアル入力の説明
116   text("Auto:_" + (autoPlay ? "ON" : "OFF"), 30, 170); // 自動演奏の状態を表示
117
118   if (autoPlay && millis() - lastTriggerTime >= 500) { // 自動演奏中で500ms経過した
      ら
119     lastTriggerTime = millis(); // 発音時刻を更新
120
121     if (autoStep == 0) { // ステップ0では
122       playKickSound(); // キックを鳴らす
123     } else if (autoStep == 1) { // ステップ1では
124       playCymbalSound(); // シンバルを鳴らす
125     } else if (autoStep == 2) { // ステップ2では
126       playKickSound(); // キックを鳴らす
127     } else if (autoStep == 3) { // ステップ3では
128       playCymbalSound(); // シンバルを鳴らす
129     }
130
131     autoStep = (autoStep + 1) % 4; // ステップを進める(4拍で1周)
132   }
133 }

```

```

134
135 // =====
136 // シリアル受信
137 // =====
138
139 void serialEvent(Serial port) { // シリアル受信時に呼ばれる関数
140     String receivedText = port.readStringUntil('\n'); // 1行読み取る
141
142     if (receivedText == null) return; // 空なら戻る
143
144     receivedText = trim(receivedText); // 前後の空白を除去
145     if (receivedText.length() == 0) return; // 空行なら戻る
146
147     println("recv:_" + receivedText); // 受信内容をログ表示
148
149     try { // 数値変換に備えて例外処理
150         int note = Integer.parseInt(receivedText); // 受信文字列を整数に変換
151
152         if (note == 1) { // 1なら
153             playKickSound(); // キックを鳴らす
154         } else if (note == 2) { // 2なら
155             playCymbalSound(); // シンバルを鳴らす
156         }
157     }
158     catch (NumberFormatException e) { // 数値でなければ
159         println("invalid_data:_" + receivedText); // 不正データとしてログ
160     }
161 }
162
163 // =====
164 // キー入力
165 // =====
166
167 void keyPressed() { // キー入力時に呼ばれる関数
168     if (key == '1') { // 1キーで
169         playKickSound(); // キックを鳴らす
170     } else if (key == '2') { // 2キーで
171         playCymbalSound(); // シンバルを鳴らす
172     } else if (key == 'a' || key == 'A') { // aキーで
173         autoPlay = !autoPlay; // 自動演奏をON/OFF切替
174         lastTriggerTime = millis(); // 発音時刻を初期化
175         autoStep = 0; // ステップを先頭に戻す
176     }
177 }
178
179 // =====
180 // キック音
181 // =====

```

```

182
183 void playKickSound() { // キック音を鳴らす関数
184     new Thread(new Runnable() { // 別スレッドでエンベロープを制御
185         public void run() { // スレッドの本体
186             final int durationMs = 650; // 発音の総時間(650ms)
187             final int updateMs = 2; // 音量を更新する周期(2ms)
188             int steps = durationMs / updateMs; // 更新回数
189
190             kickAmp.setValue(0.0); // 基音の音量を0に初期化
191             kickHarmonicAmp.setValue(0.0); // 倍音の音量を0に初期化
192
193             for (int i = 0; i < steps; i++) { // 各更新ステップについて
194                 float t = i * updateMs / 1000.0; // 経過時間(秒)
195
196                 float frequency = 48.0 + 122.0 * exp(-28.0 * t); // 高い音から48Hzへ急降下す
                    // 周波数
197                 float bodyEnvelope = exp(-8.0 * t); // 胴鳴りの減衰エンベロープ
198                 float attack = constrain(t / 0.004, 0.0, 1.0); // 4msの立上りを掛ける
199
200                 float bodyVolume = 0.82 * bodyEnvelope * attack; // 基音の音量を計算
201                 float harmonicVolume = 0.025 * exp(-18.0 * t) * attack; // 倍音の音量を計算
202
203                 kickOsc.setFrequency(frequency); // 基音の周波数を更新
204                 kickAmp.setValue(bodyVolume); // 基音の音量を設定
205
206                 kickHarmonic.setFrequency(frequency * 2.0); // 倍音を基音の2倍に設定
207                 kickHarmonicAmp.setValue(harmonicVolume); // 倍音の音量を設定
208
209                 sleepMs(updateMs); // 2ms待つ次の更新へ
210             }
211
212             kickAmp.setValue(0.0); // 最後に基音を無音化
213             kickHarmonicAmp.setValue(0.0); // 倍音も無音化
214         }
215     }).start(); // スレッドを開始
216 }
217
218 // =====
219 // シンバル音
220 // =====
221
222 void playCymbalSound() { // シンバル音を鳴らす関数
223     new Thread(new Runnable() { // 別スレッドでエンベロープを制御
224         public void run() { // スレッドの本体
225             final int durationMs = 700; // 発音の総時間(700ms)
226             final int updateMs = 4; // 音量を更新する周期(4ms)
227             int steps = durationMs / updateMs; // 更新回数
228

```

```

229     cymNoiseAmp.setValue(0.0); // ノイズ音量を0に初期化
230     cymMetalAmp1.setValue(0.0); // 金属倍音1を0に初期化
231     cymMetalAmp2.setValue(0.0); // 金属倍音2を0に初期化
232     cymMetalAmp3.setValue(0.0); // 金属倍音3を0に初期化
233
234     for (int i = 0; i < steps; i++) { // 各更新ステップについて
235         float t = i / float(steps - 1); // 0~1に正規化した進行度
236
237         float fastDecay = exp(-13.0 * t); // 速い減衰成分
238         float slowDecay = exp(-4.0 * t); // 遅い減衰成分
239
240         float noiseEnvelope = 0.25 * fastDecay + 0.10 * slowDecay; // ノイズのエンベ
           ロープ
241         float metalEnvelope = 0.09 * exp(-9.0 * t); // 金属倍音のエンベロープ
242         float attack = constrain(t * 45.0, 0.0, 1.0); // 急峻な立上りを掛ける
243
244         cymNoiseAmp.setValue(noiseEnvelope * attack); // ノイズの音量を設定
245         cymMetalAmp1.setValue(metalEnvelope * attack); // 金属倍音1の音量を設定
246         cymMetalAmp2.setValue(metalEnvelope * 0.75 * attack); // 金属倍音2の音量を設
           定
247         cymMetalAmp3.setValue(metalEnvelope * 0.50 * attack); // 金属倍音3の音量を設
           定
248
249         sleepMs(updateMs); // 4ms待つて次の更新へ
250     }
251
252     cymNoiseAmp.setValue(0.0); // 最後にノイズを無音化
253     cymMetalAmp1.setValue(0.0); // 金属倍音1を無音化
254     cymMetalAmp2.setValue(0.0); // 金属倍音2を無音化
255     cymMetalAmp3.setValue(0.0); // 金属倍音3を無音化
256 }
257 }).start(); // スレッドを開始
258 }
259
260 // =====
261 // 待機処理
262 // =====
263
264 void sleepMs(int milliseconds) { // 指定ミリ秒だけ待つ関数
265     try { // 割り込み例外に備える
266         Thread.sleep(milliseconds); // スレッドを一時停止
267     }
268     catch (InterruptedException e) { // 割り込まれたら
269         Thread.currentThread().interrupt(); // 割り込み状態を復元
270     }
271 }
272

```

```

273 // =====
274 // 終了処理
275 // =====
276
277 void stop() { // プログラム終了時の後始末関数
278     if (kickAmp != null) kickAmp.setValue(0.0); // キック基音を無音化
279     if (kickHarmonicAmp != null) kickHarmonicAmp.setValue(0.0); // キック倍音を無音化
280     if (cymNoiseAmp != null) cymNoiseAmp.setValue(0.0); // シンバルノイズを無音化
281     if (cymMetalAmp1 != null) cymMetalAmp1.setValue(0.0); // 金属倍音1を無音化
282     if (cymMetalAmp2 != null) cymMetalAmp2.setValue(0.0); // 金属倍音2を無音化
283     if (cymMetalAmp3 != null) cymMetalAmp3.setValue(0.0); // 金属倍音3を無音化
284
285     if (arduino != null) arduino.stop(); // シリアルポートを閉じる
286     if (out != null) out.close(); // 音声出力を閉じる
287     if (minim != null) minim.stop(); // Minimを停止
288
289     super.stop(); // 親クラスの終了処理を呼ぶ
290 }

```

G LCD 表示子機用 Arduino プログラム

LCD 表示に必要なプログラムを以下に示す。

```

1 #define DECODE_SONY // IRremoteでSony SIRCプロトコルのデコードを有効化
2 #include <IRremote.hpp> // 赤外線受信ライブラリを読み込む
3 #include <LiquidCrystal.h> // LCD制御ライブラリを読み込む
4
5 int childId = 0; // 自分の担当ビット番号
6 const int PIN_IR_RECV = 6; // 赤外線受光モジュールの入力ピン(D6)
7
8 // LCD 1台のみ (EN=11)
9 LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // LCDのピン接続を指定して初期化
10
11 // ---- カエルのカスタム文字 ----
12 // 左右共通 (char 0-2: 上段固定, char 3-5: 下段口パク)
13
14 byte frog0[8] = { // 左カエル上段左のドット絵(char0)
15     B00000, B00011, B00100, B01101, // 上4行のドットパターン
16     B01100, B00111, B00111, B01111 // 下4行のドットパターン
17 };
18 byte frog1_open[8] = { // 左カエル下段左・口開き(char3)
19     B11111, B11110, B11100, B11110, // 上4行のドットパターン
20     B01111, B00001, B00000, B00000 // 下4行のドットパターン
21 };
22 byte frog1_close[8] = { // 左カエル下段左・口閉じ(char3)
23     B11111, B11100, B11111, B11111, // 上4行のドットパターン
24     B01111, B00001, B00000, B00000 // 下4行のドットパターン

```

```

25 };
26 byte frog2[8] = { // カエル上段中央のドット絵(char1)
27     B00000, B10001, B01010, B01010, // 上4行のドットパターン
28     B01110, B11111, B11111, B10101 // 下4行のドットパターン
29 };
30 byte frog3_open[8] = { // カエル下段中央・口開き(char4)
31     B11111, B00000, B00000, B00000, // 上4行のドットパターン
32     B00000, B11111, B00000, B00000 // 下4行のドットパターン
33 };
34 byte frog3_close[8] = { // カエル下段中央・口閉じ(char4)
35     B11111, B00000, B11111, B11111, // 上4行のドットパターン
36     B11111, B11111, B00000, B00000 // 下4行のドットパターン
37 };
38 byte frog4[8] = { // カエル上段右のドット絵(char2)
39     B00000, B11000, B00100, B10110, // 上4行のドットパターン
40     B00110, B11100, B11100, B11110 // 下4行のドットパターン
41 };
42 byte frog5_open[8] = { // カエル下段右・口開き(char5)
43     B11111, B01111, B00111, B01111, // 上4行のドットパターン
44     B11110, B10000, B00000, B00000 // 下4行のドットパターン
45 };
46 byte frog5_close[8] = { // カエル下段右・口閉じ(char5)
47     B11111, B00111, B11111, B11111, // 上4行のドットパターン
48     B11110, B10000, B00000, B00000 // 下4行のドットパターン
49 };
50
51
52 // ---- 歌詞テーブル ----
53 #define TOTAL_TICKS 32 // 歌詞の総コマ数(1周32tick)
54
55 const char* lyrics[TOTAL_TICKS] = { // tickごとに表示する歌詞テーブル
56     "KA", // 0
57     "E", // 1
58     "RU", // 2
59     "NO", // 3
60     "U", // 4
61     "TA", // 5
62     "GA", // 6
63     "_", // 7
64     "KI", // 8
65     "KO", // 9
66     "E", // 10
67     "TE", // 11
68     "KU", // 12
69     "RU", // 13
70     "YO", // 14
71     "_", // 15
72     "GU", // 16

```

```

73  "WA",    // 17
74  "GU",    // 18
75  "WA",    // 19
76  "GU",    // 20
77  "WA",    // 21
78  "GU",    // 22
79  "WA",    // 23
80  "KERO",  // 24
81  "KERO",  // 25
82  "KERO",  // 26
83  "KERO",  // 27
84  "GWA!",  // 28
85  "GWA!",  // 29
86  "GWA!",  // 30
87  "_",     // 31
88  };
89
90
91  // ---- 状態変数 ----
92  long localPos = -1; // 現在の楽譜位置(-1は未受信)
93  bool frogOpen = false; // カエルの口の開閉状態
94
95  // 歌詞エリア: 列3-12 (幅×10) 2行
96  #define LYRIC_COL_START 3 // 歌詞表示エリアの開始列
97  #define LYRIC_WIDTH 10 // 歌詞表示エリアの幅(列数)
98
99  int displayCol = 0; // 現在の歌詞書き込み列
100 int displayRow = 0; // 現在の歌詞書き込み行
101
102
103  // ---- 関数 ----
104
105  // カスタム文字3-5を口の開閉に合わせて更新し、左右カエルを再描画
106  void updateFrogs(bool open) { // 口の開閉に合わせてカエルを再描画する関数
107      if (open) { // 口を開く場合
108          lcd.createChar(3, frog1_open); // char3を口開きに登録
109          lcd.createChar(4, frog3_open); // char4を口開きに登録
110          lcd.createChar(5, frog5_open); // char5を口開きに登録
111      } else { // 口を閉じる場合
112          lcd.createChar(3, frog1_close); // char3を口閉じに登録
113          lcd.createChar(4, frog3_close); // char4を口閉じに登録
114          lcd.createChar(5, frog5_close); // char5を口閉じに登録
115      }
116      // createChar後はカーソルをホームに戻してから書き直す
117      // 左カエル (列0)
118      lcd.setCursor(0, 0); // 左カエル上段の先頭へカーソル移動
119      lcd.write(byte(0)); lcd.write(byte(1)); lcd.write(byte(2)); // 上段3文字を描画
120      lcd.setCursor(0, 1); // 左カエル下段の先頭へ移動

```

```

121 lcd.write(byte(3)); lcd.write(byte(4)); lcd.write(byte(5)); // 下段3文字(口)を描画
122 // 右カエル (列13)
123 lcd.setCursor(13, 0); // 右カエル上段の先頭へ移動
124 lcd.write(byte(0)); lcd.write(byte(1)); lcd.write(byte(2)); // 上段3文字を描画
125 lcd.setCursor(13, 1); // 右カエル下段の先頭へ移動
126 lcd.write(byte(3)); lcd.write(byte(4)); lcd.write(byte(5)); // 下段3文字(口)を描画
127 }
128
129 // 歌詞エリア (列3-12) だけをスペースで埋めてクリア
130 void clearLyricArea() { // 歌詞エリアだけを消去する関数
131     for (int row = 0; row < 2; row++) { // 上下2行について
132         lcd.setCursor(LYRIC_COL_START, row); // 歌詞エリア先頭へ移動
133         for (int c = 0; c < LYRIC_WIDTH; c++) lcd.print(' '); // 幅の分だけ空白で埋める
134     }
135     displayCol = 0; // 書き込み列を先頭に戻す
136     displayRow = 0; // 書き込み行を先頭に戻す
137 }
138
139 void showLyric(int pos) { // 指定位置の歌詞を表示する関数
140     const char* word = lyrics[pos]; // 表示する語を取得
141     int len = strlen(word); // 語の文字数を求める
142
143     if (displayCol + len > LYRIC_WIDTH) { // 現在行に収まらなければ
144         if (displayRow == 0) { // まだ上段なら
145             displayRow = 1; // 下段へ改行
146             displayCol = 0; // 列を先頭に戻す
147         } else { // すでに下段なら
148             clearLyricArea(); // エリアを消して先頭から書き直す
149         }
150     }
151
152     lcd.setCursor(LYRIC_COL_START + displayCol, displayRow); // 書き込み位置へカーソル
        移動
153     lcd.print(word); // 歌詞を表示
154     displayCol += len; // 次の書き込み列へ進める
155 }
156
157
158 void setup() { // 起動時の初期化関数
159     Serial.begin(115200); // シリアル通信を開始
160     lcd.begin(16, 2); // 16桁2行のLCDを初期化
161
162     lcd.createChar(0, frog0); // char0にカエル絵を登録
163     lcd.createChar(1, frog2); // char1にカエル絵を登録
164     lcd.createChar(2, frog4); // char2にカエル絵を登録
165     lcd.createChar(3, frog1_close); // char3を口閉じで登録
166     lcd.createChar(4, frog3_close); // char4を口閉じで登録
167     lcd.createChar(5, frog5_close); // char5を口閉じで登録

```



```

168
169 lcd.clear(); // 画面を消去
170 updateFrogs(false); // 口を閉じた状態でカエルを描画
171
172 IrReceiver.begin(PIN_IR_RECV); // 赤外線受信をD6で開始
173 Serial.println("setup_OK"); // 初期化完了をログ出力
174 }
175
176 void loop() { // メインループ
177     if (!IrReceiver.decode()) return; // IRを受信していなければ戻る
178     IrReceiver.printIRResultShort(&Serial); // 受信内容を簡易表示
179
180     if (IrReceiver.decodedIRData.protocol == SONY) { // Sony SIRC信号のときだけ処理
181         uint8_t mask = IrReceiver.decodedIRData.command; // コマンド(参加マスク)を取り出す
182
183         if (mask & (1 << childId)) { // 自分の担当ビットが立っていれば
184             localPos++; // 楽譜位置を1進める
185             if (localPos >= TOTAL_TICKS) { // 末尾を超えたら
186                 localPos = 0; // 先頭へ戻す
187                 clearLyricArea(); // 歌詞エリアを消去
188             }
189
190             showLyric((int)localPos); // 現在位置の歌詞を表示
191             Serial.println("kasiOK"); // 歌詞表示完了をログ出力
192             delay(10); // 10ms待つ
193
194             frogOpen = !frogOpen; // 口の開閉を反転
195             updateFrogs(frogOpen); // カエルを再描画(口パク)
196             Serial.println("kaoOK"); // 顔更新完了をログ出力
197             delay(10); // 10ms待つ
198         }
199     }
200
201     IrReceiver.resume(); // 次の受信に備えて再開
202 }

```